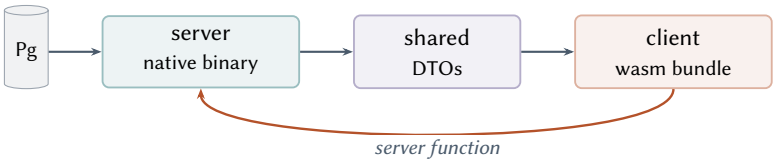


egonik-site

Architecture and Continuation Guide

Building a personal website in fullstack Rust: the patterns to adopt, the traps already in the code, and the order in which to do the remaining work.

Eduardo Gonik

Revision 1 · 4 August 2026

Written against commit `befcba5` · Leptos 0.8.20 · Actix-web 4.14 · Diesel 2.3.11 · PostgreSQL 17

This is not a description of a finished system. It is a description of a *half-built* one, written at the moment when the shape of the remaining work is still negotiable. Everything asserted here about the current code was checked by reading it, compiling it, or running it against the live database; the places where a claim rests on documentation rather than observation are marked as such.

Contents

Part I

Orientation

Chapter 1

How to read this document

1.1 What this document is for

You have written the skeleton of a personal website in Rust. The database exists, four migrations have been applied, four domain modules each have a model and a repository, and the server compiles. What does *not* yet exist is any connection between the data and the page: `src/app.rs` is still the Leptos starter template with a click counter in it.

This document exists so that the next stretch of work — the part where the data finally reaches the screen — is built on decisions made deliberately rather than by accident. That distinction matters more in this stack than in most. A Leptos application compiles the same source file twice, once for a native server and once for WebAssembly in the browser, and the consequences of that fact reach into the type of every struct you write. If you place a struct in the wrong module, or gate a dependency behind the wrong feature flag, you do not get a runtime bug you can debug later — you get a compile error whose message points somewhere unhelpful, and the natural way out of it is to work around the symptom and entrench the mistake. Several such mistakes are already latent in the code. They are catalogued here, with the reasoning that makes them obvious in hindsight.

> *The goal is not to describe the system you have. It is to give you the mental model that makes the right next move feel obvious, and the concrete list of things to fix before that move becomes harder.*

1.2 How it is organised

The document runs from concepts to consequences to a plan.

Part I — Orientation
takes inventory. What is in the repository today, what each dependency is actually responsible for, and what the toolchain does when you type `cargo leptos watch`.

Part II — The mental model
is the part to read even if you skip everything else. It explains the two-compilation model, the lifecycle of a request through server rendering and hydration, and the boundary that separates code allowed into the browser from code that must never go there. Almost every structural decision later in the document is a consequence of these three things.

Part III — Architecture
turns the model into layout: where domain logic lives, why a persistence model and a transport DTO must be two different types, why the existing `Repository<T>` trait will not survive contact with async code, and how the connection pool should reach a server function.

Part IV — Data
reviews the schema as it is against the schema it should be, and establishes migration discipline.

Three of the four existing migrations cannot be rolled back; this part shows why and how to repair them.

Part V — The front end
covers page structure, the data-loading patterns Leptos provides, and the exact configuration for Tailwind CSS, which you have said you intend to adopt.

Part VI — Quality and operations
describes the testing strategy this stack rewards, plus logging, configuration, containerisation and deployment.

Part VII — The plan
is the actionable residue: a numbered register of every defect found, a milestone-ordered roadmap, and the conventions worth committing to now, while the codebase is still small enough that consistency is cheap.

1.3 Conventions used here

Five kinds of marginal box appear throughout. They exist so you can skim for the parts you need.

> **The pattern**
>
> A pattern to adopt. If you read only these boxes you would still end up with a defensible architecture.

> **The anti-pattern**
>
> Something to avoid, usually because it is already present in the code and will cost more to remove later than now.

> **Pitfall**
>
> A trap that is not wrong exactly, but that will waste an afternoon if you meet it without warning.

> **Why this matters**
>
> The reasoning behind a recommendation. Advice you cannot justify is advice you cannot adapt when circumstances change, so the justification is given whenever it is not obvious.

> **Verified on this machine**
>
> A claim confirmed by running something on this machine, with the real output quoted. Where a box like this is absent and a factual claim is made about tool behaviour, treat it as documented-but-unverified.

Defects are numbered **D1**, **D2**, and so on. The numbers are assigned in Chapter 25 and referenced from wherever the underlying topic is discussed, so you can read the register first and jump to the explanation, or read forward and collect the numbers as you go.

Chapter 2

The state of the code today

2.1 An honest inventory

The repository holds 589 lines of Rust across 25 files, plus four migrations, a `docker-compose.yml` and the untouched remains of the Leptos Actix starter template. The useful way to read that inventory is not file-by-file but by asking, of each file, *is this load-bearing?* Table 2.1 does that.

Table 2.1. Every Rust source file, and whether it currently does anything.

File	Lines	Status	Note
`src/main.rs`	88	live	Starter template, unmodified. No pool, no server-fn wiring.
`src/lib.rs`	27	live	Module roots; every server module gated on `ssr`.
`src/app.rs`	66	live	Starter template. A counter and a 404 page.
`src/schema.rs`	103	live	Diesel-generated, matches the live database.
`src/core/mod.rs`	4	live	The `Repository<T>` trait. Read-only.
`src/database/connection.rs`	17	**orphan**	Builds a pool that nothing ever calls.
`src/portfolio/model.rs`	26	live	`PortfolioItem`, `Tag`. Public fields.
`src/portfolio/repository.rs`	44	**orphan**	Compiles; never constructed.
`src/publications/model.rs`	14	live	`PublicationItem`. Public fields.
`src/publications/repository.rs`	43	**orphan**	Compiles; never constructed.
`src/job_experience/model.rs`	26	live	**Private fields.**
`src/job_experience/repository.rs`	46	**orphan**	Compiles; never constructed.
`src/personal_information/model.rs`	27	live	**Private fields.**
`src/personal_information/repository.rs`	47	**orphan**	Compiles; never constructed.
`src/entrypoint/mod.rs`	2	live	Declares the two files below.
`src/entrypoint/http.rs`	0	**empty**	Declared and compiled; contains nothing.
`src/entrypoint/application_state.rs`	0	**empty**	Same. Created during this document's writing.

> **This table has a shelf life**
>
> The `src/entrypoint/` rows changed twice while this document was being written. As of writing, `routes/` has been deleted and replaced by `http.rs` and `application_state.rs`, both empty but both properly declared — which is a genuine improvement over the previous state, described below. Where this document discusses that directory, it discusses the shape rather than the file list. Chapter 12 is where `application_state.rs` gets an opinion.

Three words in that table deserve definitions, because the difference between them is the difference between a gap and a bug.

live
The compiler sees it and something depends on it.

orphan
The compiler sees it, it type-checks, but no code path reaches it. Orphans are normal in a half-built system — they are work in progress. The risk is that an orphan can be quietly wrong for weeks, because nothing exercises it.

dead
The compiler never sees it at all. This is worse than an orphan, because you get no type-checking, no warnings, and no signal that the file is not part of the program.

2.1.1 Dead modules, and how to detect one

Until an hour before this was written, `src/entrypoint/` contained a subtree the compiler had never seen. It is worth reconstructing, because the mechanism is invisible unless you trace module declarations by hand and it will happen again.

Rust does not discover files. A file is part of the crate only if some ancestor module declares it with `mod`. The chain broke immediately:

```
// src/lib.rs
#[cfg(feature = "ssr")]
pub mod entrypoint;            // <-- declares entrypoint, so mod.rs IS read

// src/entrypoint/mod.rs
                               // <-- EMPTY. Declares nothing.
                               //     Therefore http.rs and routes/ were never read.

// src/entrypoint/routes/job_experience/mod.rs   (unreachable anyway)
pub mod request;
pub mod response;              // <-- and note: route.rs was not declared here either
```

Listing 2.1. The declaration chain as it was, traced.

Five files, four of them empty, none of them compiled — and, tellingly, even the one file that did declare its siblings had forgotten `route.rs`. Two independent layers of the same mistake.

Verified: the compiler really does not see such files

An investigation for this document wrote the text `this is not valid rust !!!` into three of those files and ran `cargo check --features ssr`. Zero errors mentioned `entrypoint`; rustc never opened them. The files were then restored.

The silence of dead modules

An undeclared `.rs` file produces no warning of any kind. `cargo check` is clean, `clippy` is clean, and your editor will happily give you completions inside a file the compiler is ignoring. If you are ever unsure whether a file is in the build, put a deliberate syntax error in it and run `cargo check`. Silence means it is dead.

The current state is better — `http.rs` and `application_state.rs` are both declared, so both are compiled — and it has traded one smell for a milder one: two declared modules that are empty. That is fine as scaffolding and worth watching, because an empty declared module is a promise.

Chapter 12 takes a position on what `application_state.rs` should contain, since a state struct at the composition root is a good idea rather than a leftover.

2.2 What the database looks like

The database is further along than the Rust. A PostgreSQL 17 container is running, and all four migrations plus Diesel's bookkeeping migration have been applied to a database named `my-site`.

Verified on this machine

```
$ diesel migration list
Migrations:
  [X] 00000000000000_diesel_initial_setup
  [X] 2026-08-02-211314-0000_create_portfolio_item
  [X] 2026-08-03-231439-0000_create_publications_item
  [X] 2026-08-03-232042-0000_create_job_experience
  [X] 2026-08-03-233332-0000_create_personal_information
```

```
$ psql -c 'select count(*) from portfolio_items'   ->  0
  ... and 0 rows in every other table.
```

Seven tables exist and every one of them is empty. That is worth knowing before you write the first page, because a page that renders an empty list looks identical to a page whose query is broken. Chapter 17 covers seeding, and it should happen *before* the first component, not after.

Two databases, one of them a decoy

`docker-compose.yml` creates a database called `mydatabase`. Your `.env` points `DATABASE_URL` at a database called `my-site`. Both exist on the server — `my-site` because `diesel setup` created it, `mydatabase` because Compose did. `mydatabase` is empty and unused. Anyone who clones this repository, runs `docker compose up`, and trusts the Compose file will connect to the wrong, empty database and see nothing. This is defect **D14**.

2.3 What is missing, stated plainly

It is easy to lose the shape of the remaining work in a list of small defects, so here is the shape. Four things are absent, and they are absent in a specific order of dependency:

1. **No transport.** There is no server function, no route, and no HTTP endpoint that returns domain data. The repositories can read from Postgres, and nothing can call the repositories.
2. **No shared types.** Even if a transport existed, the current models cannot cross it. They are compiled only in the server build, and they derive `Serialize` from a dependency that only the server build enables. Chapter 6 works through exactly what happens.
3. **No presentation.** `src/app.rs` has one route and a counter. There are no pages for portfolio, publications, experience or contact, no layout, no navigation.
4. **No writes.** `Repository<T>` offers `get_all` and `get_by_id`. There is no insert, update or delete anywhere in the codebase, and no path by which you could author content other than by hand-writing SQL.

Everything else — Tailwind, tests, CI, deployment, tracing — is genuinely optional until those four are done. Part VII sequences them.

Chapter 3

The stack, and who is responsible for what

Six moving parts sit between a browser request and a row in Postgres. Confusion about which one owns a given responsibility is the most common source of misplaced code in this stack, so this chapter draws the boundaries once.

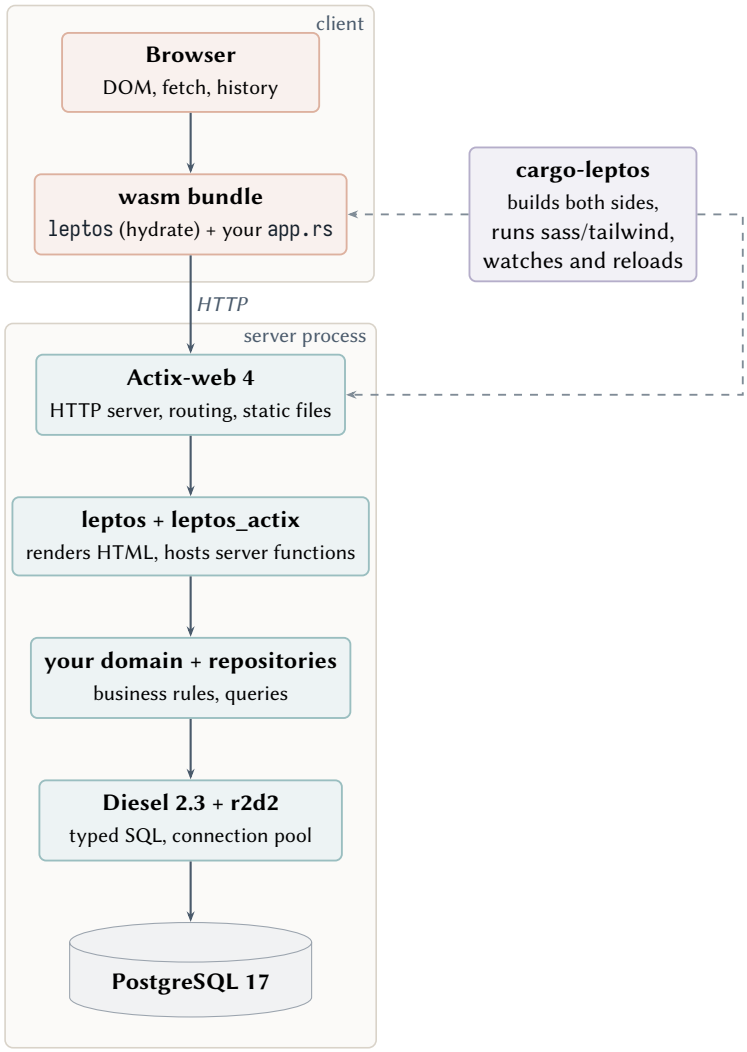


Figure 3.1. The stack, with each component's single responsibility. Solid arrows are runtime data flow; the dashed arrow is a build-time relationship.

3.1 The pieces, one at a time

Leptos 0.8 — the reactive UI framework

Leptos is the only piece that exists on both sides of the network. You write components once, as Rust functions returning `impl IntoView`, and Leptos either renders them to an HTML string (on the server) or mounts them onto existing DOM and makes them interactive (in the browser). Which of those two things it does is chosen by a Cargo feature, not at runtime. That single fact is the source of most of Part II.

Leptos also owns two things people often expect Actix to own. *Routing of application pages* is a Leptos concern — `<Routes>` in `src/app.rs`, not `App::route` in `src/main.rs`. And *the client–server call protocol* is a Leptos concern: the `#[server]` macro generates the endpoint, the serialisation, and the client-side stub for you. You do not write controllers.

Actix-web 4 — the HTTP server

Actix's job in this application is narrow, and keeping it narrow is a design goal. It listens on a socket, serves static files out of `target/site`, hands unmatched requests to Leptos, and holds shared application state (the connection pool) so that server functions can extract it. It should not contain business logic and it should not contain page routes. If you find yourself writing an Actix handler that returns JSON for your own front end to consume, stop — that is what a server function is for, and it will give you type safety across the wire that a hand-written JSON endpoint cannot.

cargo-leptos — the build orchestrator

This is the piece most worth understanding concretely, because it is doing more than it appears to. Version 0.3.6 is installed. When you run `cargo leptos watch` it:

1. compiles the library target to `wasm32-unknown-unknown` with the `hydrate` feature, then runs `wasm-bindgen` over the result;
2. compiles the binary target natively with the `ssr` feature;
3. compiles `style/main.scss` using a `sass` binary it downloaded itself;
4. copies `assets/` into `target/site/`;
5. starts the server binary, and restarts or reloads on change.

Point three surprises people. `sass` is *not* installed on this machine, yet the build works, because cargo-leptos fetches and caches its own toolchain.

> **Verified on this machine**
>
> ```
> $ command -v sass  ->  (not found)
> $ ls ~/.cache/cargo-leptos/
>   sass-1.86.0/           tailwindcss-v4.2.1/
>   wasm-bindgen-0.2.125/  wasm-bindgen-0.2.126/
> ```

Note the second entry. A Tailwind 4 CLI has *already* been downloaded into that cache. This matters for Chapter 20: adopting Tailwind here requires no `npm`, no `package.json`, and no Node dependency in your build — cargo-leptos will manage the binary the same way it manages `sass`.

Diesel 2.3 with r2d2 — typed SQL and connection pooling

Diesel gives you compile-time-checked queries against the schema in `src/schema.rs`. It is worth being precise about one property that shapes the whole server design: **Diesel, as configured**

here, is synchronous. Every query blocks the thread that issues it until Postgres answers. Actix and Leptos server functions are asynchronous. Reconciling those two facts is not optional and it is not a micro-optimisation — Chapter 10 explains why a direct call will damage your server under concurrency, and gives the two-line wrapper that fixes it.

PostgreSQL 17 — and `diesel-cli`

The database runs in Docker on port 5439. Migrations are plain SQL pairs under `migrations/`, applied by `diesel migration run` and reverted by `diesel migration revert`. Chapter 16 is about the discipline that keeps that second command working, because right now it does not.

Tailwind CSS — not yet present

Currently `style/main.scss` is three lines of Sass. Chapter 20 covers the switch. The short version: Tailwind replaces the Sass file rather than joining it, and the change is four lines of `Cargo.toml` plus one new CSS file.

3.2 Where new code goes

To close the chapter, the practical form of the above. When you are about to write something and are unsure where it belongs, this table answers it.

Table 3.1. A decision table for placing new code.

You are writing…	It belongs in…
a page or a visual component	`src/ui/` (new), reached from `src/app.rs`
a URL that a human types or bookmarks	`<Routes>` in `src/app.rs`
a call from browser to server	`src/<domain>/api.rs` (new), *ungated*
a type that travels over the wire	`src/<domain>/dto.rs` (new), *ungated*
a SQL query	`src/<domain>/repository.rs`, gated on `ssr`
a business rule or invariant	`src/<domain>/` beside the model, gated on `ssr`
an error the UI has to display	`src/error.rs` (new), *ungated*
process startup and wiring	`src/main.rs`, once, before the server starts
an Actix route	almost certainly nothing — reconsider; use a server function

The word *ungated* in that table is doing a lot of work, and the reason is the subject of Chapter 6. The new files it names are laid out in full in Chapter 8.

Part II

The mental model

Chapter 4

One source tree, two programs

4.1 The fact everything else follows from

> *`src/lib.rs` is compiled twice, into two different programs, for two different machines. Every module, every struct and every dependency is either in both programs, or in exactly one of them, and it is your job — not the compiler's — to say which.*

This is the single idea that, once internalised, makes Leptos stop feeling arbitrary. Read `Cargo.toml` again with it in mind:

```
[lib]
crate-type = ["cdylib", "rlib"]   # cdylib -> wasm target; rlib -> so main.rs can `use` it

[features]
ssr     = ["dep:actix-web", "dep:actix-files", "dep:leptos_actix", "dep:diesel",
           "dep:serde", "dep:serde_json",
           "leptos/ssr", "leptos_meta/ssr", "leptos_router/ssr"]
hydrate = ["leptos/hydrate"]
csr     = ["leptos/csr"]
```

Listing 4.1. The features that select which program is being built.

`ssr` and `hydrate` are not two configurations of one program. They are the names of the two programs. Figure 4.1 shows what `cargo leptos build` actually does with them.

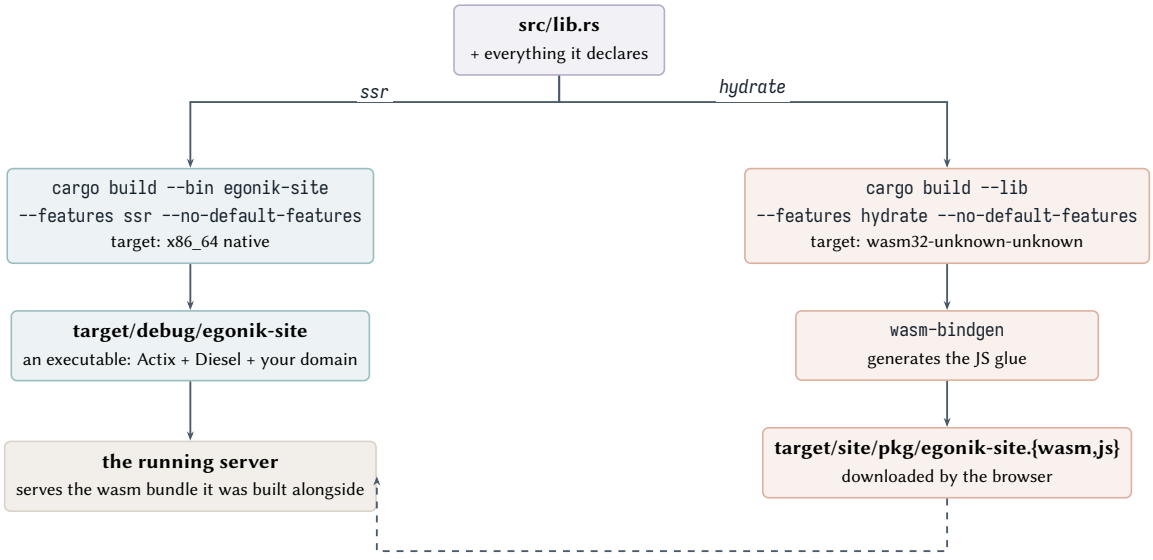


Figure 4.1. What `cargo leptos build` produces. The same `src/lib.rs` is read by both compilations; the feature flag decides which parts of it exist.

Two consequences are worth stating explicitly, because both bite in practice.

First: a dependency you add is, by default, in both programs. It is not enough to only *use* a crate on the server; if it is listed unconditionally under `[dependencies]`, it is compiled into the

browser bundle too. That is measurable in the current code.

Verified: what is actually in the wasm bundle today

```
$ cargo tree --target wasm32-unknown-unknown --no-default-features \
             --features hydrate --depth 1
egonik-site v0.1.0
|-- anyhow v1.0.104
|-- chrono v0.4.45
|-- console_error_panic_hook v0.1.7
|-- dotenvy v0.15.7          <-- reads .env files. In a browser.
|-- leptos v0.8.20
|-- leptos_meta v0.8.6
|-- leptos_router v0.8.15
`-- wasm-bindgen v0.2.126
```

`dotenvy` in a WebAssembly bundle is not merely wasteful, it is nonsensical: it exists to read a file from a filesystem the browser does not have. `anyhow` is server-side error plumbing. `chrono` is genuinely needed on both sides if dates are rendered — but only its serialisation and formatting parts, not its system-clock parts. These are defects **D9** and **D10**.

Second: `#[cfg(feature = "ssr")]` is a scalpel, and it is currently being used as a tourniquet. Look at what `src/lib.rs` does:

```
#[cfg(feature = "ssr")] pub mod core;
#[cfg(feature = "ssr")] pub mod database;
#[cfg(feature = "ssr")] pub mod job_experience;
#[cfg(feature = "ssr")] pub mod personal_information;
#[cfg(feature = "ssr")] pub mod portfolio;
#[cfg(feature = "ssr")] pub mod publications;
#[cfg(feature = "ssr")] pub mod schema;
```

Gating `database` and `schema` is correct and necessary — Diesel must never reach the browser. But gating the entire `portfolio`, `publications`, `job_experience` and `personal_information` modules means the *types describing your content* exist only on the server. Chapter 6 shows why that becomes a wall the moment you write your first server function, and Chapter 9 gives the split that removes it.

Why this gate was added, and why that was reasonable

> The gate is almost certainly there because without it the wasm build failed. It *would* have: the models derive `Serialize`, `Queryable` and `Selectable`, and none of those derive macros are available in the `hydrate` build. Adding the gate made the error go away. The problem is that it made the error go away by removing the types from the client entirely, which is the opposite of what you will need. The right fix separates the Diesel-flavoured struct from the wire-flavoured struct, rather than hiding both.

4.2 Reading a `cfg` error correctly

When either build breaks, the first diagnostic question is always *which of the two programs is failing?* `cargo leptos` interleaves both, so the output can be confusing. Check them independently:

```
# program 1: the server
cargo check --no-default-features --features ssr

# program 2: the browser
cargo check --lib --no-default-features --features hydrate \
            --target wasm32-unknown-unknown
```

Verified: both programs compile today

Both commands complete successfully against the current tree — the server in 18.5 s, the wasm library in 59.3 s from a warm cache. Keep it that way; make these two commands the definition of "it builds".

Make the two-build check a habit

Before every commit, run both commands. A change that satisfies `cargo leptos watch` can still have broken the other target, because `watch` may not have rebuilt it yet. This is also the single most valuable thing to put in CI — see Chapter 22.

4.3 A note on the `csr` feature

`Cargo.toml` also defines `csr`, and `src/main.rs` carries two extra `main` functions to support it. Client-side rendering means shipping an empty HTML shell and letting the wasm bundle build the whole page — no server rendering at all. It is useful for testing components under `trunk serve`, and useless for a personal website, where server rendering is the entire point (search engines, first-paint speed, working without JavaScript).

You have three configurations to keep straight and only two that you use. Deleting `csr` from `Cargo.toml` and removing the two dead `main` functions from `src/main.rs` costs ten minutes and permanently shrinks the space of states you have to reason about. This is recorded as defect **D20**, at low severity, because it is housekeeping rather than a fault.

Chapter 5

The life of a request

Understanding *when* your code runs is what allows you to place it correctly. A Leptos component function may execute on the server, in the browser, or both, and it will do different things in each case. This chapter follows four distinct journeys through the system.

5.1 Journey 1: a cold page load

Someone types your URL. Nothing is cached. Figure 5.1 traces what happens.

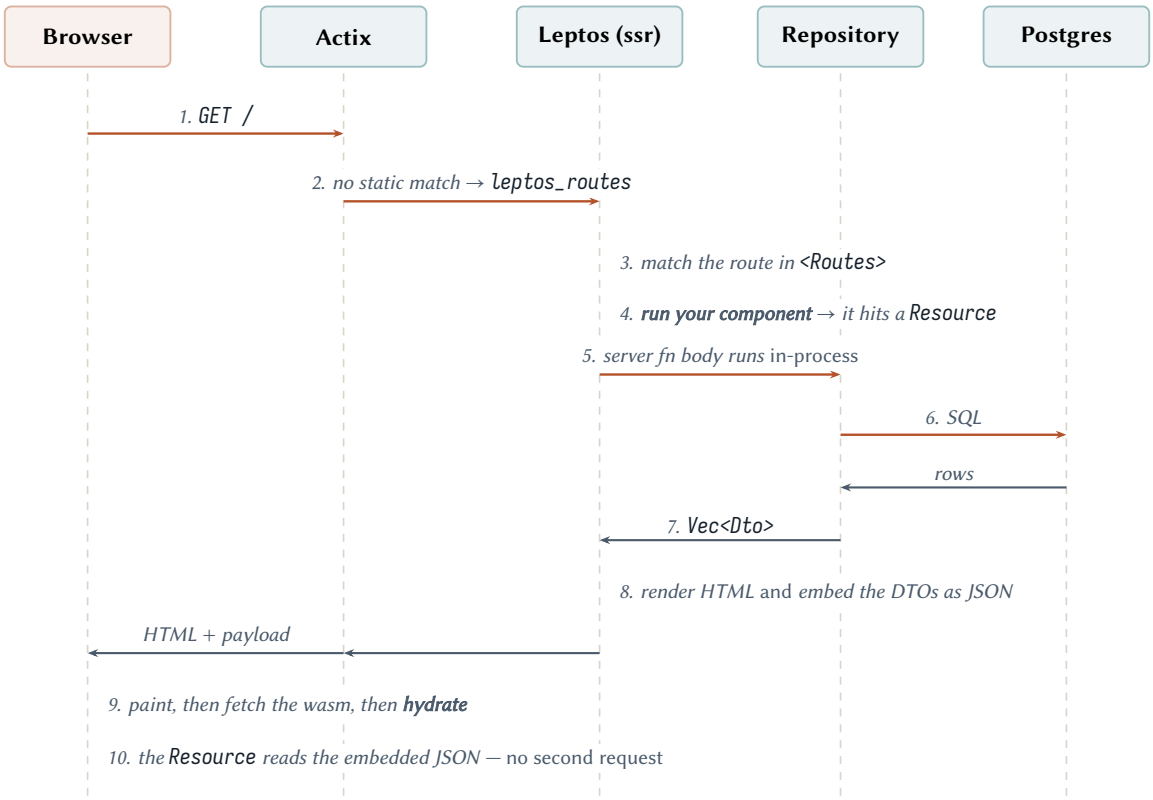


Figure 5.1. A cold page load. Note that your component function runs *twice* — once on the server at step 4, once in the browser at step 9 — and that the data fetched at step 5 is serialised into the HTML so the browser does not have to fetch it again.

Three details in that figure carry real weight.

At step 5, the server function is not an HTTP call. When a `#[server]` function is invoked during server rendering, Leptos calls the function body directly, in the same process, on the same request. There is no localhost round trip. This is why a server function is cheap during SSR and why it is safe to have several of them on one page.

At step 8, the data is serialised into the HTML. This is the mechanism that makes hydration fast, and it is also the reason your DTOs must implement `Serialize` on the server *and* `Deserialize` in the browser. Not one or the other — both, in the same type, in both builds. Chapter 6 is entirely about this constraint.

At step 9, the page is already visible before the wasm arrives. Server-rendered HTML is real HTML. If the wasm bundle fails to download, the visitor still sees your content; they just cannot interact with anything that requires reactivity. For a personal website that is a very good failure mode, and it is worth preserving deliberately by keeping content in server-rendered markup rather than fetching it client-side after load.

5.2 Journey 2: navigating to a second page

The visitor clicks a link to `/portfolio`. Now the wasm bundle is loaded and Leptos is in control.

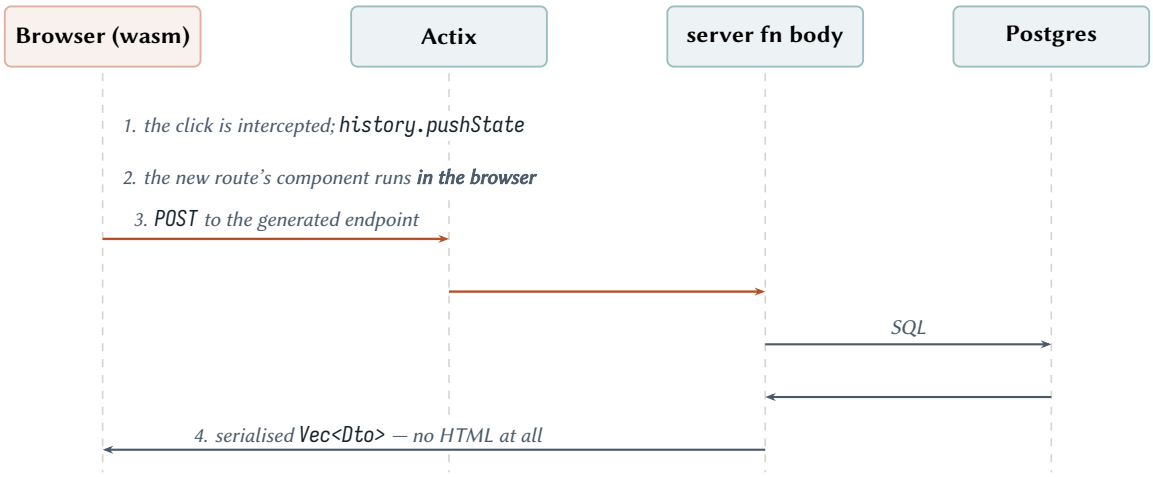


Figure 5.2. Client-side navigation. Actix serves no HTML at all; only the server function is called, and only the data crosses the network.

The same component function you wrote for journey 1 now runs in WebAssembly, and the same `#[server]` function you called now performs an HTTP request instead of a direct call. You did not write two versions of anything. That is the payoff of the two-build model, and it is why the discipline it demands is worth accepting.

5.3 Journey 3: no JavaScript at all

If the visitor has JavaScript disabled, or the bundle 404s, journey 1 still completes through step 8. They get a fully rendered page. Forms submitted through Leptos's `<ActionForm>` even continue to work, because they degrade to ordinary HTML form posts.

> **Design for the no-wasm case first**
>
> Render content on the server. Use reactivity for interaction, not for content delivery. A personal website whose text only appears after 400 KB of WebAssembly has loaded is strictly worse than one whose text is in the HTML — for readers, for search engines, and for you when you are debugging.

5.4 Journey 4: a write

You do not have writes yet, so this is the shape to aim for rather than a description of existing behaviour. A write is a server function that mutates, wrapped in an `Action` rather than a `Resource`:

- a `Resource` is for *reading*: it runs automatically when its inputs change, and its result is serialised into SSR output;

- an `Action` is for *writing*: it runs when you tell it to, exactly once per dispatch, and never runs during server rendering.

Getting this pairing wrong is a classic Leptos error — a mutation inside a `Resource` will fire during server rendering and again on hydration, and you will insert two rows. Keep the rule: **reads are Resources, writes are Actions.**

5.5 Where the current code sits in these journeys

Nowhere, yet, and that is the point of the chapter. `src/app.rs` has a `RwSignal` counter, which participates only in journey 2, and only in the browser. There is no `Resource`, so no data is fetched at step 5; no server function, so nothing is registered at step 3 of journey 2; and no `provide_context`, so nothing the repositories need would be reachable if there were. Chapter 12 wires it, and Chapter 19 writes the first `Resource`.

Chapter 6

The boundary: what may enter the browser

6.1 The rule

> *A type that crosses between server and browser must compile in* both *programs, must derive* `Serialize` *and* `Deserialize` *in* both *programs, and must not transitively depend on anything server-only. A type that does not cross must be unavailable to the browser* on purpose.

Figure 6.1 draws the boundary. The left column is server-only, the right is shared, and the seam between them is where DTOs live.

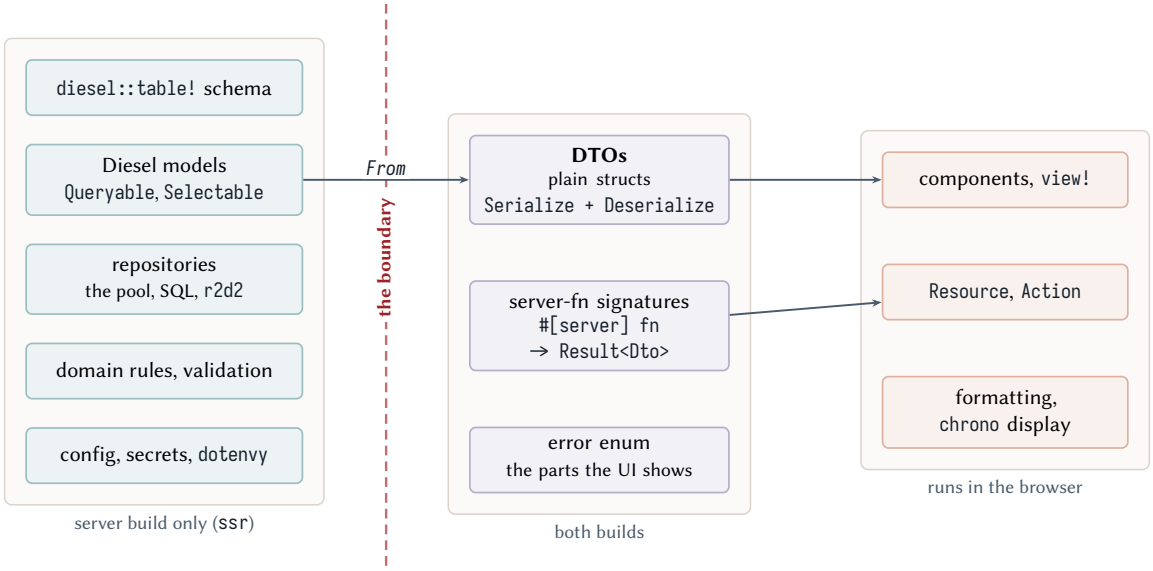


Figure 6.1. The boundary. Everything on the left is invisible to the browser by construction; everything in the middle column is compiled into both programs and must therefore be free of server-only dependencies.

6.2 The failure you are going to hit

This is not hypothetical, so it is worth showing exactly. Suppose you decide to return your existing model straight from a server function — the obvious first attempt:

```
// the natural thing to write, and it cannot work
#[server]
pub async fn list_publications() -> Result<Vec<PublicationItem>, ServerFnError> { ... }
```

Two things break at once.

The type does not exist in the browser build. `PublicationItem` lives in `src/publications/model.rs`, and `src/lib.rs` gates that module on `ssr`. The `hydrate` build has never heard of it, and the `#[server]`

macro needs the signature to compile on *both* sides — it generates the client stub from the same declaration.

And `serde` is not available in the browser build either. `serde` is declared `optional = true` and enabled only by the `ssr` feature. Remove the module gate and you trade one error for another:

Verified: the exact error, reproduced

A struct deriving `serde::Serialize` was added to the crate root without a `cfg` gate, and the client program was compiled:

```
$ cargo check --lib --no-default-features --features hydrate \
              --target wasm32-unknown-unknown
error[E0432]: unresolved import `serde`
error: could not compile `egonik-site` (lib) due to 1 previous error
```

The file was then removed and the tree restored. This is defect D6.

The confusing part

`serde` *is* physically compiled into the wasm build — Leptos depends on it. But because your crate declares it as an optional dependency that only `ssr` enables, *your* code is not permitted to name it. The crate is present and the path is unresolvable. That is why the error message is `unresolved import` rather than something about a missing dependency, and it is why the fix is in `Cargo.toml`, not in the `use` statement.

6.3 The fix, in two lines of `Cargo.toml`

```toml
[dependencies]
serde = { version = "1", features = ["derive"] }         # both programs need it
serde_json = { version = "1", optional = true }          # server only, unless you need it client-side
chrono = { version = "0.4", features = ["serde"], default-features = false }
diesel = { version = "2.3", optional = true, features = [...] }
dotenvy = { version = "0.15", optional = true }           # <-- make optional
anyhow  = { version = "1",    optional = true }           # <-- make optional

[features]
ssr = ["dep:actix-web", "dep:actix-files", "dep:leptos_actix", "dep:diesel",
       "dep:serde_json", "dep:dotenvy", "dep:anyhow",
       "leptos/ssr", "leptos_meta/ssr", "leptos_router/ssr"]
```

Listing 6.1. `serde` becomes unconditional; the server-only crates stay optional.

`serde` is a small crate and is already in the bundle; making it unconditional costs nothing and removes an entire category of error. `dotenvy` and `anyhow` move the other way — into `ssr` — which is defect D9's fix and shrinks the wasm bundle.

Note the `chrono` line. Dates *do* need to cross the boundary — a publication year, an employment period — so `chrono` must stay unconditional, but with `default-features = false` it drops the parts that want a system clock and a timezone database, which is most of its weight. Keep `features = ["serde"]` so `NaiveDate` serialises.

6.4 The checklist

Before adding any type or dependency, ask in order:

1. **Does this need to be visible to a component?** If no, gate it on `ssr` and stop. Most things stop here: queries, the pool, migrations, secrets.
2. **Does it need to travel over the wire?** If yes, it is a DTO. It goes in the ungated `src/dto/` module, derives `Serialize + Deserialize`, and contains *only* primitives, `String`, `Option`, `Vec` and other DTOs — plus `NaiveDate`, which is fine.
3. **Does it pull in a crate?** Check the crate compiles for `wasm32-unknown-unknown` and does not want the filesystem, threads, or a socket. When in doubt, run the two-build check from Chapter 4.

Reaching for `cfg` to silence an error

When the wasm build complains about a type, the tempting fix is `#[cfg(feature = "ssr")]`. Sometimes that is right — but if the type is something a page needs to display, the `cfg` is postponing the problem, not solving it. The question to ask is not "how do I hide this from the client" but "does the client need this shape, and if so what should that shape be?" The answer to the second question is a DTO, which is Chapter 9.

6.5 A per-dependency verdict for this `Cargo.toml`

The checklist above is general. Here it is applied to every dependency currently declared, because four of the eleven are on the wrong side and two are not used at all.

Table 6.1. Every declared dependency, and where it belongs.

Crate	Declared	Should be	Why
`leptos`, `leptos_meta`, `leptos_router`	both	both	the framework; each has its own `ssr` feature
`actix-web`, `actix-files`, `leptos_actix`	ssr	ssr	correct
`diesel`	ssr	ssr	correct, and mandatory: the `postgres` feature links native `libpq`
`serde`	**ssr**	**both**	**D6**. Wire types must serialise in the browser
`serde_json`	ssr	delete	used nowhere in `src/`
`http`	optional, *no feature*	delete	enabled by nothing, used by nothing
`chrono`	both	both, trimmed	**D10**. Dates do cross; add `default-features = false`
`dotenvy`	both	ssr	**D9**. Reads `.env` from a filesystem the browser has not got
`anyhow`	both	ssr	**D9**. Server-side error plumbing
`wasm-bindgen`	both	`hydrate`	needed only by `pub fn hydrate()`; currently also compiled natively
`console_error_panic_hook`	both	`hydrate`	browser-only — see the next chapter for why it matters

Verified: two of these are entirely dead

```
$ grep -n 'http' Cargo.toml
13:http = { version = "1.3.1", optional = true }     <-- in no feature list
$ grep -rn 'http::' src/
src/app.rs:60: resp.set_status(actix_web::http::StatusCode::NOT_FOUND);   <-- actix's, not this one
$ grep -rn 'serde_json' src/
  (no output)
```

Both declarations can go. `http` in particular is invisible: because no feature enables it, it is never compiled, so it produces no warning and no build cost — just a line that implies a dependency you do not have. These are defects **D34** and **D35**.

CHAPTER 7

The run contract, and how to tell hydration happened

7.1 Why the binary behaves differently from `cargo leptos watch`

> *The compiled server binary is not self-contained. It reads its own configuration from `LEPTOS_*` environment variables at startup, and `cargo leptos` sets them for you. Run the binary yourself without them and you get a site that renders perfectly and never becomes interactive.*

This is the single most confusing failure available in this stack, because every visible signal says the application is fine. It is worth showing exactly, since you will meet it the first time you run the binary directly or put it in a container.

Verified: running `target/debug/egonik-site` with no `LEPTOS_*` variables

The binary prints its own warning, and then serves a page:

```
$ env -u LEPTOS_OUTPUT_NAME -u LEPTOS_SITE_ROOT ./target/debug/egonik-site
It looks like you're trying to compile Leptos without the LEPTOS_OUTPUT_NAME environment
variable being set. There are two options
 1. cargo-leptos is not being used, but get_configuration() is being passed None. This
    needs to be changed to Some("Cargo.toml")
 2. You are compiling Leptos without LEPTOS_OUTPUT_NAME being set with cargo-leptos.
    This shouldn't be possible!
listening on http://127.0.0.1:3000
```

And here is what that page contains:

```
$ curl -s localhost:3000/ | grep -oE '(src|href)="/pkg/[^"]*"'
href="/pkg/_bg.wasm"                <-- the output name is EMPTY
href="/pkg/egonik-site.css"         <-- but this one is hardcoded in app.rs, so it works
href="/pkg/.js"                     <-- and this one is broken

$ curl -s -o /dev/null -w '%{http_code}' localhost:3000/pkg/.js
400

$ curl -s localhost:3000/ | grep -o 'Click Me[^<]*'
Click Me:                           <-- the HTML is complete and correct
```

Read that carefully, because the shape of the failure is the lesson. The server-rendered HTML is complete. The stylesheet loads — because `app.rs` hardcodes `href="/pkg/egonik-site.css"` rather than deriving it. So the page *looks entirely correct* in a browser and in view-source. Only the WebAssembly is missing, so nothing is interactive: the counter button does nothing, and no error appears anywhere except a 400 in the network tab for a file called `.js`.

The two options in that warning message, and which one you want

The message offers two remedies. Take neither at face value.

Option 1 — "pass `Some("Cargo.toml")` to `get_configuration`" — makes the running binary read `Cargo.toml` from disk at startup. It works in your source tree and fails in any container that ships only the binary and the site directory. Do not do this.

The right answer is to **set the environment variables**, which is what `cargo leptos` does in development and what your `Dockerfile` must do in production:

```
LEPTOS_OUTPUT_NAME=egonik-site \
LEPTOS_SITE_ROOT=target/site \
LEPTOS_SITE_PKG_DIR=pkg \
LEPTOS_ENV=PROD \
./target/release/egonik-site
```

With `LEPTOS_OUTPUT_NAME` set, the link becomes `/pkg/egonik-site.js` and returns 200.

Table 7.1. Symptoms of a missing runtime variable. None of these produce a Rust error.

Variable	If unset	Symptom
`LEPTOS_OUTPUT_NAME`	bundle name is empty	`/pkg/.js` → 400; page renders, never hydrates
`LEPTOS_SITE_ROOT`	defaults to `"."`	static files 404; page renders unstyled
`LEPTOS_SITE_ADDR`	`127.0.0.1:3000`	in a container: unreachable from outside. Set `0.0.0.0:8080`
`LEPTOS_ENV`	`DEV`	the live-reload client is injected and retries `localhost:3001` forever

7.2 How to tell whether hydration actually happened

Because a non-hydrating page looks identical to a working one, you need a deliberate test. Three checks, cheapest first:

1. **Does `/pkg/<output-name>.js` return 200?** If not, nothing else matters. This is the check that catches the entire failure above, and it is one `curl`.
2. **Does an interactive element work?** The starter template's counter is genuinely useful for this, which is a reason to keep something clickable on the page until you have real interaction. If the button increments, the wasm loaded and took over.
3. **Is the browser console clean?** Which brings us to the panic hook.

What `console_error_panic_hook` is actually for

`src/lib.rs` calls `console_error_panic_hook::set_once()` inside `hydrate()`. Without it, a panic in WebAssembly aborts with the message `unreachable executed` and no indication of where. With it, you get the real panic message and a stack trace in the browser console.

This is not optional infrastructure — it is the only diagnostic you have on the client side. Keep the call, and make "check the console" part of how you verify a page, because a hydration panic silently leaves the page in the same non-interactive state as a missing bundle.

7.3 The component body runs twice, and that has consequences

From Chapter 5: on a cold load your component function executes on the server, and then again in the browser during hydration. Leptos requires that both executions produce the *same* markup. If they differ, hydration mismatches — the framework attaches event listeners to the wrong nodes, and behaviour becomes inexplicable.

Anything non-deterministic inside a `view!`

```
// renders one time on the server, a different time in the browser
view! { <p>"Rendered at " {chrono::Local::now().to_string()}</p> }

// a different value on each of the two runs
view! { <li id=format!("item-{}", rand::random::<u32>())>...</li> }
```

Both compile. Both produce a mismatch. Anything time-dependent or random must either be computed once on the server and passed through as data, or computed only on the client inside an `Effect`, which does not run during server rendering.

`chrono::Local` needs a timezone the browser will not agree about

This is a live risk for this project, because `chrono` is compiled into both builds and the schema is full of dates. `Local::now()` resolves against the machine's timezone — the server's timezone on one run, the visitor's on the other. Format dates on the server, or format them from a `NaiveDate` with no timezone involved at all, which is what every date column in this schema already is.

7.4 Where the assets actually end up

One more piece of the run contract, and a genuinely surprising one. `Cargo.toml` says `assets-dir = "assets"`. The contents of that directory are copied *into the root of the site directory*, not into a subdirectory named `assets`.

Verified on this machine

```
$ find target/site -maxdepth 2
target/site
target/site/favicon.ico          <-- assets/favicon.ico landed at the ROOT
target/site/pkg
target/site/pkg/egonik-site.wasm
target/site/pkg/egonik-site.js
target/site/pkg/egonik-site.css

$ curl -s -o /dev/null -w '%{http_code}' localhost:3000/favicon.ico
200
```

So a file you place at `assets/media/me.jpg` is served at **`/media/me.jpg`** — not at `/assets/media/me.jpg`. This matters concretely: `personal_informations.image_url` is a column you are about to fill in, and the intuitive value is the wrong one.

And /assets/ currently serves the entire site root

`main.rs` contains `Files::new("/assets", &site_root)` — note that the second argument is the site root, not the assets directory. So the URL prefix `/assets` is mapped onto everything.

```
$ curl -s -o /dev/null -w '%{http_code} %{size_download}' \
       localhost:3000/assets/pkg/egonik-site.wasm
200 2874275
```

Every file is reachable by two different URLs. Nothing under `target/site` is secret, so this is not an information leak — but two URLs for one resource is bad for caching and confusing to debug. It is inherited from the upstream template. Defect **D18**.

7.5 The check that proves nothing

A closing warning for this part, because it undermines the habit recommended in Chapter 4 if you get it wrong.

Verified: a bare `cargo check` compiles neither program

```
$ cargo check --no-default-features
    Finished `dev` profile [unoptimized + debuginfo] target(s)
```

It succeeds — and it has checked *neither* the server nor the browser build. With no features enabled, `leptos` has no renderer, every server module is `cfg`'d out, and `main.rs` falls through to its `#[cfg(not(any(feature = "ssr", feature = "csr")))]` stub, which is an empty `fn main()`.

A green `cargo check` in this repository is therefore meaningless unless it names a feature. Always both, always explicitly:

```
cargo check --no-default-features --features ssr
cargo check --lib --no-default-features --features hydrate --target wasm32-unknown-unknown
```

Part III

Architecture

Chapter 8

Layers, and the shape to grow into

8.1 The organising principle: vertical slices, horizontal gates

You already made a good structural decision without necessarily framing it as one. The code is organised by *subject* — `portfolio/`, `publications/`, `job_experience/`, `personal_information/` — rather than by *technical kind* (a `models/` directory, a `repositories/` directory). That is the right instinct. Each subject is a vertical slice, and everything about publications lives in one place, so adding a field to a publication touches one directory.

What is missing is the second axis. Within each slice, code needs to be separated by *which of the two programs it belongs to.* Right now that separation happens at the top of `src/lib.rs`, at whole-module granularity, which is too coarse: it forces an entire subject to be server-only when only half of it needs to be.

> *Organise* directories *by subject. Organise* files within a directory *by which side of the boundary they live on. The* `cfg` *gates then sit on individual files in* `<subject>/mod.rs`*, not on whole subjects in* `lib.rs`*.*

8.2 The target tree

Here is the layout to grow into. Compare it against the inventory in Chapter 2; most of it is a redistribution of files you already have.

```
src/
  lib.rs                  # module roots. Gates live here only for genuinely server-only subjects.
  main.rs                 # the composition root: config, pool, Actix. ssr only by construction.
  app.rs                  # <App/>: the shell, <Routes>, global layout
  error.rs                # AppError: the one error type the browser is allowed to see

  ui/                     # subject-AGNOSTIC presentation only
    mod.rs
    layout.rs             #   header, nav, footer
    components.rs         #   reusable pieces: Card, TagList, DateRange, ExternalLink
    pages/                #   one file per route; a page composes several subjects

  portfolio/              # a vertical slice
    mod.rs                #   declares the files below, and holds the cfg gates
    dto.rs                #   PortfolioItemDto, TagDto        -- crosses the wire
    api.rs                #   #[server] list_portfolio(), get_portfolio_item()
    components.rs         #   <PortfolioList/>, <PortfolioCard/>  -- subject-specific
    model.rs              # ssr  Diesel structs: Queryable, Selectable
    repository.rs         # ssr  the SQL
  publications/           # same six files
  job_experience/         # same six files
  personal_information/   # same six files

  schema.rs               # ssr  diesel table! macros, generated -- never edit by hand
  database/
    mod.rs
    connection.rs         # ssr  build the r2d2 pool from a config value (not from env)
```

```
config.rs                # ssr  typed AppConfig, loaded once in main
```

Listing 8.1. The target module tree. *ssr* marks files compiled only into the server; everything else is compiled into both programs.

Three things changed relative to today, and each one has a reason.

`src/entrypoint/` narrowed to composition, not controllers. The original `routes/request/response` shape was reaching for a controller layer that this stack generates for you: server functions *are* the inbound adapter, and `api.rs` inside each slice is where they live. Writing Actix handlers that return JSON for your own front end duplicates `#[server]` and loses type safety across the wire.

That subtree has since been deleted in favour of `http.rs` and `application_state.rs`, which is a much better use of the directory — see §12.2. The remaining risk is only that two declared-but-empty modules are an invitation; fill them or remove them, rather than leaving them indefinitely. Defect **D2**.

Each slice gained `dto.rs` and `api.rs`, and both are ungated. This is the single most important structural change in the document. Chapter 9 explains the two-struct rule and Chapter 12 shows the wiring; the short version is that a `#[server]` function *must* be visible to the browser build, so the module holding it cannot be gated.

`src/ui/` is new. Components do not belong in `app.rs`. `app.rs` should stay small enough to read in one screen: it is a table of contents for the site, not a place where markup accumulates.

> **The gate that will bite you**
>
> `#[cfg(feature = "ssr")] pub mod publications;` makes every server function inside `publications/` invisible to the browser build. The error is `error[E0433]: failed to resolve: could not find `publications` in the crate root`, and rustc helpfully adds `found an item that was configured out ... the item is gated behind the `ssr` feature`. The fix is never to un-gate the *query* code — it is to move the gate down one level, onto the files that actually contain Diesel. This is defect **D5**.

> **This is happening right now, in `src/publications/`**
>
> While this document was being written, a component appeared in the repository:
>
> ```rust
> // src/publications/mod.rs
> pub mod components;      // <-- new
> pub mod model;
> pub mod repository;
>
> // src/publications/components.rs
> use leptos::prelude::*;
>
> #[component]
> fn Publications() -> impl IntoView {
>     view! {  "Aca van las publicaciones"  }
> }
> ```
>
> This is a good instinct — keeping the component next to the data it renders — and it has walked straight into the gate. Two things are wrong, and neither produces an error yet:
>
> **The component is in the server-only build.** `src/lib.rs` still says `#[cfg(feature = "ssr")] pub`

`mod publications;`, so `components.rs` is compiled into the server and *not* into the wasm bundle. It compiles today only because nothing references it. The moment `app.rs` adds `<Publications/>`, the browser build fails with the `E0433` above. Moving the gate off `publications` and onto `model` and `repository` fixes it — and that gate move is exactly the change in `<subject>/mod.rs` shown below.

The component is private. `fn Publications()` has no `pub`, so even in the server build nothing outside `components.rs` can name it. Components that a route renders need to be `pub`.

Components in the slice, or in `ui/`?

Both are defensible, and the tree above shows only one of them. Co-locating a component with its data — `publications/components.rs` — keeps a feature in one directory and is easier to delete cleanly. A shared `ui/` directory keeps the pieces that are genuinely reusable (`Card`, `TagList`, `DateRange`) from being buried inside whichever subject happened to need them first.

The split that works in practice: **subject-specific components live in the subject** (`publications/components.rs` renders publications and nothing else); **subject-agnostic components live in `ui/`**. Pages — the things a route maps to — go in `ui/pages/`, because a page usually composes more than one subject. The one hard rule is the gate: any file containing a component or a `#[server]` function must be reachable in both builds.

8.2.1 What `<subject>/mod.rs` looks like

```rust
pub mod api;                              // #[server] functions: BOTH programs
pub mod components;                       // <Publications/>:     BOTH programs
pub mod dto;                              // wire types:          BOTH programs

#[cfg(feature = "ssr")]
pub mod model;                            // Diesel structs: server only
#[cfg(feature = "ssr")]
pub mod repository;                       // SQL:           server only
```

Listing 8.2. `src/publications/mod.rs` — the gate lives here, per file. This is the change that unblocks the component already sitting in that directory.

and correspondingly `src/lib.rs` loses four of its seven gates:

```rust
pub mod app;
pub mod error;
pub mod ui;
pub mod job_experience;                   // ungated: each holds DTOs, server fns, components
pub mod personal_information;
pub mod portfolio;
pub mod publications;

#[cfg(feature = "ssr")] pub mod config;
#[cfg(feature = "ssr")] pub mod database;
#[cfg(feature = "ssr")] pub mod schema;
```

Listing 8.3. `src/lib.rs` — only the genuinely server-only roots stay gated.

An equivalent and equally idiomatic convention — the one the upstream Leptos examples use — is to keep a slice in a single file with an inner gated module:

```rust
// src/publications/mod.rs
```

```rust
use crate::error::AppError;
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct PublicationDto { /* ... */ }

#[cfg(feature = "ssr")]
mod ssr {                                   // everything server-only, in one place
    pub use crate::database::connection::DbPool;
    pub use crate::schema::publication_items;
    pub use diesel::prelude::*;
}

#[server]
pub async fn list_publications() -> Result<Vec<PublicationDto>, AppError> {
    #[cfg(feature = "ssr")]
    use self::ssr::*;                       // the body is compiled only for ssr anyway
    // ...
}
```

Listing 8.4. The single-file variant, for slices small enough not to need four files.

Pick one convention and hold to it. Four files per slice is better once a slice has more than one query and more than one DTO, which all four of yours will.

8.3 Dependency direction

Layers are only useful if the arrows point one way. Figure 8.1 states the rule for this project.

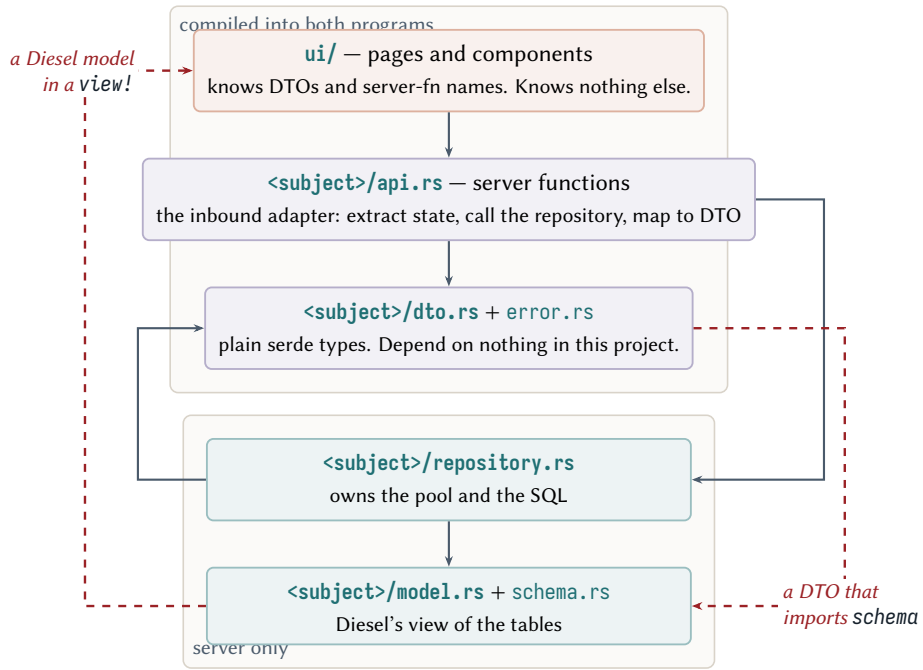


Figure 8.1. Permitted dependency directions. Every arrow points *down or inward.* The two crossings marked in red are the ones to watch for — both are easy to write and both invert the architecture.

The two red arrows are worth naming, because they are the two ways this design decays:

1. **A Diesel model rendered directly in a `view!`.** It compiles, if you also un-gate the model, and then `diesel` follows it into the wasm build and the build breaks with errors about `mio` or missing `core` items. That failure mode is documented in the Leptos book precisely because it is

common; `diesel` with the `postgres` feature links native `libpq`, which has no WebAssembly target.

2. **A DTO that reaches for `schema.rs`** — usually because someone added `#[diesel(table_name = ...)]` to the DTO to avoid writing a conversion. That drags the schema into the shared layer and the boundary is gone.

8.4 One crate or a workspace?

Worth deciding now, because migrating later is tedious. Both shapes are supported by cargo-leptos.

Table 8.1. The two project shapes, judged for this project specifically.

	Single crate (today)	**Workspace: `shared` / `app` / `frontend` / `server`**
Boundary enforced by	discipline plus `cfg` gates	the compiler — `server` is never built for wasm
Config	`[package.metadata.leptos]`	`[[workspace.metadata.leptos]]` with `bin-package` and `lib-package`
Cost	one `Cargo.toml`; every gate is yours to get right	four manifests, and the shared crate *still* needs `ssr`/`hydrate` gates
Risk	a mis-gated module reaches wasm	`cargo build --workspace` unifies features and silently enables `ssr+hydrate` together

Stay with one crate

For a personal website with four content types, the single-crate layout is the right call. The workspace's benefit is that it makes the boundary a compiler-enforced fact rather than a convention — genuinely valuable at scale, and not worth four manifests here. The gates in `lib.rs` plus the two-build check from Chapter 4 give you most of the safety for a fraction of the ceremony.

Revisit this if either becomes true: the server grows logic you want to unit-test without compiling wasm at all, or the wasm bundle grows past roughly a megabyte and you need to audit what is in it.

If you ever do move to a workspace

Two traps, both silent. `[[workspace.metadata.leptos]]` is an *array* of tables — single brackets and cargo-leptos simply will not find the project. And profile sections (`[profile.wasm-release]`) are honoured only in the *root* manifest; put them in a member and your `opt-level = 'z'` is ignored with a warning you will not read.

Chapter 9

Domain models and DTOs: the two-struct rule

9.1 The rule

> *For every content type, write two structs. One describes the* table *and lives on the server. One describes the* payload *and travels to the browser. Convert between them with* `From`*. Do not try to make one struct do both jobs.*

This will feel like duplication for about a week, and then it will start paying for itself. The reasons are not aesthetic.

Why this matters

They have different lifetimes. The table struct changes when you run a migration; the payload changes when the page needs different information. Coupling them means every schema tweak is a wire-format change.

They have different derives, and the derives are mutually hostile. The table struct needs `Queryable` and `Selectable` from Diesel, which do not exist in the browser build. The payload needs `Serialize` and `Deserialize` present in *both* builds. One struct cannot satisfy both sets without `cfg_attr` gymnastics that break in a way described below.

They have different obligations about truth. The table struct mirrors the database exactly, including columns the page must never see. Your `portfolio_items` table has a `public` boolean; the DTO for a public listing should not have that field at all, because a field that does not exist cannot be leaked by a careless `view!`.

The payload is a published API. A server function is a public HTTP endpoint. Anyone can call it. The DTO is the contract you are publishing, and it deserves to be designed rather than inherited from whatever your table happens to look like.

9.1.1 What it looks like

```rust
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
pub struct PublicationDto {
    pub id: i32,
    pub title: String,
    pub abstract_text: String,   // renamed: `abs` is a SQL function name and an unclear field name
    pub year: i32,
    pub journal: String,
    pub link: String,
}
```

Listing 9.1. `src/publications/dto.rs` — ungated, no Diesel anywhere.

```rust
use diesel::prelude::*;

#[derive(Debug, Queryable, Selectable)]
#[diesel(table_name = crate::schema::publication_items)]
#[diesel(check_for_backend(diesel::pg::Pg))]
pub struct PublicationItem {
    pub id: i32,
    pub title: String,
    pub abstract_text: String,
    pub year: i32,
    pub journal: String,
    pub link: String,
}

impl From<PublicationItem> for crate::publications::dto::PublicationDto {
    fn from(row: PublicationItem) -> Self {
        Self {
            id: row.id,
            title: row.title,
            abstract_text: row.abstract_text,
            year: row.year,
            journal: row.journal,
            link: row.link,
        }
    }
}
```

Listing 9.2. `src/publications/model.rs` — server only, unchanged in spirit from today.

Note that `Serialize` and `Deserialize` have left the model entirely. The model has no business being serialised — it never crosses a network. That deletion is not cosmetic; it is what makes the boundary real.

Put the `From` impl next to the model

The conversion is a server-side concern and depends on the server-side type, so it belongs in `model.rs`, not `dto.rs`. Keeping it there means `dto.rs` imports nothing from your crate at all, which is exactly the property that guarantees it is safe for the browser.

9.2 The mistake that looks like the fix

There is an obvious-seeming way to avoid writing two structs: keep one struct and gate the derives.

Gating serde behind `cfg_attr`

```rust
#[derive(Debug, Clone)]
#[cfg_attr(feature = "ssr", derive(serde::Serialize, serde::Deserialize))]
#[cfg_attr(feature = "ssr", derive(diesel::Queryable, diesel::Selectable))]
pub struct PublicationItem { /* ... */ }
```

This is worse than the original problem, because it fails *later* and more obscurely. The type now exists in the browser build but without its serde impls, so the failure moves from the type definition to every use site.

The reported errors are worth showing, because recognising them saves an hour. Research for this document reproduced them against `leptos 0.8.20`: you get four `error[E0277]`s per DTO. Two come from the `#[server]` macro:

```
error[E0277]: the trait bound `Dto: serde::Serialize` is not satisfied
  = note: required for `JsonEncoding` to implement `Encodes<Vec<Dto>>`
  = note: required for `Vec<Dto>` to implement
          `IntoRes<Post<JsonEncoding>, BrowserMockRes, ServerFnError>`
  = note: required by a bound in `leptos::server_fn::ServerFn::Protocol`
```

and two more come from `Resource::new` independently of the server function, anchored at `leptos_server-0.8.7/src/resource.rs:1030`, where the bound is `JsonSerdeCodec: Encoder<T> + Decoder<T>`.

Read the type names in the error

`BrowserMockRes`, `BrowserClient`, `BrowserMockServer`. Those names are the diagnostic. They tell you the bound is being checked *in the client build*, which tells you the missing impl has to exist in the client build, which tells you the `cfg_attr` is the bug. Whenever a confusing trait error mentions a `Browser*` type, ask what is missing on the client side.

Only *server-only* derives belong in `cfg_attr`. If you ever do collapse a model and a DTO into one type — reasonable for a slice that is a pure pass-through — it looks like this, with serde unconditional:

```rust
#[derive(Debug, Clone, serde::Serialize, serde::Deserialize)]        // always
#[cfg_attr(feature = "ssr", derive(diesel::Queryable, diesel::Selectable))]
#[cfg_attr(feature = "ssr", diesel(table_name = crate::schema::publication_items))]
#[cfg_attr(feature = "ssr", diesel(check_for_backend(diesel::pg::Pg)))]
pub struct Publication { /* ... */ }
```

9.3 Private fields, and why they are a problem now

Two of your four models declare their fields private:

```rust
pub struct JobExperienceItem {
    id: i32,
    date_from: NaiveDate,
    date_to: Option<NaiveDate>,
    job_title: String,
    accomplishments: String,
    responsabilities: String,
}
```

Listing 9.3. `src/job_experience/model.rs` — no `pub` on any field.

`PersonalInformation`, `ContactInformation` and `JobInstitution` are the same; `PortfolioItem`, `Tag` and `PublicationItem` correctly use `pub`. This compiles today only because nothing outside those modules reads the fields. The moment a `From` impl in another module, or a page in `ui/`, tries to read `job_title`, you get `error[E0616]: field `job_title` of struct `JobExperienceItem` is private`. This is defect **D7**.

Two acceptable resolutions, and the choice is a real design decision rather than a formality:

- **Make the fields `pub`.** Correct for a row struct. A Diesel model is a transparent record of what the database returned; it has no invariants to protect, and adding getters for all six fields would be ceremony with no content. Do this.
- **Keep them private and add a constructor plus accessors.** Correct only if the type is a real domain aggregate that enforces invariants — "`date_to` is either `None` or on/after `date_from`". If you want that invariant enforced in Rust rather than only by a database `CHECK` constraint, then the invariant-holding type is a *third* type, distinct from both the row and the DTO, and it deserves the privacy. That is a reasonable thing to want later. It is not what these structs are today.

Whichever you choose, choose it consistently. Right now the codebase does both, and the difference is not deliberate — it is which file was written on which evening. Consistency here is worth more than either option.

Chapter 10

Repositories, blocking, and the pool

10.1 Why `Repository<T>` should not survive

The current abstraction is four lines:

```rust
pub trait Repository<T> {
    fn get_all(&mut self) -> anyhow::Result<Vec<T>>;
    fn get_by_id(&mut self, id: i32) -> anyhow::Result<T>;
}
```

Listing 10.1. `src/core/mod.rs`, in full.

It is a familiar shape from other languages, and in Rust it has four problems. Taken one at a time, because only the last one is fatal:

1. The generic parameter buys nothing

You cannot write `impl<T> Repository<T> for PgStore` and specialise it per entity — coherence forbids overlapping impls, and specialisation is unstable. So every implementation is hand-written anyway, exactly as it is now: four structs, four impls, eight method bodies. The generic parameter adds a type parameter to every signature and returns nothing. A trait per subject with the methods that subject actually needs — `PublicationRepository` with `list()`, `by_id()`, `by_year()` — says more and costs less.

2. Two methods is not an interface, it is a coincidence

`get_all` and `get_by_id` happen to be what all four subjects need *today*, because you have not written any interesting queries yet. The first time you want "publications grouped by year, newest first" or "portfolio items with their tags", it will not fit, and it will go somewhere else — an inherent method on the struct, next to the trait impl. Then the trait describes half of each repository and you have to read both to know what a repository can do.

3. `&mut self` is a misattribution, and it is contagious

This one is worth being precise about, because the reasoning that produces it is so plausible. Diesel query methods take `&mut conn`. So a repository method needs a mutable connection. So — it seems to follow — the repository must be `&mut self`. It does not follow. The `&mut` belongs to the *connection*, which you obtain locally:

```rust
pub fn get(&self) -> Result<PooledConnection<M>, Error>   // r2d2::Pool::get -- note &self
```

`r2d2::Pool` is `pub struct Pool<M>(Arc<SharedPool<M>>)`. It is `Send`, `Sync`, and `Clone`, and its `Clone` is an atomic refcount bump. Concurrent immutable access is exactly what it is built for. Declaring the repository `&mut self` throws that away: every caller now needs exclusive access, which in an async server means putting the repository behind a `Mutex` and serialising every request through it — destroying the concurrency the pool exists to provide. This is defect D8.

Take `&self`, clone the pool

```rust
#[derive(Clone)]                                // cheap: one Arc bump
pub struct PublicationRepository { pool: DbPool }

impl PublicationRepository {
    pub fn new(pool: DbPool) -> Self { Self { pool } }

    pub async fn list(&self) -> Result<Vec<PublicationItem>, RepoError> {
        let pool = self.pool.clone();           // clone into the 'static closure
        // ... see the next section for the closure
    }
}
```

And do not wrap it in `Arc<Pool<…>>`. It is already an `Arc`; a second layer is pure indirection.

4. The signatures are synchronous, and that is the fatal one

`fn get_all(&mut self) -> anyhow::Result<Vec<T>>` is a blocking function. Called from a server function, it blocks an Actix worker thread. The rest of this chapter is about why that matters more than it sounds like it should.

10.2 Blocking: the bug that no test will find

> *Diesel is synchronous. A server function is asynchronous. Calling the former directly from the latter does not fail — it silently converts your concurrent server into a sequential one.*

10.2.1 The mechanism

An async runtime multiplexes many tasks onto few threads by requiring that every task yield when it waits. A blocking call does not yield: it holds its thread for the whole duration of the wait, so the scheduler on that thread cannot poll any other task. Tokio's own documentation is blunt about it — a blocking operation in a task "would block the entire thread, preventing other tasks from running".

Actix makes the consequence sharper than generic Tokio, because each HTTP worker processes its accepted connections on one thread, and the default worker count is `std::thread::available_parallelism()`. So with W workers and a query taking T seconds, exactly W queries progress at a time, and throughput caps near W/T requests per second regardless of how idle Postgres is.

10.2.2 What it looks like when it happens

Not a crash and not a panic — which is why it is easy to ship. The symptoms are:

- flat, low CPU while p99 latency climbs;
- request times that cluster in multiples of your query time;
- *static assets and health checks timing out too*, because they share the same workers;
- no improvement when you raise the r2d2 pool size — the bottleneck is the worker thread, not the pool. This symptom is the giveaway.

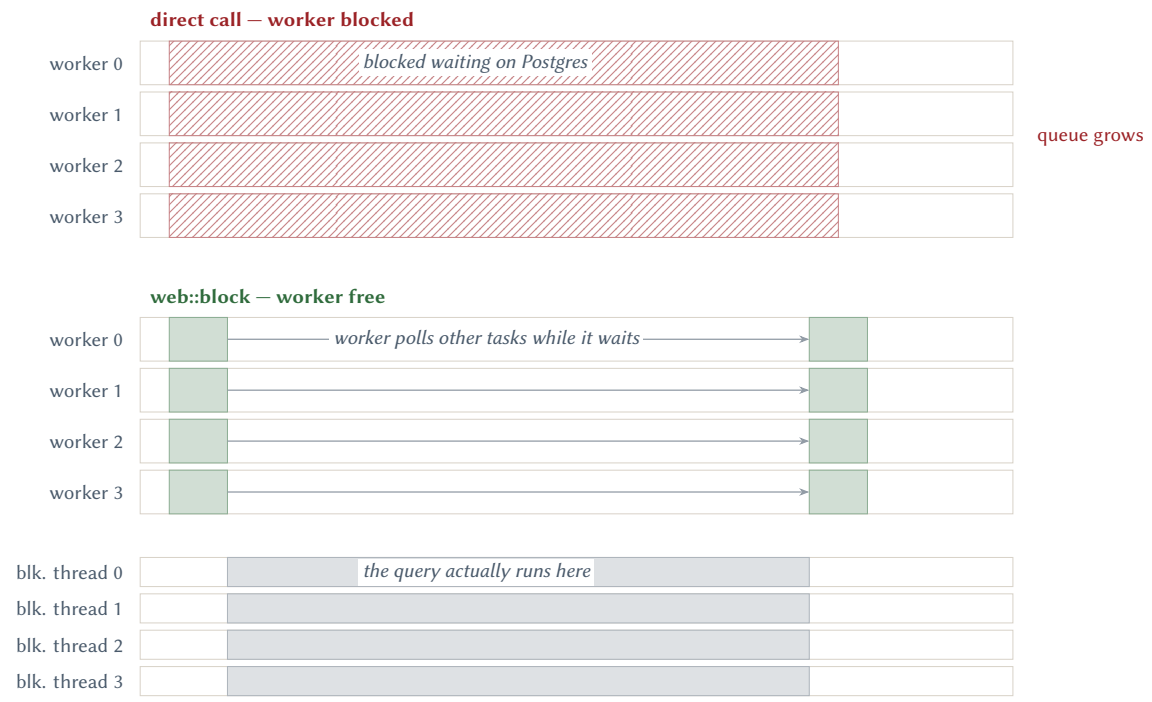


Figure 10.1. Four Actix workers, each handling one request. Above: Diesel called directly — each worker is held for the whole query and cannot serve anyone else. Below: the query is handed to the blocking pool — workers return to the event loop immediately.

10.3 The fix

Wrap every query in `web::block`

```rust
pub async fn list(&self) -> Result<Vec<PublicationItem>, RepoError> {
    use crate::schema::publication_items::dsl::*;
    let pool = self.pool.clone();

    let rows = actix_web::web::block(move || {
        let mut conn = pool.get()?;                  // NOTE: get() is INSIDE the closure
        publication_items
            .select(PublicationItem::as_select())
            .load(&mut conn)
    })
    .await?                                          // BlockingError
    .map_err(RepoError::from)?;                      // the inner error

    Ok(rows)
}
```

Four details in those twelve lines, each of which is a bug if you get it wrong.

`pool.get()` goes inside the closure.
`get()` is itself a blocking call: it parks the calling thread for up to `connection_timeout`, which defaults to *30 seconds*. Acquiring the connection before the closure reintroduces the exact bug you are fixing, in its worst form. The official Actix Diesel example puts `get()` inside the closure for precisely this reason.

There are two `Results`, so there are two `?`.
`web::block(...).await` gives you `Result<Result<T, E>, BlockingError>`. One `?` will not compile; an `.unwrap()` on the outer one swallows the real database error.

Never hold a `PooledConnection` across an `.await`.
Acquire it and drop it inside one closure. Holding it pins a pooled connection for the whole async section, and connections are the scarcest resource you have.

Blocking tasks are not cancellable.
If the visitor closes the tab, Actix drops the request future — but the closure already handed to the blocking pool runs to completion and keeps holding its connection. This is how a pool gets exhausted by clients that already gave up, and it is the reason to set a short `connection_timeout`.

10.3.1 The alternatives, and why not to take them

`tokio::task::spawn_blocking`
is what `web::block` calls internally — its body is literally `let fut = actix_rt::task::spawn_blocking(f); async { fut.await.map_err(|_| BlockingError) }`. Note what that `map_err` discards: `BlockingError` is a fieldless unit struct, so whether the closure panicked or was cancelled is lost. If you want that detail for logging, call `spawn_blocking` yourself and keep the `JoinError`, which has `is_panic()` and `into_panic()`. A reasonable choice for one central helper; not worth it at every call site.

`deadpool-diesel`
gives you an async `pool.get()` and an `interact()` method that does the thread hop for you. It adds a dependency to solve a problem `web::block` already solves, and its newest release (0.6.1) is from May 2024. Skip it unless contention on connection *acquisition* becomes your bottleneck, which for a personal site it will not.

`diesel-async`
removes the thread hop entirely and is genuinely production-viable with Postgres. It is also still 0.x (0.9.2 as of mid-2026), reimplements the Postgres wire protocol rather than using `libpq`, and has had subtle lifetime bugs — notably a "Cannot access shared transaction state" panic when a pooled connection returns to the pool before the futures borrowing it are awaited. Adopting it would trade a solved problem for API churn with no measurable benefit below thousands of requests per second.

> **The decision**
>
> Keep synchronous Diesel with r2d2, and wrap every query in `web::block`. Revisit only if you ever measure connection-acquisition contention.

10.4 Pool sizing

The defaults will work for a personal site, but two of them are worth changing and the reasoning generalises.

```
Pool::builder()
    .max_size(cfg.db_max_connections)            // default is 10
    .connection_timeout(Duration::from_secs(5))   // default is 30 -- far too long
    .test_on_check_out(true)
    .build(ConnectionManager::<PgConnection>::new(&cfg.database_url))
```

Listing 10.2. A pool built from configuration, with deliberate limits.

`connection_timeout` matters because of the non-cancellability point above: a thread waiting 30 seconds for a connection is a blocking-pool slot doing nothing for 30 seconds. Five seconds fails fast instead.

`max_size` sits inside two independent bounds, and it is easy to reason about only one:

- **Below the blocking capacity.** Actix's blocking pool is *per worker*, not global — `worker_max_blocking_threads` defaults to 512 divided by available parallelism, so on an eight-core machine that is 64 per worker, 512 in total. Sizing math borrowed from plain-Tokio advice (one global 512-thread pool) is wrong under Actix.
- **Below Postgres's budget.** `max_size` multiplied by the number of running instances must stay under the server's `max_connections`. r2d2's default of 10 per instance quietly exhausts a small managed Postgres as soon as you run two replicas.

Chapter 11

Errors that cross the wire

11.1 Four layers, four error types

Error handling is where half-built applications accumulate the most incidental damage, because the easy thing — one error type everywhere — works fine until the day it leaks a connection string to a browser. The layering below is small enough to adopt now.

Table 11.1. Which error type belongs where.

Layer	Type	Rule
driver	`diesel::result::Error`, `r2d2::Error`	never escapes `repository.rs`
repository / domain	`thiserror` enums, one per subject or use case	named variants for what callers can act on, plus `Unknown(anyhow::Error)`
wire	one `AppError` in `src/error.rs`	serde-serialisable, ungated, *carries no internal detail*
process	`anyhow`	`main.rs` and adapter constructors only

Why `anyhow` does not belong in the repository

Your repositories currently return `anyhow::Result<T>` with a `.context("Can't get publication items from db")`. That is fine for logging and useless for deciding. A caller cannot ask an `anyhow::Error` "was this a missing row or a dead connection?" without matching on strings. `anyhow` is right where the only reasonable response is to log and give up — which is `main.rs` — and wrong wherever a caller might branch.

There is also a copy-paste artefact worth noticing: three of the four repositories say `"Can't get publication items from db"` regardless of which table they queried. That is defect D11, and it is a symptom rather than a cause. A typed error would have made the mistake impossible, because `PortfolioError::NotFound` cannot say "publication".

11.2 The wire error type

Leptos 0.8 changed the recommended approach here, and tutorials written for 0.6 or 0.7 will lead you astray. `ServerFnError<MyError>` — the `WrappedServerError` variant — is **deprecated as of 0.8.0**. The supported way to return a domain-shaped error is to implement `FromServerFnError` on your own enum.

```
use leptos::prelude::ServerFnErrorErr;
use leptos::server_fn::codec::JsonEncoding;
use leptos::server_fn::error::FromServerFnError;
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, Deserialize, thiserror::Error)]
pub enum AppError {
```

```
    #[error("not found")]
    NotFound,
    #[error("invalid input: {0}")]
    Validation(String),
    #[error("internal error")]
    Internal,                          // deliberately carries nothing
    #[error("transport: {0:?}")]
    ServerFn(ServerFnErrorErr),        // required by the trait below
}

impl FromServerFnError for AppError {
    type Encoder = JsonEncoding;
    fn from_server_fn_error(value: ServerFnErrorErr) -> Self {
        AppError::ServerFn(value)
    }
}
```

Listing 11.1. `src/error.rs` — ungated, and the only error the browser sees.

`Internal` must be empty

`AppError::Internal` carries no payload on purpose. Everything unexpected collapses into it, and the real cause is logged server-side with `tracing::error!`. The alternative — `ServerFnError::ServerError(e.to_string(` — sends the string to the browser verbatim, and `e.to_string()` on a Diesel or r2d2 error can contain your connection string. This is the single most common way a Rust web application leaks infrastructure detail.

```
#[cfg(feature = "ssr")]
impl From<RepoError> for AppError {
    fn from(e: RepoError) -> Self {
        use actix_web::http::StatusCode;
        let status = expect_context::<leptos_actix::ResponseOptions>();
        match e {
            RepoError::NotFound => {
                status.set_status(StatusCode::NOT_FOUND);
                AppError::NotFound
            }
            other => {
                tracing::error!(error = ?other, "unhandled repository error");
                status.set_status(StatusCode::INTERNAL_SERVER_ERROR);
                AppError::Internal          // nothing leaks
            }
        }
    }
}
```

Listing 11.2. The mapping layer: where a domain error becomes a wire error.

Server-function errors do not set a status code by themselves

An error returned from a `#[server]` function travels as an encoded body plus a marker header (`SERVER_FN_ERROR_HEADER`). The HTTP status stays 200 unless you set it. If you want a genuine 404 — and you do, for crawlers, caches and uptime monitoring — you must call `ResponseOptions::set_status` explicitly, as above.

`?` stops working when you leave the default error type

With `Result<T, ServerFnError>` you get `impl<E: std::error::Error> From<E> for ServerFnError` for free, so `?` converts anything. The moment you switch to your own `AppError`, that blanket impl is gone and you must write the conversions. In particular `leptos_actix::extract()` returns `ServerFnErrorErr`, not `ServerFnError`, so it becomes `.map_err(AppError::from_server_fn_error)?`.

Start with `ServerFnError`, migrate deliberately

For the first few server functions, `Result<T, ServerFnError>` is fine and keeps `?` ergonomic while you are still finding the shape of the code. Move to `AppError` when the UI first needs to *distinguish* two failures — typically the first time you want a real 404 page. Do not do both at once; a codebase with two wire-error conventions is worse than either.

Chapter 12

The composition root

12.1 What is wrong with `main.rs` today

`src/database/connection.rs` builds a pool. Nothing calls it. `src/main.rs` is the starter template verbatim: it configures Leptos, serves static files, and never mentions a database. Closing that gap is the single highest-value change in this document, because nothing else can be tested until data can reach a page.

There are also three smaller issues in the current file, all in the same few lines:

```
HttpServer::new(move || {
    let routes = generate_route_list(App);          // (a) fine, but see below
    let leptos_options = &conf.leptos_options;
    let site_root = leptos_options.site_root.clone().to_string();

    println!("listening on http://{}", &addr);      // (b) printed once PER WORKER
    // ...
        .service(Files::new("/assets", &site_root)) // (c) serves the whole site root
```

Listing 12.1. `src/main.rs`, lines 15–21 — three problems in seven lines.

(a)

`generate_route_list(App)` inside the factory is correct — it is required per worker, and it is what the official template does. Mentioned only so you do not "fix" it.

(b)

The `println!` is inside the app factory, so it runs once per worker thread and you get one line per CPU core at startup. Harmless, confusing, and a sign the line is in the wrong scope. Move it above `HttpServer::new`. Defect **D17**.

(c)

`Files::new("/assets", &site_root)` maps the URL `/assets` onto *the whole site root*, not onto the assets subdirectory — so `/assets/pkg/egonik-site.wasm` is also reachable. It comes from the upstream template and is not a security hole (everything under `site_root` is public anyway), but it is surprising. Defect **D18**.

12.2 `AppState`, and the one line that must move

This section was going to propose an `AppState`. While the document was being written you wrote one, so instead it endorses what you wrote and flags a single line that will exhaust your database.

What is in the tree now

```
#[derive(Debug, Clone)]
pub struct AppState {
    pub job_experience_repository: JobExperienceItemRepository,
    pub personal_information_repository: PersonalInformationRepository,
    pub portfolio_items_repository: PortfolioItemsRepository,
    pub publications_repository: PublicationsRepository,
```

```
}

impl AppState {
    pub fn new(pool: DbPool) -> Self {
        // one pool, cloned into four repositories -- each clone is an Arc bump
        Self {
            personal_information_repository: PersonalInformationRepository::new(pool.clone()),
            job_experience_repository: JobExperienceItemRepository::new(pool.clone()),
            portfolio_items_repository: PortfolioItemsRepository::new(pool.clone()),
            publications_repository: PublicationsRepository::new(pool),
        }
    }
}
```

Listing 12.2. `src/entrypoint/application_state.rs`, as written.

This is the right shape — keep it

Holding the four *repositories* rather than the raw pool is better than what this chapter was going to suggest. A server function then asks for the capability it needs (`state.publications_repository`) rather than for a database handle it has to know what to do with, and the pool stops being ambient. Two details you got right and that are worth knowing you got right:

`Clone` is required, because Actix hands application data to each worker, and it is cheap because every repository is one `Arc` bump — which is why `#[derive(Clone)]` on `PublicationsRepository` was the correct accompanying change.

Taking `pool` as a parameter rather than calling `get_connection_pool()` inside `new` keeps the dependency in the signature, which is exactly the property §12.4 argues for.

Two things to add when convenient: `config: Arc<AppConfig>` alongside the repositories, so that things like a page size or a base URL stop being constants; and `new` returning `anyhow::Result<Self>` once the pool construction stops panicking.

12.2.1 The line that must move

The pool is being built inside the worker factory

```
HttpServer::new(move || {
    let pool = get_connection_pool();          // <-- HERE. Once per worker.
    let routes = generate_route_list(App);
    let app_state = AppState::new(pool);
    println!("listening on http://{}", &addr);

    App::new()
        .app_data(web::Data::new(app_state))
        // ...
})
```

Listing 12.3. `src/main.rs`, as written. The closure body runs *once per worker*.

Everything in that closure is constructed once per worker thread, and you already have proof of how many times that is: the `println!` on the following line printed **twelve times** when the

binary was run for this document. Twelve invocations of the factory means twelve independent connection pools.

Verified: this over-subscribes Postgres by 20%

```
$ nproc                                  # = Actix's default worker count
12
$ grep -c 'listening on' server.log      # = times the factory closure ran
12

r2d2's Builder::max_size default           = 10   (connection.rs sets no max_size)
pools created                              = 12
maximum connections this process will open  = 120

$ psql -tAc 'show max_connections'
100
```

So under load this process alone tries to open 120 connections against a server that permits 100, and the failures arrive as `r2d2` timeouts surfacing as 500s — after a 30-second wait each, because `connection_timeout` is also at its default. Note that it will not fail in development: each pool opens connections lazily, and one browser tab never needs more than a handful. That is what makes it worth fixing before it can be observed.

Build it once, above the factory, and clone in

```rust
let pool = get_connection_pool();          // once
let app_state = AppState::new(pool);       // once
println!/tracing::info!(...);              // once

HttpServer::new(move || {
    let routes = generate_route_list(App); // this one genuinely IS per-worker
    App::new()
        .app_data(web::Data::new(app_state.clone()))   // one Arc bump per worker
        // ...
})
```

Why `generate_route_list` stays inside and the pool does not

They look symmetrical and they are not. `generate_route_list(App)` produces a `Vec<ActixRouteListing>` that `leptos_routes` consumes when building *that worker's* service tree, so it belongs to the factory — and it allocates nothing scarce. The pool holds operating-system sockets against a server with a hard limit. The rule is not "move everything out of the factory"; it is **anything holding a finite external resource is constructed once**.

This is defect D38, and it is the only new blocker introduced since the register was written.

12.3 Getting the pool into a server function

Two mechanisms exist. Both work; they differ in what they couple you to.

12.3.1 The Actix route — start here

```rust
let pool = build_pool(&cfg)?;                    // built ONCE, outside the factory
```

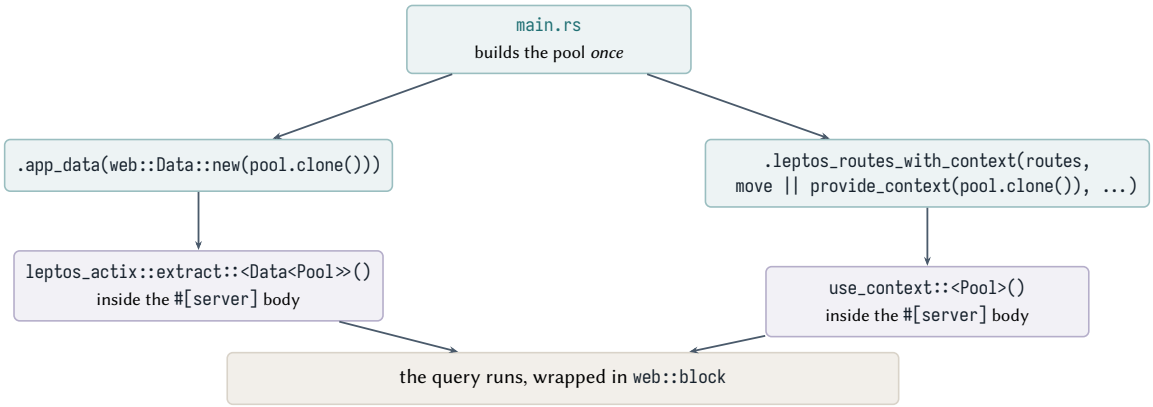


Figure 12.1. Two routes from `main` to a server-function body. The Actix route (left) is what the Leptos book documents; the context route (right) is what you want once a server function should receive a domain service rather than an Actix type.

```
HttpServer::new(move || {
    App::new()
        .app_data(web::Data::new(pool.clone()))   // one Arc bump per worker
        .app_data(web::Data::new(leptos_options.to_owned()))
        .service(Files::new("/pkg", format!("{site_root}/pkg")))
        .service(favicon)
        .leptos_routes(routes, { /* the HTML shell */ })
})
```

Listing 12.4. The wiring, in `main.rs`.

```
#[server]
pub async fn list_publications() -> Result<Vec<PublicationDto>, ServerFnError> {
    use actix_web::web::Data;
    use crate::database::connection::DbPool;
    use crate::publications::repository::PublicationRepository;

    let pool: Data<DbPool> = leptos_actix::extract().await?;
    let repo = PublicationRepository::new(pool.get_ref().clone());
    Ok(repo.list().await?.into_iter().map(Into::into).collect())
}
```

Listing 12.5. And reading it back, in `src/publications/api.rs`.

Why no extra route registration is needed

A reasonable expectation, carried over from earlier Leptos versions, is that server functions need their own Actix route — something like `.route("/api/{tail:.*}", leptos_actix::handle_server_fns())`.

In `leptos_actix` 0.8.7 that is **not** required: `leptos_routes` registers every path from `server_fn::actix::server_fn_paths()` itself, deliberately *before* the router's wildcard route — the source carries the comment "register server functions first to allow for wildcard route in Leptos's Router". `leptos_routes` is literally `leptos_routes_with_context(paths, || {}, app_fn)`.

Adding a second mount is not just redundant: if the duplicate lacks the same context closure it will silently drop context for browser-initiated calls, which produces a bug that appears only after hydration and never during SSR. Do not add it.

`extract()` sees the request head, not the body

`leptos_actix::extract` is implemented as `use_context::<Request>()` followed by `T::extract(&req)`. The body has already been consumed by the server-function decoder, so `web::Json<T>` and `web::Form<T>` extractors will fail. `web::Data<T>`, `Query<T>`, `ConnectionInfo`, `HttpRequest` and headers all work.

12.3.2 The context route — where you will end up

The Actix route works, and it couples your server-function bodies to `actix_web::web::Data`. That is a small cost now and an annoying one later, if you ever want to call a server function from a test without an Actix request in scope. The alternative injects your own type:

```
let pool_for_ctx = pool.clone();
App::new()
    .leptos_routes_with_context(
        routes,
        move || provide_context(pool_for_ctx.clone()),   // both SSR *and* server-fn routes
        { /* the HTML shell */ },
    )
```

```
let pool = use_context::<DbPool>()
    .ok_or_else(|| ServerFnError::new("pool missing from context"))?;
```

Why this is safe in 0.8.7 and was not always

`leptos_routes_with_context` registers the server-function routes with the *same* context closure it uses for rendering. That matters because a server function is reached two different ways — called directly by the renderer during SSR, and posted to over HTTP after hydration — and both paths must have the context. The `leptos_actix` docs warn about exactly this: if you mount `handle_server_fns` by hand you must provide the context to both handlers yourself. Using `leptos_routes_with_context` and nothing else avoids the whole class of mistake.

The order to do it in

Use `app_data` plus `extract` for the first server function, because it is three lines and gets data on screen today. Switch to `leptos_routes_with_context` when you introduce a service type that owns several repositories — at that point you are injecting your own abstraction rather than an Actix wrapper, and the context route reads better.

12.4 Configuration, and the `dotenv` call in the wrong place

```
pub fn get_connection_pool() -> DbPool {
    dotenv().ok();                                          // <-- (1)
    let database_url = env::var("DATABASE_URL").expect("..."); // <-- (2)
    // ...
    .build(manager).expect("Could not build connection pool")    // <-- (3)
}
```

Listing 12.6. `src/database/connection.rs` as it stands.

Three things to change, and the third is the one that will actually hurt.

1. **`dotenv()` inside a constructor.** It mutates process-global state from deep inside a data-access function. Call it once, at the top of `main`, as the dev-time convenience it is. Defect D12.
2. **Reading the environment inside a constructor** hides a dependency from the type signature and makes the function untestable without mutating the process environment — which in Rust 2024 is genuinely awkward, since `std::env::set_var` is `unsafe` because it is not thread-safe. Pass a value in: `fn build_pool(cfg: &AppConfig) -> anyhow::Result<DbPool>`.
3. **`.expect()` in a library function.** A panic here aborts the process with a message and no context. Return `anyhow::Result` and let `main` decide; `main` is where "log it and exit non-zero" is the correct response.

```rust
#[derive(Debug, serde::Deserialize)]
pub struct AppConfig {
    pub database_url: String,
    #[serde(default = "default_pool_size")]
    pub db_max_connections: u32,
}

impl AppConfig {
    pub fn load() -> anyhow::Result<Self> {
        Ok(config::Config::builder()
            .add_source(config::File::with_name("config/base").required(false))
            .add_source(config::Environment::with_prefix("EGONIK").separator("__"))
            .build()?
            .try_deserialize()?)
    }
}
fn default_pool_size() -> u32 { 8 }
```

Listing 12.7. `src/config.rs` — typed, loaded once, validated at the edge.

> **Do not use the `LEPTOS_` prefix for your own variables**
>
> `LEPTOS_` belongs to `leptos_config`. Every field of `LeptosOptions` — `site_root`, `site_addr`, `reload_port`, `hash_files`, and a dozen more — maps to a `LEPTOS_`-prefixed variable, and a future field could collide with yours. Use your own prefix. `EGONIK_DATABASE_URL` in the snippet above.

> **`get_configuration(Some("./Cargo.toml"))` reads Cargo.toml at runtime**
>
> Keep the `None` that the template gave you. Passing a path makes the running binary read `Cargo.toml` on startup, which works in development and fails in a container that ships only the binary and the site directory. With `None`, `LeptosOptions` comes from `LEPTOS_*` environment variables — which is exactly what a Dockerfile should set.

12.5 The whole composition root

Putting it together, this is what `main.rs` should look like. It is longer than the template and it is the only place in the codebase that knows how the pieces connect.

```rust
#[cfg(feature = "ssr")]
#[actix_web::main]
async fn main() -> anyhow::Result<()> {
    use actix_files::Files;
    use actix_web::{web, App, HttpServer};
    use leptos::config::get_configuration;
    use leptos::prelude::*;
```

```
    use leptos_actix::{generate_route_list, LeptosRoutes};
    use leptos_meta::MetaTags;
    use egonik_site::{app::*, config::AppConfig, database::connection::build_pool};

    dotenvy::dotenv().ok();                       // once, here, dev convenience only
    tracing_subscriber::fmt::init();              // see Chapter on operations

    let cfg = AppConfig::load()?;                 // typed and validated before anything starts
    let pool = build_pool(&cfg)?;                 // fail fast if the database is unreachable

    let conf = get_configuration(None)?;          // LEPTOS_* only
    let addr = conf.leptos_options.site_addr;
    tracing::info!(%addr, "starting server");     // ONCE, not per worker

    HttpServer::new(move || {
        let routes = generate_route_list(App);
        let leptos_options = &conf.leptos_options;
        let site_root = leptos_options.site_root.clone().to_string();

        App::new()
            .app_data(web::Data::new(pool.clone()))
            .app_data(web::Data::new(leptos_options.to_owned()))
            .service(Files::new("/pkg", format!("{site_root}/pkg")))
            .service(favicon)
            .leptos_routes(routes, {
                let leptos_options = leptos_options.clone();
                move || view! {
                    <!DOCTYPE html>
                    <html lang="en">
                        <head>
                            <meta charset="utf-8"/>
                            <meta name="viewport" content="width=device-width, initial-scale=1"/>
                            <AutoReload options=leptos_options.clone()/>
                            <HydrationScripts options=leptos_options.clone()/>
                            <MetaTags/>
                        </head>
                        <body><App/></body>
                    </html>
                }
            })
    })
    .bind(&addr)?
    .run()
    .await?;
    Ok(())
}
```

Listing 12.8. The target `src/main.rs`.

Fail fast at startup

`build_pool` returning `Result` into a `main` that returns `anyhow::Result<()>` means an unreachable database stops the process immediately with a readable message, instead of producing a 500 on the first request. For a personal site running under a supervisor that restarts it, this is the behaviour you want: a container that will not start is far easier to diagnose than one that starts and silently serves errors.

Chapter 13

Routing, and what `entrypoint/` is for

13.1 There are three kinds of route, and you only write two

The confusion about where routes live comes from the word "route" meaning three different things in this stack. Separating them answers most of the question by itself.

Table 13.1. The three kinds of route. Note the middle row: you never write those.

Kind	Owned by	Where you write it
A URL a human visits or bookmarks	`leptos_router`	`<Routes>` in `app.rs`. Not Actix.
A browser-to-server data call	the `#[server]` macro	**Nowhere.** The macro creates the endpoint and `leptos_routes` registers it.
A machine-facing HTTP endpoint	Actix	`entrypoint/routes/`. This is the only kind that belongs there.

> *Server functions really do remove the need for a controller layer — for anything your own front end calls. What they do* not *cover is everything that is not your front end: health checks, crawlers, feed readers, webhooks. That residue is what `entrypoint/` is for, and it is small but permanent.*

13.1.1 So what is actually left for Actix?

A short and closed list. If a new endpoint is not on it, it should probably be a server function or a page route instead.

Health and readiness.
`/healthz`. Must be callable by a monitor that knows nothing about Leptos, WebAssembly or your DTOs. Ch. 22.

Things crawlers and machines fetch.
`robots.txt`, `sitemap.xml`, `rss.xml`, `.well-known/*`. These are not HTML, so they are not pages; and they are not called by your front end, so they are not server functions. `robots.txt` can be a static file in `assets/`; `sitemap.xml` and `rss.xml` need database content, so they need a handler.

Inbound calls from third parties.
Webhooks, OAuth callbacks. A server function is the wrong shape — its request encoding is Leptos's, not the caller's.

Streaming uploads and downloads.
A CV PDF, an image endpoint with resizing. Server functions serialise their whole payload; Actix can stream.

Middleware and mounts.
Compression, tracing, security headers, the `/pkg` and `/assets` static services.

The `App` factory itself.
The builder currently inlined in `main.rs`.

> **Register your own services *before* `leptos_routes`**
>
> Actix matches routes in registration order, and `leptos_routes` installs a wildcard that catches every remaining path so the Leptos router can handle client-side URLs. Anything you register *after* it is unreachable.
>
> The template already gets this right — `Files::new("/pkg", ...)`, the `/assets` mount and `favicon` all come before `.leptos_routes(...)` in `main.rs`. Keep that ordering, and add new services to the same group. When a new endpoint returns 404 and you are certain it is registered, this is the first thing to check.

13.2 A shape for `entrypoint/`

Your instinct that this directory holds "the server" is right, and it is worth stating what that means precisely: **`entrypoint/` is the process edge**. It owns how the world reaches the application and how the application starts, and it owns no business logic at all.

```
src/entrypoint/
  mod.rs                # #[cfg(feature = "ssr")] on each child -- already correct
  application_state.rs  # AppState: the repositories and services -- already written
  http.rs               # the Actix App factory, moved out of main.rs
  routes/               # machine-facing endpoints ONLY -- never a page, never a server fn
    mod.rs
    health.rs           #   GET /healthz
    seo.rs              #   GET /sitemap.xml, /rss.xml
  jobs/                 # scheduled work
    mod.rs
    schedule.rs         #   the in-process scheduler, if you choose one (see below)
```

Listing 13.1. `src/entrypoint/` filled in. Everything here is ssr-only by nature, which `mod.rs` already declares correctly.

13.2.1 What goes in `http.rs`

Actix has a purpose-built mechanism for moving registration out of `main`: `App::configure`, which takes a closure over a `ServiceConfig`. It keeps `main.rs` about *starting* the process and `http.rs` about *what the process serves*.

```rust
use actix_web::web::{self, ServiceConfig};
use crate::entrypoint::{application_state::AppState, routes};

/// Everything Actix serves that is not a Leptos page or a server function.
pub fn configure(state: AppState) -> impl FnOnce(&mut ServiceConfig) + Clone {
    move |cfg: &mut ServiceConfig| {
        cfg.app_data(web::Data::new(state.clone()))
            .service(routes::health::healthz)
            .service(routes::seo::sitemap)
            .service(routes::seo::rss);
    }
```

```
}
```

Listing 13.2. src/entrypoint/http.rs

```
HttpServer::new(move || {
    let routes = generate_route_list(App);
    let leptos_options = &conf.leptos_options;
    let site_root = leptos_options.site_root.clone().to_string();

    App::new()
        .configure(http::configure(app_state.clone()))          // <-- your endpoints, first
        .service(Files::new("/pkg", format!("{site_root}/pkg")))
        .service(Files::new("/assets", format!("{site_root}/assets")))
        .service(favicon)
        .leptos_routes(routes, { /* the shell */ })             // <-- the wildcard, last
        .app_data(web::Data::new(leptos_options.to_owned()))
})
```

Listing 13.3. src/main.rs then reads as a list of decisions, not a wall.

AppState is registered in one place

Note that configure is where app_data(AppState) now lives, so the state is registered once and the reason is visible. leptos_actix::extract::<Data<AppState>>() inside a server function keeps working unchanged — app_data is App-wide regardless of which builder call installed it.

13.3 Scheduled work

You raised cron, and it is a real question because src/bin/seed_publications.rs is exactly the kind of thing that wants to run on a timer. Three options, and they are genuinely different rather than a matter of taste.

Table 13.2. Where to put recurring work.

Option	For	Against
systemd timer or host cron, invoking the existing binary	Already works — the binary exists. Cannot take the web server down. Failures land in journalctl. Runs, retries and logs are the operating system's problem, not yours.	Needs host-level configuration, so it lives outside the repository. Awkward on a PaaS that only runs one process.
In-process tokio task spawned in main	Nothing to configure outside the app; ships with the container; shares AppState.	A panic or a leak is in your web process. Runs once per replica, so it needs a lock the day you scale past one. No history, no retry, unless you write it.
A scheduler crate (tokio-cron-scheduler, or apalis for a real queue)	Cron expressions, overlap prevention, and with apalis a persistent job store with retries.	A dependency and a concept for something a personal site does hourly at most.

Use a systemd timer, and keep the binary

For this project the operating system is the right scheduler. The publication sync is idempotent once §14.4 is applied, it does not need to share memory with the web server, and if it fails at 03:00 you want that in the system journal rather than interleaved with request logs. Concretely:

```
# egonik-sync.service
[Unit]
Description=Sync publications from OpenAlex
[Service]
Type=oneshot
EnvironmentFile=/etc/egonik/env
ExecStart=/opt/egonik/seed_publications

# egonik-sync.timer
[Unit]
Description=Daily publication sync
[Timer]
OnCalendar=daily
Persistent=true          # catch up if the machine was off
RandomizedDelaySec=30m   # do not hammer OpenAlex on the hour
[Install]
WantedBy=timers.target
```

Listing 13.4. `/etc/systemd/system/egonik-sync.service` and `.timer`

`Persistent=true` is the line that makes this better than a bare cron entry: a missed run executes on next boot instead of being skipped silently.

If you would rather keep it in-process — a reasonable choice if you deploy to somewhere that runs exactly one container and nothing else — the dependency-free version is a spawned task, and it belongs in `entrypoint/jobs/`:

```rust
use std::time::Duration;

pub fn spawn(state: AppState) {
    actix_web::rt::spawn(async move {
        // one tick immediately after boot, then every 24h
        let mut ticker = tokio::time::interval(Duration::from_secs(60 * 60 * 24));
        loop {
            ticker.tick().await;
            match state.publications_service.sync_from_openalex().await {
                Ok(report) => tracing::info!(?report, "publication sync complete"),
                // never let a job failure kill the task -- log and wait for the next tick
                Err(e)     => tracing::error!(error = ?e, "publication sync failed"),
            }
        }
    });
}
```

Listing 13.5. `src/entrypoint/jobs/schedule.rs`. Note what the ordering buys.

Three ways an in-process job goes wrong

It runs once per worker if you spawn it in the factory. `HttpServer::new`'s closure runs once per worker thread — twelve times on this machine. Spawn scheduled work *beside* `HttpServer::new`, in `main`, exactly like the pool.

`interval` **fires immediately on the first tick.** That is usually what you want, but it means a deploy loop triggers a sync on every restart. Call `ticker.tick().await` once before the loop if you would rather skip the first.

A panic inside the task is silent. The task dies, the server keeps serving, and the job simply never runs again. Catch errors in the loop as above rather than using `?`.

Chapter 14

Services: when a repository is not enough

14.1 You have already written one — it is just wearing a `main`

`src/bin/seed_publications.rs` is the best example in the codebase of work that needs a home, so it is worth reading as an architecture question rather than as a script. Stripped of noise, it does four distinct things:

```rust
#[tokio::main]
async fn main() {
    let pool = get_connection_pool();                          // 1. compose
    let mut repo = PublicationsRepository::new(pool);

    let response = reqwest::get(PUBLICATION_LOOKUP_URL)        // 2. talk to OpenAlex
        .await.unwrap().text().await.unwrap();
    let scientist_response: ScientistResponse =
        serde_json::from_str(&response).unwrap();               // 3. parse THEIR shape

    scientist_response.results.into_iter().for_each(|article| {
        let insert_article: PublicationItemDto = article.into(); // 4. map to OUR shape
        repo.create_article(insert_article).unwrap()             //    and persist, one at a time
    });
}
```

Listing 14.1. `src/bin/seed_publications.rs`, as written — four responsibilities in one `main`.

Steps 2, 3 and 4 are a *use case*: "synchronise my publications from OpenAlex". It is real application behaviour, it has nothing to do with being a command-line program, and right now it can only ever be run by typing a command. Three consequences:

- **It cannot be reused.** An admin "refresh now" button would have to copy it. A scheduled job would have to copy it. Neither can call it, because it is a `main`.
- **An executable owns your domain mapping.** `Article` and the `Into<PublicationItemDto>` impl live in `src/bin/`, so the definition of "how an OpenAlex work becomes one of my publications" is in a binary that the library cannot see.
- **It cannot be tested.** There is no seam to insert a fake HTTP response.

> *A repository answers "how do I store this". A service answers "what does the application* do*". The moment a piece of behaviour involves more than one collaborator — an HTTP client and a repository, or two repositories, or a rule that spans both — it is a service, and it needs a name and a file.*

14.2 The split: three files, one use case

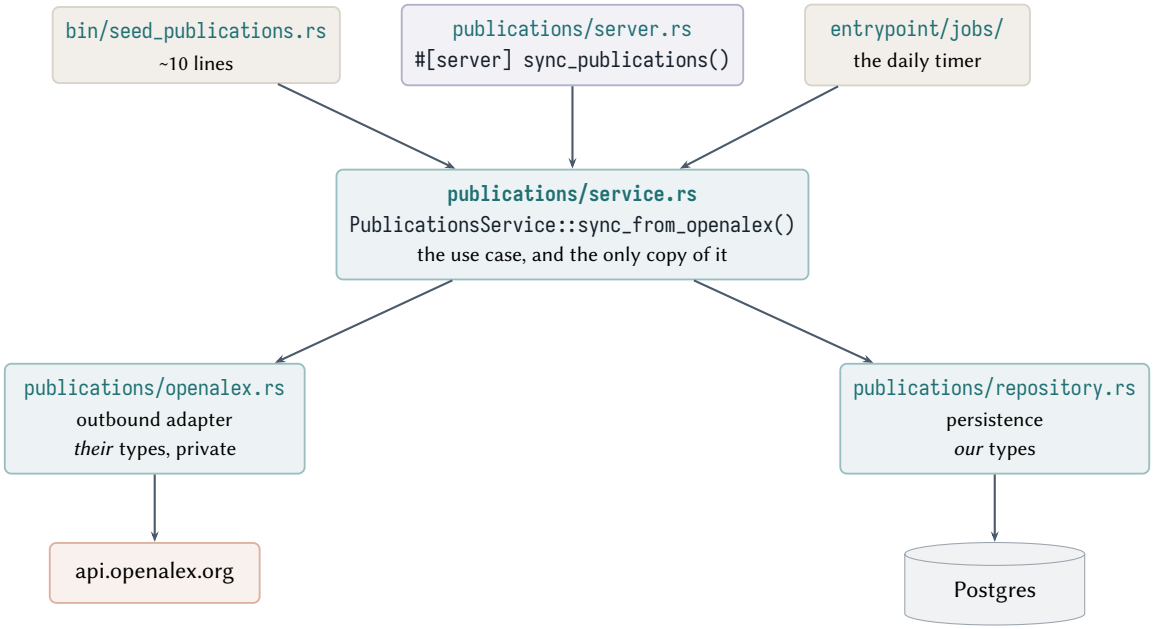


Figure 14.1. The seeder decomposed. The binary, a server function and a scheduled job all become thin callers of the same service.

14.2.1 `openalex.rs` — the outbound adapter

The types `ScientistResponse`, `Metadata` and `Article` describe *OpenAlex's* API, not your domain. They belong in one file, and they should be private to it, so that the rest of the codebase cannot accidentally start depending on a third party's field names.

```
use serde::Deserialize;
use crate::publications::dto::PublicationItemDto;

#[derive(Debug, Deserialize)]
struct WorksResponse { meta: Meta, results: Vec<Work> }     // private
#[derive(Debug, Deserialize)]
struct Meta { count: i32 }
#[derive(Debug, Deserialize)]
struct Work {
    id: String,                    // stable OpenAlex URI -- see the note on keys below
    title: Option<String>,         // OpenAlex really does return nulls here
    publication_year: Option<i32>,
    doi: Option<String>,
}

// From, not Into. You get Into free, and clippy stops complaining.
impl From<Work> for PublicationItemDto { /* ... */ }

#[derive(Debug, Clone)]
pub struct OpenAlexClient { http: reqwest::Client, author_id: String }

impl OpenAlexClient {
    pub fn new(author_id: impl Into<String>) -> Self {
        Self { http: reqwest::Client::new(), author_id: author_id.into() }
    }

    /// Returns publications in OUR shape. Callers never see a `Work`.
    pub async fn works_by_author(&self) -> Result<Vec<PublicationItemDto>, SyncError> {
        let url = format!(
            "https://api.openalex.org/works?filter=authorships.author.id:{}\
             &select=id,title,publication_year,doi&per-page=200",
            self.author_id
```

```
        );
        let body: WorksResponse = self.http.get(url).send().await?.json().await?;
        Ok(body.results.into_iter().map(Into::into).collect())
    }
}
```

Listing 14.2. src/publications/openalex.rs

Two latent bugs in the current fetch, both in the type declarations

`Article` declares `title: String`, `doi: String` and `publication_year: i32` — all required. OpenAlex returns `null` for any of them: works without a DOI are common, and untitled records exist. When one appears, `serde_json::from_str` fails, and the `.unwrap()` on the next line aborts the whole sync — having already inserted the works it processed before, since there is no transaction. Declare them `Option<...>` and decide per field whether to default or skip.

Second: `meta.count` is parsed and never read, while `per-page=200` caps the response. If you ever publish a 201st paper the sync will silently start truncating. Either assert `count ≤ 200` and fail loudly, or paginate with `&page=N`.

Key the sync on OpenAlex's `id`, not the DOI

The query already selects `id`, which is a stable OpenAlex URI present on every record. The DOI is absent on some and can change. Store the `id` in a dedicated `external_id TEXT UNIQUE` column and make that the conflict target in §14.4. This is what turns a re-run from "duplicate everything" into "update what changed".

14.2.2 `service.rs` — the use case

```
use crate::publications::{dto::PublicationItemDto, openalex::OpenAlexClient,
                          repository::PublicationsRepository};

#[derive(Debug, Clone)]                    // cheap: both fields are Arc-backed
pub struct PublicationsService {
    repo: PublicationsRepository,
    openalex: OpenAlexClient,
}

#[derive(Debug)]
pub struct SyncReport { pub fetched: usize, pub written: usize }

impl PublicationsService {
    pub fn new(repo: PublicationsRepository, openalex: OpenAlexClient) -> Self {
        Self { repo, openalex }
    }

    /// The use case. One copy, three callers.
    pub async fn sync_from_openalex(&self) -> Result<SyncReport, SyncError> {
        let incoming = self.openalex.works_by_author().await?;
        let written = self.repo.upsert_many(&incoming).await?;   // ONE round trip, idempotent
        Ok(SyncReport { fetched: incoming.len(), written })
    }

    pub async fn list_newest_first(&self) -> Result<Vec<PublicationItemDto>, SyncError> {
        Ok(self.repo.list_newest_first().await?)
    }
```

```
}
```

Listing 14.3. src/publications/service.rs

Note what the service does *not* do: it does not build a pool, read an environment variable, or know that it might be called from a command line. It receives its collaborators and exposes behaviour. That is the whole pattern.

14.2.3 The three callers

```rust
// src/bin/seed_publications.rs
#[actix_web::main]
async fn main() -> anyhow::Result<()> {
    dotenvy::dotenv().ok();
    tracing_subscriber::fmt::init();

    let cfg   = AppConfig::load()?;
    let pool  = build_pool(&cfg)?;
    let svc   = PublicationsService::new(
        PublicationsRepository::new(pool),
        OpenAlexClient::new(cfg.openalex_author_id),
    );

    let report = svc.sync_from_openalex().await?;   // no unwrap; `?` to a Result main
    tracing::info!(?report, "sync complete");
    Ok(())
}
```

Listing 14.4. The binary becomes an adapter: compose, call, report, exit.

```rust
// src/publications/server.rs
#[server]
pub async fn sync_publications() -> Result<usize, ServerFnError> {
    use crate::entrypoint::application_state::AppState;
    use actix_web::web::Data;
    let state: Data<AppState> = leptos_actix::extract().await?;
    let report = state.publications_service.sync_from_openalex().await?;
    Ok(report.written)
}
```

Listing 14.5. And the server function, for an admin “refresh” button.

If you expose that as a server function, it is a public endpoint

> Anyone can `curl` it, repeatedly, and each call hits OpenAlex. Either keep the sync out of the web process entirely (the systemd timer), or put an authorisation check in the body and rate-limit it. This is the concrete form of the warning in §19.

14.3 Does every slice need a service?

No, and adding one everywhere would be ceremony. The test is whether there is behaviour to name.

So: `publications` earns a service today, because of OpenAlex. `portfolio` will earn one when writes arrive, because an item and its tags must be inserted together. `personal_information` may never need one.

Table 14.1. When a server function may talk to a repository directly.

Server fn calls the repository directly	Server fn calls a service
one query, no decisions — "list the publications", "get this portfolio item by slug"	external I/O (OpenAlex, e-mail, an image resizer)
the mapping is a mechanical `From`	more than one repository, or one repository more than once
	anything that must be atomic across tables (portfolio item + its tags)
	an invariant that is not a database constraint
	anything that needs to run from more than one entry point

14.3.1 Where services live in `AppState`

Once a slice has a service, `AppState` should hold the *service* and the service should hold the repository — rather than exposing both and letting call sites choose.

```rust
#[derive(Debug, Clone)]
pub struct AppState {
    pub publications: PublicationsService,          // has a service: expose the service
    pub portfolio_items_repository: PortfolioItemsRepository,   // no service yet: repository is fine
    pub job_experience_repository: JobExperienceItemRepository,
    pub personal_information_repository: PersonalInformationRepository,
}

impl AppState {
    pub fn new(pool: DbPool, cfg: &AppConfig) -> Self {
        Self {
            publications: PublicationsService::new(
                PublicationsRepository::new(pool.clone()),
                OpenAlexClient::new(cfg.openalex_author_id.clone()),
            ),
            portfolio_items_repository: PortfolioItemsRepository::new(pool.clone()),
            job_experience_repository: JobExperienceItemRepository::new(pool.clone()),
            personal_information_repository: PersonalInformationRepository::new(pool),
        }
    }
}
```

Listing 14.6. `application_state.rs`, evolved. Same shape you already built, one layer up.

The migration is incremental and does not require a rewrite: add services one slice at a time, and when a slice has one, stop exposing its repository.

14.4 Making the sync idempotent

This is the part that matters most in practice, because the seeder is going to be run more than once.

Verified: a second run duplicates everything

```
$ psql -d my-site -tAc 'select count(*), count(distinct title), count(distinct link)
                        from publication_items'
8|7|8

$ psql -d my-site -c '\d publication_items'
```

```
Indexes:
    "publication_items_pkey" PRIMARY KEY, btree (id)     <-- and nothing else
```

Eight rows, no unique constraint on `title` or `link`, and `insert_into(...).values(...)` with no `on_conflict`. Nothing prevents the next run from inserting all eight again. (Note also that seven distinct titles across eight rows means two records already share a title — so `title` is not a usable key even if you constrained it.)

Two changes make the sync safe to run on a timer. First, give the table a real external key:

```
ALTER TABLE publication_items ADD COLUMN external_id TEXT;
UPDATE publication_items SET external_id = link WHERE external_id IS NULL;  -- backfill
ALTER TABLE publication_items ALTER COLUMN external_id SET NOT NULL;
ALTER TABLE publication_items ADD CONSTRAINT publication_items_external_id_key
  UNIQUE (external_id);
```

Second, make the write an upsert — and a single one, rather than one statement per row:

```
use diesel::upsert::excluded;

pub async fn upsert_many(&self, items: &[PublicationItemDto]) -> Result<usize, RepoError> {
    use crate::schema::publication_items::dsl::*;
    let rows: Vec<PublicationItemRow> = items.iter().cloned().map(Into::into).collect();
    let pool = self.pool.clone();

    let n = actix_web::web::block(move || {
        let mut conn = pool.get()?;                // inside the closure -- Ch. 10
        diesel::insert_into(publication_items)
            .values(&rows)                         // ONE statement for all rows
            .on_conflict(external_id)
            .do_update()
            .set((
                title.eq(excluded(title)),
                year.eq(excluded(year)),
                journal.eq(excluded(journal)),
                link.eq(excluded(link)),
            ))
            .execute(&mut conn)
    })
    .await??;

    Ok(n)
}
```

Listing 14.7. `repository.rs`: one round trip, idempotent, and no `&mut self`.

Why upsert rather than "delete all, then insert"?

Truncate-and-reload is tempting and looks simpler. It is worse for three reasons: it briefly empties the table, so a page load during the sync renders nothing; it destroys any local edits, which matters as soon as you hand-write an abstract OpenAlex does not have; and it churns the primary keys, which breaks any URL or foreign key pointing at a publication. An upsert keyed on the external id preserves identity, which is the property you actually want.

Note also the batch: one `insert_into(...).values(&rows)` is a single statement and a single network round trip, where the current `for_each` does one insert per publication. At eight rows this is invisible; it is the difference between a sync that takes 20 ms and one that takes two seconds when you have 200 papers.

Part IV

Data

Chapter 15

The data model, as it is and as it should be

15.1 What the four migrations built

Figure 15.1 is the schema currently in the database, read back from Postgres rather than from the migration files, so it reflects what actually exists.

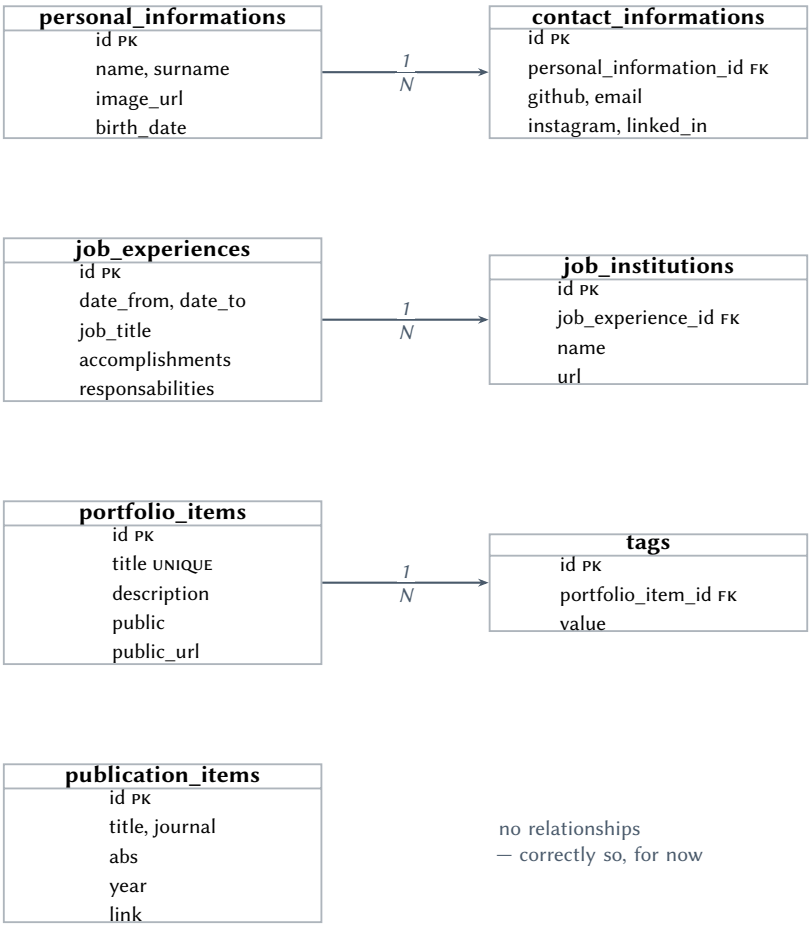


Figure 15.1. The current schema. The three relationships are all modelled as one-to-many with the foreign key on the child; two of the three should not be. Every foreign-key column carries its own sequence (see §15.4).

The structure is sound in outline: four content types, each with the fields a personal site actually needs. What follows is not a rejection of it. It is the set of decisions that are cheap to change now, while every table is empty, and expensive to change once there is content and a deployed site.

15.2 Three modelling decisions to revisit

15.2.1 `contact_informations` is one-to-one pretending to be one-to-many

There is one of you. There will be one row in `personal_informations` and one row in `contact_informations`. The schema permits four contact rows for the same person, and nothing stops it, so every query that reads contact details has to decide what to do about the possibility — `get_result` will error on multiple rows, `get_results` forces the caller to handle a `Vec` that should always have length one.

There is also a second, quieter problem: the columns are the platforms. `github`, `email`, `instagram`, `linked_in`. Adding Mastodon means a migration, a schema regeneration, a model change, a DTO change and a component change. Removing Instagram means the same in reverse, or a dead column forever.

Make contact links rows, not columns

```
CREATE TABLE contact_links (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  person_id   integer NOT NULL REFERENCES people(id) ON DELETE CASCADE,
  kind        text    NOT NULL,          -- 'github' | 'email' | 'linkedin' | ...
  label       text    NOT NULL,          -- what to show: "@egonik"
  url         text    NOT NULL,
  position    integer NOT NULL DEFAULT 0, -- display order
  UNIQUE (person_id, kind)
);
CREATE INDEX contact_links_person_idx ON contact_links (person_id, position);
```

Now a new platform is one `INSERT`. The component iterates rows and renders an icon chosen from `kind`, and adding a platform never touches Rust.

For the person table itself, if there is genuinely only ever one row, say so in the schema rather than hoping:

```
-- a singleton table: exactly one row, enforced
CREATE TABLE people (
  id         integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  ...
  CONSTRAINT people_singleton CHECK (id = 1)
);
```

15.2.2 `job_institutions` points the wrong way

This is the one worth changing before there is data, because the fix touches a foreign key.

As modelled, one `job_experience` has many `job_institutions`, and each institution belongs to exactly one experience. Read that as a sentence about the world: *one job was held at several companies, and each company hosted exactly one job.* That is backwards. A job is held at one institution, and an institution can host several jobs — which is what happens the second time you are promoted, or return to a former employer.

The practical consequence is duplication. Work at Snappler twice and you get two `job_institutions` rows both named “Snappler”, both with the same URL, and no way to know they are the same organisation. Fix the URL and you fix it once, in one of two places.

Institutions are their own entity

```
CREATE TABLE institutions (
  id   integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name text NOT NULL UNIQUE,
  url  text
);

ALTER TABLE job_experiences
  ADD COLUMN institution_id integer NOT NULL REFERENCES institutions(id) ON DELETE RESTRICT;
CREATE INDEX job_experiences_institution_idx ON job_experiences (institution_id);
```

`ON DELETE RESTRICT` rather than `CASCADE`: deleting an institution should not silently delete your employment history. The right answer is to refuse.

15.2.3 `tags` should be many-to-many

`tags(id, portfolio_item_id, value)` stores the tag *text* once per item. Tag three projects with "rust" and the string "rust" exists three times. Consequences: no canonical list of tags to render a filter UI from, no way to rename a tag in one place, and typos ("leptos"/"Leptos") that silently become two different tags.

The standard join-table shape

```
CREATE TABLE tags (
  id    integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  slug  text NOT NULL UNIQUE,            -- 'rust'      -- for URLs
  label text NOT NULL                    -- 'Rust'      -- for display
);

CREATE TABLE portfolio_item_tags (
  portfolio_item_id integer NOT NULL REFERENCES portfolio_items(id) ON DELETE CASCADE,
  tag_id            integer NOT NULL REFERENCES tags(id)            ON DELETE CASCADE,
  PRIMARY KEY (portfolio_item_id, tag_id)
);
CREATE INDEX portfolio_item_tags_tag_idx ON portfolio_item_tags (tag_id);
```

The composite primary key makes duplicate tagging impossible, and the extra index makes "all projects tagged rust" fast in the other direction. Diesel handles this shape well — `belonging_to` plus a grouped load, which is the standard pattern in its guides.

15.3 Column-level corrections

Use `GENERATED ALWAYS AS IDENTITY`, not `SERIAL`

Every table currently uses `SERIAL` for its primary key, which is the older mechanism: `SERIAL` is shorthand for "integer, plus a sequence, plus a default", and the sequence is only loosely attached to the column. Identity columns are the SQL-standard replacement, are owned properly by the column, and — crucially for the next section — cannot be applied by accident to a column that should not have a default.

Table 15.1. Column changes worth making while the tables are empty.

Now	Should be	Why
`responsabilities`	`responsibilities`	Misspelling. It is about to become a column name, a struct field, a JSON key and part of a public API. Free to fix today. **D15**
`abs TEXT`	`abstract_text`	`abs` reads as the SQL `ABS()` function. Not reserved, so it works — but nobody will guess what it holds. **D15**
`accomplishments VARCHAR(255)`	child table, or `TEXT`	255 characters is one sentence. Accomplishments are a list. A `job_highlights` child table with `(kind, position, body)` lets you render bullets. **D16**
`responsabilities VARCHAR(255)`	same	As above.
`public BOOLEAN`	`visibility` enum, or keep	Two-state today, three tomorrow (draft / unlisted / public). An enum type or a `text` + `CHECK` avoids a second boolean.
`year INTEGER`	keep	Correct. A publication year is a year, not a date.
—	`slug TEXT NOT NULL UNIQUE`	On every content table. You want `/portfolio/egonik-site`, not `/portfolio/7`. Adding this later means either breaking URLs or maintaining both.
—	`position INTEGER NOT NULL DEFAULT 0`	Every list on the site has an order you care about, and it is never "by primary key".
—	`created_at`, `updated_at` `TIMESTAMPTZ`	Cheap now, impossible to backfill later. Diesel's initial-setup migration already installed `diesel_manage_updated_at()` for exactly this and it is unused.

15.4 The SERIAL foreign key, which is a live bug

All three foreign-key columns are declared SERIAL:

```
portfolio_item_id       serial not null,   -- tags
job_experience_id       serial,            -- job_institutions
personal_information_id serial not null,   -- contact_informations
```

Listing 15.1. From the three `up.sql` files.

The intent was "an integer that references another table". What SERIAL actually does is create a *dedicated sequence for this column* and set it as the column default. The consequence is that a foreign key that should always be supplied explicitly instead has a default value that counts up on its own, entirely independently of the parent table.

Verified: the real column definitions in the live database

```
$ psql -d my-site -c '\d tags'
      Column       |  Type   | Nullable |                    Default
-------------------+---------+----------+-----------------------------------------------
 id                | integer | not null | nextval('tags_id_seq'::regclass)
 portfolio_item_id | integer | not null | nextval('tags_portfolio_item_id_seq'::regclass)
 value             | varchar |          |
```

There is a `tags_portfolio_item_id_seq`. A sequence, on a foreign key.

That is not merely untidy. It converts a mistake that Postgres would have caught into silent data corruption:

Verified: silent mis-attachment, then a confusing error

```
INSERT INTO portfolio_items (title, description, public) VALUES ('Project A','desc',true);
INSERT INTO portfolio_items (title, description, public) VALUES ('Project B','desc',true);

-- the author forgets the foreign key. There is a DEFAULT, so this SUCCEEDS:
INSERT INTO tags (value) VALUES ('rust');      -- INSERT 0 1
INSERT INTO tags (value) VALUES ('leptos');    -- INSERT 0 1

SELECT t.value, p.title AS silently_attached_to FROM tags t
  JOIN portfolio_items p ON p.id = t.portfolio_item_id;

  value  | silently_attached_to
 --------+----------------------
  rust   | Project A               <-- wrong, and no error was raised
  leptos | Project B               <-- wrong

INSERT INTO tags (value) VALUES ('this-one-explodes');
ERROR:  insert or update on table "tags" violates foreign key constraint
DETAIL:  Key (portfolio_item_id)=(3) is not present in table "portfolio_items".
```

Read that sequence of events carefully, because it is the worst shape a bug can have. The first two inserts are wrong and succeed. The third fails — with an error that points at the foreign key, which is where you will look, but the actual cause is a default on a column three lines up in a migration written two days earlier. Had the column been a plain `integer NOT NULL`, the very first insert would have failed immediately with `null value in column "portfolio_item_id" violates not-null constraint`, which is a message that leads directly to the mistake.

This is defect **D1**, and it is the highest-priority item in this document. The fix is mechanical:

```
ALTER TABLE tags               ALTER COLUMN portfolio_item_id      DROP DEFAULT;
```

```
ALTER TABLE job_institutions     ALTER COLUMN job_experience_id       DROP DEFAULT;
ALTER TABLE contact_informations ALTER COLUMN personal_information_id DROP DEFAULT;

DROP SEQUENCE IF EXISTS tags_portfolio_item_id_seq;
DROP SEQUENCE IF EXISTS job_institutions_job_experience_id_seq;
DROP SEQUENCE IF EXISTS contact_informations_personal_information_id_seq;

-- job_institutions.job_experience_id is also nullable, which it should not be
ALTER TABLE job_institutions ALTER COLUMN job_experience_id SET NOT NULL;
```

Listing 15.2. A migration that removes the three stray defaults and drops the sequences.

Because all four tables are empty, the honest alternative is better: rewrite the four `up.sql` files correctly, drop the database, and re-run from scratch. Section 16.4 explains when that is legitimate.

15.5 Missing constraints and indexes

Postgres will enforce whatever you tell it to and nothing more. The current schema tells it very little. This checklist is worth running against every future migration.

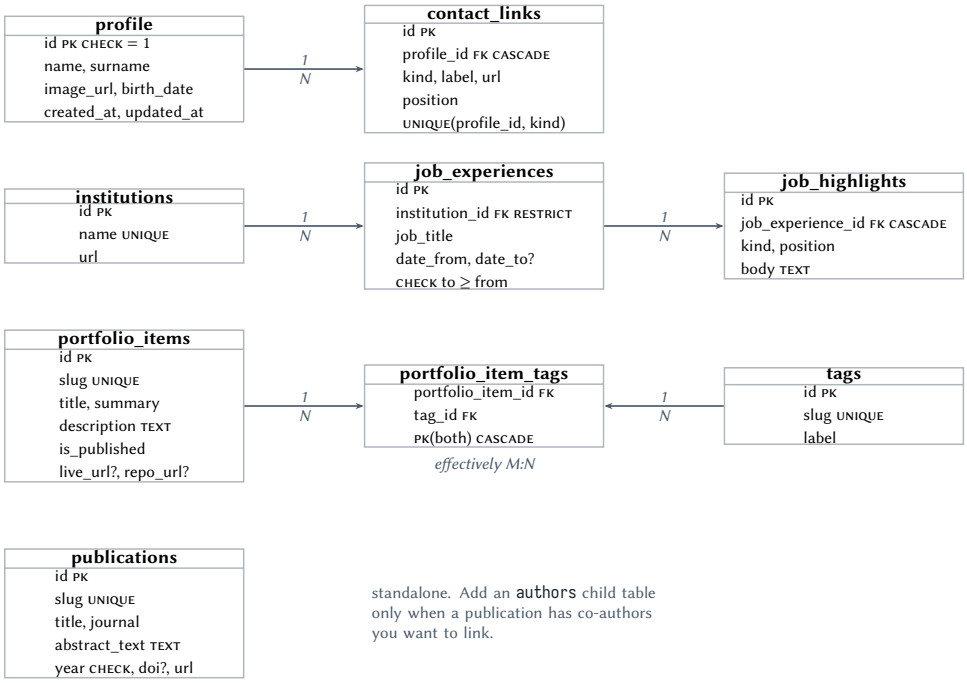
Table 15.2. Constraint checklist, with the current state of each item.

Constraint	Now	Why it earns its place
Index on every FK column	none	Postgres indexes the *referenced* key automatically but never the referencing column. Be honest about the reason: at a few dozen rows Postgres will seq-scan whichever way you index it, and should. The argument is that `ON DELETE CASCADE` takes row locks on the child scan, and that the index costs nothing.
`ON DELETE` on every FK	none	The default is `NO ACTION`, so deleting a portfolio item with tags fails with a foreign-key error. Decide deliberately: `CASCADE` for tags, `RESTRICT` for institutions.
`CHECK (date_to IS NULL OR date_to ≥ date_from)`	no	A job that ended before it began will render as a negative duration. One line prevents it forever.
`CHECK (year BETWEEN 1900 AND 2100)`	no	Catches a typo'd year at insert time rather than in a rendered page.
`UNIQUE` on `slug`	n/a	Once slugs exist, this is what makes a URL a key.
`NOT NULL` where meant	partial	`job_institutions.job_experience_id` is nullable — an institution attached to no job. Almost certainly unintended.
`title UNIQUE` on `portfolio_items`	yes	Already there. Good — though once slugs exist, the slug is the better uniqueness key and the title constraint can relax.

15.6 The target schema, decided

The sections above argue for individual changes in isolation, which is how you evaluate them and a bad way to apply them. Here is the whole thing decided at once, so that M0 in Chapter 26 is a single coherent migration rather than a dozen negotiations.

Five renames are folded in, and each is here because the current name is either wrong or ambiguous:



Every table also carries `created_at TIMESTAMPTZ NOT NULL DEFAULT now()` *and* `updated_at`*, and every primary key is* `GENERATED ALWAYS AS IDENTITY`*. Every FK column has an index.*

Figure 15.2. The target schema. Every relationship has an explicit cardinality and referential action; every content table has a `slug`, a `position` and timestamps. Compare with Figure 15.1.

- `personal_informations` → `profile`. The old name is ungrammatical ("informations") and plural for a table that holds one row.
- `contact_informations` → `contact_links`, restructured from columns to rows.
- `publication_items` → `publications`. The `_item` suffix carries no information; `portfolio_items` keeps it because a portfolio *is* a collection of items.
- `abs` → `abstract_text`; `responsabilities` → a `kind` value in `job_highlights`.
- `public` / `public_url` → `is_published` / `live_url`. Those two columns share a prefix and mean unrelated things — one is a visibility flag, the other is a hyperlink — and the shared prefix invites exactly the wrong mental model.

Get here in one migration, not twelve

Because every table is empty (Chapter 2), this is not a migration sequence — it is a rewrite of the four existing `up.sql` files plus `diesel database reset`. Do not write twelve `ALTER TABLE` migrations to reach a schema that has never held a row. §16.4 sets out when that shortcut is legitimate, and why the door closes the day you deploy.

Chapter 16

Migration discipline

16.1 The rule

> *A migration is not "the SQL that got the schema into the state I wanted". It is a* pair *of scripts, and the pair is only correct when* `up` *followed by* `down` *returns the database to exactly where it started. If* `down` *does not work, you do not have a migration — you have a one-way change with a file next to it.*

Right now `down` does not work. Not for one migration — for three of the four, and in a way that makes the whole chain unrevertible.

16.2 Verified: the chain cannot be rolled back

Verified on a scratch database

A fresh database was created, all migrations applied cleanly, and then reverted one at a time:

```
$ diesel migration run
Running migration 2026-08-02-211314-0000_create_portfolio_item
Running migration 2026-08-03-231439-0000_create_publications_item
Running migration 2026-08-03-232042-0000_create_job_experience
Running migration 2026-08-03-233332-0000_create_personal_information

$ diesel migration revert
Rolling back migration 2026-08-03-233332-0000_create_personal_information
ERROR diesel::database: Failed to execute query
  query="drop table personal_informations;\ndrop table contact_informations;\n"
  err=DatabaseError(Unknown,
      "cannot drop table personal_informations because other objects depend on it")
Failed to run migrations: Failed to run 2026-08-03-233332-0000_create_personal_information
  with: cannot drop table personal_informations because other objects depend on it
```

Running it four more times produces the identical error. The chain is stuck at the top: because the most recent migration cannot be reverted, *none* of the earlier ones can be reached.

16.2.1 Cause 1: drops in the wrong order

```
drop table personal_informations;      -- <-- the PARENT, dropped first
drop table contact_informations;       -- <-- the CHILD, which still references it
```

Listing 16.1. `2026-08-03-233332-0000_create_personal_information/down.sql`

`contact_informations` holds a foreign key into `personal_informations`, so the parent cannot be dropped while the child exists. `DROP` order must be the reverse of `CREATE` order: children first, parents last. The `create_job_experience` migration has the identical bug. Interestingly, `create_portfolio_item` gets it right — `DROP table tags` before `DROP TABLE portfolio_items` — which shows the correct order was understood once and then lost. Defect **D3**.

16.2.2 Cause 2: a typo'd table name

```
drop table publication_itens
```

Listing 16.2. 2026-08-03-231439-0000_create_publications_item/down.sql

Verified on this machine

```
$ psql -d scratch -f migrations/..._create_publications_item/down.sql
ERROR:  table "publication_itens" does not exist
```

`itens` is the Portuguese plural of *item*; the table is `publication_items`. A one-character slip that makes the migration unrevertible, and that no amount of testing the `up` direction would ever surface. Defect **D4**.

16.2.3 The corrected files

```
-- create_publications_item/down.sql
DROP TABLE IF EXISTS publication_items;

-- create_job_experience/down.sql
DROP TABLE IF EXISTS job_institutions;      -- child first
DROP TABLE IF EXISTS job_experiences;       -- then parent

-- create_personal_information/down.sql
DROP TABLE IF EXISTS contact_informations;  -- child first
DROP TABLE IF EXISTS personal_informations; -- then parent
```

Listing 16.3. All three `down.sql` files, fixed. `IF EXISTS` makes a partially-applied revert re-runnable.

16.3 The habit that would have caught all of this

Never write a migration without running `redo`

`diesel migration redo` reverts the most recent migration and immediately re-applies it. It is one command, it takes a second, and it exercises the `down` path that you will otherwise discover is broken on the day you actually need it.

```
diesel migration generate create_thing     # writes up.sql and down.sql
$EDITOR migrations/*_create_thing/{up,down}.sql
diesel migration run                       # apply
diesel migration redo                      # <-- the step that was skipped
diesel print-schema > src/schema.rs        # regenerate, then commit both together
```

Make the `redo` step non-negotiable. Every defect in this chapter would have been caught by it within seconds of being written.

`redo` on a migration whose `down` is broken leaves you stuck

Diesel runs each migration in a transaction, so a failed revert rolls back and the database is unharmed. But the migration stays applied and `revert` will keep failing, so you cannot reach any earlier migration until you fix the `down.sql`. That is the state this repository is in right

now.

16.4 When to rewrite history instead

Normally, editing an already-applied migration is forbidden — other people's databases and your production database have recorded that it ran, and changing it silently desynchronises them.

That reasoning does not apply here, and it is worth saying so explicitly rather than following the rule out of habit. There is exactly one database, on your machine, with zero rows in every table, and one commit in the repository. Nothing has been deployed. Nobody else has cloned it.

Reset now; never again after the first deploy

```
# 1. fix all four up.sql (SERIAL -> integer on FKs, constraints, indexes,
#    naming, created_at/updated_at) and all four down.sql (order, typo)
# 2. throw the database away and rebuild it from the fixed migrations
diesel database reset          # drop + create + run all migrations
# 3. prove the pair is symmetric, for every migration, not just the last
diesel migration redo -a       # -a = redo all
# 4. regenerate the schema and commit migrations + schema.rs together
diesel print-schema > src/schema.rs
```

After the site is deployed once, this door closes. From then on every schema change is a new migration, forward-only, and the `down` script exists for your development loop rather than for production rollback.

`schema.rs` is generated — never hand-edit it

`src/schema.rs` carries the header `// @generated automatically by Diesel CLI`. It is the compiler's model of your tables, and if it drifts from the database you get type errors that describe a schema nobody has. Regenerate it after every migration, in the same commit as the migration, so that a single `git show` tells the whole story of a schema change.

16.5 Housekeeping in the migrations directory

Three small things, all defect-register entries:

`migrations/.diesel_lock` is untracked and unignored.
It is an ephemeral lock file created while the CLI runs. Add it to `.gitignore`. D21

The timestamps carry a `-0000` suffix.
Diesel's own format is `YYYY-MM-DD-HHMMSS_name`; yours are `2026-08-02-211314-0000_create_portfolio_item`. Harmless — ordering still works — but let `diesel migration generate` name the directories so they stay consistent. D22

Names should be plural and consistent.
`create_portfolio_item` creates `portfolio_items` *and* `tags`; `create_publications_item` mixes plural and singular in one name. Pick "one migration named after what it creates" and stick to it. D22

Chapter 17

Content: seeding, and who writes it

17.1 Seed before you render

Every table is empty. That matters more than it sounds, because **an empty list and a broken query render identically**. If you build the publications page against an empty table you cannot tell whether it works, and the first time you insert a row you will be debugging the page and the query at once.

A seed migration, or a seed binary

Two reasonable mechanisms, and the choice depends on whether the content *is* the data or merely stands in for it.

For real content — your actual CV — use an ordinary migration. It is versioned, reviewed, applied identically everywhere, and rolls back with the schema. Your employment history changes about once a year; a migration per change is entirely proportionate.

For development fixtures — ten fake portfolio items to test pagination — use a small binary at `src/bin/seed.rs`, run with `cargo run --bin seed --features ssr`. Keep it out of the migration chain so it never runs in production, and make it idempotent so you can run it twice.

```
INSERT INTO people (id, name, surname, image_url, birth_date)
VALUES (1, 'Eduardo', 'Gonik', '/assets/me.jpg', '1990-01-01');

INSERT INTO contact_links (person_id, kind, label, url, position) VALUES
  (1, 'github',   '@egonik',        'https://github.com/egonik',           0),
  (1, 'email',    'ia@snappler.com','mailto:ia@snappler.com',              1),
  (1, 'linkedin', 'Eduardo Gonik',  'https://linkedin.com/in/...',         2);
```

Listing 17.1. A seed migration, with explicit foreign keys — which is what defect D1 currently lets you forget.

17.2 The authoring question you have not answered yet

There is a decision waiting here that is easy to defer past the point where it is cheap. Every repository in the codebase is read-only: `get_all` and `get_by_id`. So how does content get in?

Three answers, in increasing order of cost:

SQL migrations, forever.
Content lives in the repository as SQL. Deploying is how you publish. No admin UI, no authentication, no attack surface. For a personal site updated a few times a year this is not a compromise — it is arguably the correct answer, and it is what a static-site generator would give you with more machinery.

Markdown files in the repository, read at startup.
Content lives in `content/*.md` and is loaded into the database (or straight into memory) on boot.

You get to write prose in an editor instead of inside a SQL string literal, and Postgres holds only what benefits from being queryable. A good fit if the site grows a blog.

An admin UI behind authentication.
Now you need write repositories, `Action`s and `<ActionForm>`s, session management, password hashing, CSRF considerations, and a permanent obligation to keep all of it patched. Worth it only if you will genuinely edit content from a phone.

> **Choose the first, and revisit only when it hurts**
>
> Start with SQL migrations. The reason is not laziness — it is that server functions are public HTTP endpoints. The moment a write path exists, it exists for everyone on the internet unless every one of those functions checks authorisation on every call. That is a real, ongoing obligation, and it is a strange one to take on for a site whose content changes annually.
>
> The decision that matters is not which of the three you pick today. It is that you pick *deliberately*, and that if you defer, you defer knowing you have deferred — rather than discovering in six months that there is no way to add a project without hand-writing SQL.

Part V

The front end

Chapter 18

Pages, routes, and layout

18.1 Decide the URLs before you write the components

URLs are the part of a website that is hardest to change later, because other people's links point at them. They are also the cheapest thing to get right today, since there are none yet apart from /.

Table 18.1. A route plan for this site. The `:slug` routes are the reason Chapter 15 argues for a `slug` column.

Path	Reads from	Note
/	`people`, `contact_links`	who you are; the only page that must load fast
`/portfolio`	`portfolio_items` + tags	the list; filterable by tag
`/portfolio/:slug`	one `portfolio_item`	the detail page; must 404 properly
`/publications`	`publication_items`	grouped by year, newest first
`/experience`	`job_experiences` + institutions	the CV, reverse chronological
`/tags/:slug`	join table	optional; only if tags earn a page of their own
`*any`	—	404, with a real 404 status

Slugs, not integers

`/portfolio/egonik-site` is a URL you can put in a CV. `/portfolio/7` is a URL that changes meaning if you ever re-seed the database. Add the `slug` column in the same migration that fixes defect D1, before there is any content to migrate.

18.2 Layout: one shell, many pages

Every page shares a header, navigation and footer. Leptos expresses that with a nested route: a parent route renders the shell and an `<Outlet/>`, and the children render into it.

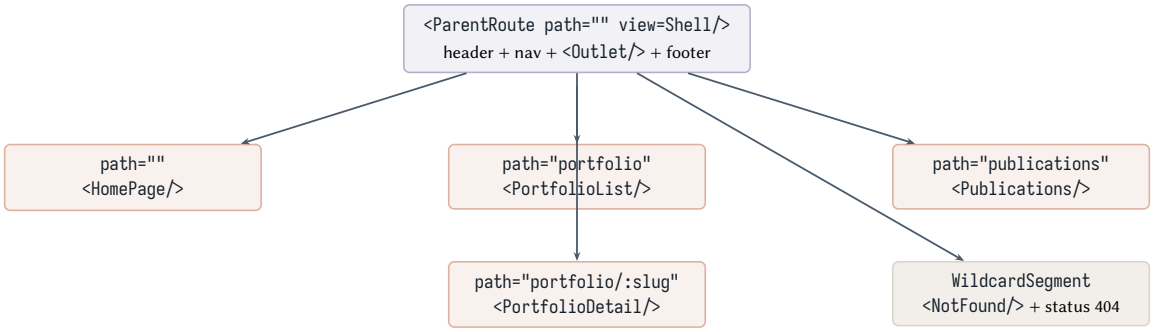


Figure 18.1. The route tree. `ParentRoute` renders the shell once; only the `<Outlet/>` contents change on navigation, so the header is never re-rendered and never flashes.

```
use leptos::prelude::*;
use leptos_meta::{provide_meta_context, Meta, Stylesheet, Title};
use leptos_router::components::{Outlet, ParentRoute, Route, Router, Routes};
use leptos_router::{ParamSegment, StaticSegment, WildcardSegment};

use crate::ui::layout::Shell;
use crate::ui::pages::{HomePage, NotFound, PortfolioDetail, PortfolioList, Publications, Experience};

#[component]
pub fn App() -> impl IntoView {
    provide_meta_context();

    view! {
        <Stylesheet id="leptos" href="/pkg/egonik-site.css"/>
        <Title text="Eduardo Gonik"/>
        <Meta name="description" content="Personal site: portfolio, publications, experience."/>

        <Router>
            <Routes fallback=NotFound>
                <ParentRoute path=StaticSegment("") view=Shell>
                    <Route path=StaticSegment("")             view=HomePage/>
                    <Route path=StaticSegment("portfolio")    view=PortfolioList/>
                    <Route path=(StaticSegment("portfolio"), ParamSegment("slug"))
                                                              view=PortfolioDetail/>
                    <Route path=StaticSegment("publications") view=Publications/>
                    <Route path=StaticSegment("experience")   view=Experience/>
                    <Route path=WildcardSegment("any")        view=NotFound/>
                </ParentRoute>
            </Routes>
        </Router>
    }
}
```

Listing 18.1. `src/app.rs` — the whole file, and it should stay roughly this size.

Use `<A>` for internal links, `<a>` for external ones

`leptos_router`'s `<A href="/portfolio">` intercepts the click and navigates client-side — journey 2 from Chapter 5. A plain `<a href="/portfolio">` triggers a full page load, throwing away the wasm bundle you just downloaded and re-running SSR. Both work; one is much slower. For links off your site, plain `<a>` is correct.

18.3 Per-page titles, and the SEO detail that is easy to miss

`leptos_meta`'s `<Title>` works inside any component, including a page, and the last one rendered wins. So each page sets its own:

```
#[component]
pub fn Publications() -> impl IntoView {
    view! {
        <Title text="Publications | Eduardo Gonik"/>
        // ...
    }
}
```

There is a subtlety when the title depends on *loaded data* — a portfolio detail page whose title is the project name. During server rendering, the HTML `<head>` is sent before the body finishes. If the title comes from a normal `Resource`, the head may already be on the wire by the time the data arrives, and the crawler sees the generic title.

`Resource::new_blocking` for data that feeds the head

`Resource::new_blocking` has the same signature as `Resource::new` but holds the HTTP response until the data has loaded. Use it for exactly the one resource whose value determines `<Title>`, `<Meta>` or the HTTP status — and for nothing else, because it costs you streaming.

```
// the project name is in the title and the meta description -> blocking
let item = Resource::new_blocking(move || slug(), |s| get_portfolio_item(s));
```

18.4 404 must be a real 404

The starter template already gets this right — verifiably, and this is worth checking rather than assuming, because the mechanism is easy to break later.

Verified against the running binary

```
$ curl -s -o /dev/null -w '%{http_code}\n' localhost:3000/nonexistent-path
404
$ curl -s localhost:3000/nonexistent-path | grep -o '<h1>[^<]*</h1>'
<h1>Not Found</h1>
```

So the 404 path works today. One subtlety, though: `app.rs` declares *both* a wildcard route and a `Routes` fallback:

```
<Routes fallback=move || "Not found.">                  // <-- unreachable
    <Route path=StaticSegment("") view=HomePage/>
    <Route path=WildcardSegment("any") view=NotFound/>  // <-- matches everything else first
</Routes>
```

The wildcard route matches every unmatched path, so the fallback string is dead code that will never render. That is harmless and misleading in equal measure — it looks like the 404 handler, and it is not. Delete it, or point it at `NotFound` so that the two agree.

`NotFound` sets the status inside a `cfg` block:

```
#[cfg(feature = "ssr")]
{
    let resp = expect_context::<leptos_actix::ResponseOptions>();
    resp.set_status(actix_web::http::StatusCode::NOT_FOUND);
}
```

The `cfg` is not defensive noise — `ResponseOptions` exists only during server rendering, because only then is there an HTTP response to modify. If a visitor navigates to a bad URL *after* hydration there is no new request to set a status on, which is a real and unavoidable asymmetry: the first load of a missing page returns 404, a client-side navigation to one does not. That is correct behaviour and matches how every SPA works.

The same mechanism applies to a `/portfolio/:slug` that does not exist, and it is the reason Chapter 11 insists on calling `set_status` in the error mapping layer. A detail page for a deleted project must return 404, or search engines will index it forever.

Chapter 19

Loading data into a page

19.1 The three resource types, and which one you want

This is the most common place to make a wrong choice that appears to work. All three types load async data; they differ in whether they participate in server rendering.

Table 19.1. Resource types in Leptos 0.8.

Type	Runs in SSR?	Constructor	Use it when
`Resource`	yes	`new(source, fetcher)`	the default. Reactive to its source; result is serialised into the HTML
`Resource ::new_blocking`	yes	same	the value determines `<Title>`, `<Meta>` or the HTTP status
`OnceResource`	yes	`new(future)`	loads exactly once, never re-fetches, no source signal
`LocalResource`	**no**	`new(fetcher)`	browser-only APIs, or a future that is not `Send`

`LocalResource` is the trap

Its documentation says it "will only begin loading data if you are on the client". Reach for it — perhaps because a `Send` bound was inconvenient — and your page silently stops being server-rendered: the HTML ships as an empty shell, the content appears only after the wasm loads, search engines see nothing, and none of it produces a warning. If you find yourself reaching for `LocalResource` to make a `Send` error go away, the real problem is almost always a non-`Send` value held across an `.await` inside the server function — typically a database connection. Fix that instead; Chapter 10 shows how.

A `Send` error that points at the wrong line

`Resource::new` requires the fetcher's future to be `Send` and `T` to be `Send + Sync + Serialize + DeserializeOwned`. If your server function holds a `PooledConnection` across an `.await`, the resulting `Send` violation surfaces at `Resource::new` — in a different file from the actual mistake. When a `Send` error names a resource, look inside the server function it calls.

19.2 The canonical read pattern

`Resource` + `Transition` + `ErrorBoundary` + `Suspend`

```
use leptos::prelude::*;
use leptos::either::Either;
use crate::publications::api::list_publications;
```

```rust
#[component]
pub fn Publications() -> impl IntoView {          // NOTE: pub
    let publications = Resource::new(|| (), |_| list_publications());

    view! {
        <Transition fallback=move || view! { <p class="text-slate-500">"Loading..."</p> }>
            <ErrorBoundary fallback=|errors| view! {
                <p class="text-red-700">
                    "Could not load publications: "
                    {move || errors.get()
                        .into_iter()
                        .map(|(_, e)| e.to_string())
                        .collect::<Vec<_>>()
                        .join("; ")}
                </p>
            }>
                {move || Suspend::new(async move {
                    publications.await.map(|rows| if rows.is_empty() {
                        Either::Left(view! { <p>"Nothing published yet."</p> })
                    } else {
                        Either::Right(rows.into_iter()
                            .map(|p| view! { <PublicationCard publication=p/> })
                            .collect_view())
                    })
                })}
            </ErrorBoundary>
        </Transition>
    }
}
```

Listing 19.1. `src/publications/components.rs` — the shape to grow the current placeholder into.

Five details in that snippet are load-bearing:

`Transition`, not `Suspense`.
`<Suspense>` shows its fallback every time any nested resource reloads, so a re-fetch makes the list flash back to "Loading…". `<Transition>` shows the fallback on first load only and keeps the stale content visible while new data arrives. For lists that refetch, `Transition` is almost always what you want.

`ErrorBoundary` goes *inside* the transition.
Not outside. The upstream Leptos example still carries a comment noting that the outside-in arrangement breaks `Suspense` under Actix. Inside-out works; take the ordering as given.

`Suspend::new` lets you `.await` the resource in the view.
The alternative is `publications.get()` returning `Option<Result<…>>` and a nest of matches. `Suspend` removes the `Option`, because inside a suspense boundary the framework guarantees the value is there.

`Either::Left` / `Either::Right`.
Two `view!` invocations produce two *different static types* in Leptos 0.8, so an `if`/`else` that returns markup will not compile without wrapping the arms. `Either` is the cheap fix; `.into_any()` is the type-erasing one, slightly slower and sometimes clearer.

Errors surface by themselves.
A `Result` rendered in a view renders the `Ok` value and, on `Err`, renders nothing while registering

the error with the nearest `ErrorBoundary`. You do not match on the error — you let it propagate to the boundary.

19.3 Writes: `Action`, not `Resource`

You have no writes yet. When you add them, the pairing is strict.

A mutation inside a `Resource`

A `Resource` runs during server rendering *and* may run again on the client. Put an `INSERT` in one and you will insert twice, and the second row will appear only in production where the timing differs. Reads are Resources; writes are Actions.

```
let add_item = ServerAction::<AddPortfolioItem>::new();

// the resource re-runs whenever the action's version changes
let items = Resource::new(
    move || add_item.version().get(),
    |_| list_portfolio(),
);

view! {
    <ActionForm action=add_item>
        <input type="text" name="title"/>
        <button type="submit">"Add"</button>
    </ActionForm>
}
```

Listing 19.2. A write, and the read that refreshes itself when the write lands.

Why `<ActionForm>` rather than an `on:click` handler

`<ActionForm>` degrades to an ordinary HTML `<form method="POST">`. With no JavaScript it still submits, the server function still runs, and `leptos_actix` notices that the request accepts `text/html` and responds with a 302 back to the referring page. You get progressive enhancement free. A click handler that calls the server function directly does not.

`Vec` parameters need `#[server(default)]`

The default URL encoding omits an empty `Vec` entirely, and deserialisation on the server then fails with a missing-argument error at runtime — not at compile time. Any server function taking a `Vec` needs the annotation:

```
#[server]
pub async fn set_tags(#[server(default)] tags: Vec<String>) -> Result<(), ServerFnError> { ... }
```

19.4 One thing to internalise about server functions

A server function is a public HTTP endpoint. Anyone can `curl` it. It is not a private mechanism for your front end to talk to your back end.

The Leptos book puts it as "server functions are not magic; they're syntax sugar for defining a public API". Two practical consequences for this site:

- `portfolio_items` has a `public` boolean. A server function called `list_portfolio` must filter on it *in the query*, not in the component. Filtering in the component leaves the private rows in the JSON payload, visible in the network tab and in the server-rendered HTML.
- Every write function needs its own authorisation check, in its own body. There is no middleware that will do it for you by virtue of the function being "internal", because it is not internal.

Chapter 20

Tailwind CSS

20.1 What you are switching from and to

`style/main.scss` is three lines of Sass. cargo-leptos compiles it with a `sass` binary it downloaded itself, and injects the result at `/pkg/egonik-site.css` — which is already what `src/app.rs` references, so that part needs no change.

The good news is that the Tailwind side needs no Node, no `package.json`, and no `npm install`. cargo-leptos manages the Tailwind CLI exactly as it manages Sass.

Verified: the Tailwind CLI is already on this machine

```
$ ls ~/.cache/cargo-leptos/
  sass-1.86.0/  tailwindcss-v4.2.1/  wasm-bindgen-0.2.125/  wasm-bindgen-0.2.126/
```

cargo-leptos 0.3.6 hardcodes `v4.2.1` as its default Tailwind version — so this is **Tailwind 4**, not 3, and the difference matters for every instruction below. The binary is downloaded from the `tailwindlabs/tailwindcss` GitHub releases; `LEPTOS_TAILWIND_VERSION` overrides the version if you ever need to.

20.2 The change, in four steps

Adopting Tailwind 4

1. Create the input file. One line is genuinely sufficient in Tailwind 4:

```
# style/tailwind.css
@import "tailwindcss";
```

2. Point cargo-leptos at it, in `Cargo.toml`:

```
[package.metadata.leptos]
tailwind-input-file = "style/tailwind.css"
# style-file = "style/main.scss"     <-- delete this line (see below)
# do NOT add tailwind-config-file    <-- see the pitfall below
```

3. Delete `style/main.scss` and the three lines in it. Tailwind's preflight already sets sensible defaults, and the `text-align: center` in that file is not something you want site-wide.

4. Rebuild. `cargo leptos watch`. The href in `app.rs` does not change: the combined output is always `<site-root>/<site-pkg-dir>/<output-name>.css`, which for this project is `target/site/pkg/egonik-site.css`, served at `/pkg/egonik-site.css`.

20.3 The traps, in order of how much time they cost

Do not set `tailwind-config-file`

This is the highest-value warning in the chapter. Under cargo-leptos 0.3.6 the default Tailwind is v4, and **the v4 CLI has no `--config` flag at all**. It accepts the argument silently, exits 0, and ignores it — even if the path does not exist.

Worse: if you set the key and the file is missing, cargo-leptos *writes a v3-style `tailwind.config.js` into your project for you*, complete with `content` globs. That file then sits in your repository looking authoritative while having no effect whatsoever. When purging misbehaves, you will edit it, and nothing will happen.

Setting `tailwind-config-file` without `tailwind-input-file` is also a hard error, with the message `The Cargo.toml `tailwind-input-file` is required when using `tailwind-config-file`]` — the stray bracket is really in the source.

A `tailwindcss` on your PATH silently wins

cargo-leptos calls `which::which("tailwindcss")` first and uses whatever it finds, *with no version check*. If a Tailwind 3 CLI is installed globally, you build with v3 while cargo-leptos still takes its v4 code path and omits `--config` — so you get v3 with no configuration, which purges nearly everything. The same happens if a `package.json` exists at the workspace root, because cargo-leptos prepends `node_modules/.bin` to `PATH`.

`end2end/package.json` exists in this repository. It is in `end2end/`, not the workspace root, so it does not currently trigger this — but if you ever add a root-level `package.json`, remember this paragraph. Diagnose with `cargo leptos watch -vv` and read the logged `tailwindcss --input …` line.

`target/` must stay in `.gitignore`

Tailwind 4 detects classes by scanning the filesystem, and it skips whatever `.gitignore` excludes. `target/` contains the previous CSS output and vendored crate sources; scanning it is slow — one reported case went from 2 ms to 54 s — and it can resurrect classes you deleted, making purging look broken. Your `.gitignore` lists `target/` already. Leave it there.

Three different path bases in one pipeline

This is the usual cause of "my classes are being purged".

- `tailwind-input-file` resolves against the directory of the `Cargo.toml` holding the metadata table.
- `@source` directives inside the CSS resolve against *the CSS file*.
- Tailwind's automatic detection resolves against *the process working directory*, and cargo-leptos `chdir`s to the cargo workspace root.

For this single-crate project all three coincide, so it works with no configuration. If you ever move to a workspace, pin it explicitly and stop relying on the coincidence:

```
@import "tailwindcss" source("../src");
```

20.4 Class detection inside `view!` macros

The question everyone asks: does Tailwind find classes written inside a Rust macro? Yes, without configuration.

Verified against the cached v4.2.1 binary

With a bare `@import "tailwindcss";` and no `@source`, all of these forms produced their rules in the output:

```
view! {
    <div class="flex gap-4 text-sky-500">
        <p class:underline=true class:font-bold=cond>"hi"</p>
        <span class=("bg-red-500", move || active())>"t"</span>
    </div>
}
```

Note in particular that `class:underline=true` yields `underline`. The v3-era workaround — a JavaScript `transform` that stripped `class:` prefixes — is unnecessary under v4, and there is no `transform` option in v4 anyway.

Computing class names at runtime

```
let colour = if item.public { "green" } else { "red" };
view! { <span class=format!("text-{colour}-600")>...</span> }   // BROKEN
```

Tailwind scans *source text*. `text-green-600` never appears in your source, so the rule is never generated and the element is unstyled. Write both class names out in full and choose between them:

```
let colour = if item.public { "text-green-600" } else { "text-red-600" };
view! { <span class=colour>...</span> }                        // works
```

This is the single most common Tailwind bug in any language, and it is easier to hit in Rust because string formatting is so natural.

20.5 Two subtleties worth knowing before they surprise you

20.5.1 If you keep the Sass file, it will override Tailwind

`style-file` and `tailwind-input-file` are *not* mutually exclusive. cargo-leptos runs Sass and Tailwind as two independent processes and simply concatenates the results, Sass first, then Tailwind.

That ordering suggests Tailwind wins ties. It does not. Tailwind 4 wraps everything in real cascade layers (`@layer theme, base, components, utilities;`), and **unlayered CSS beats layered CSS** in the cascade regardless of order. So your hand-written Sass will override Tailwind utilities of equal specificity even though it appears first in the file. Two consequences:

- If you keep both, wrap your own CSS in a layer so it participates in the cascade normally.
- `@apply`, `@theme` and `@plugin` work *only* in the Tailwind input file. Put `@apply` in the `.scss` and dart-sass passes it through untouched — you get invalid CSS in the output rather than an error.

The simplest resolution is the one in the runbook: drop `style-file` entirely and put everything, including `@theme` customisation, in `style/tailwind.css`.

20.5.2 `browserquery = "defaults"` downlevels modern CSS

cargo-leptos runs the combined CSS through `lightningcss` on *every* build — not only release — using `browserquery` as its target list. Tailwind 4 output depends on `@property`, `oklch()` and `color-mix()`. With a broad target list like the current `"defaults"`, lightningcss will rewrite that output for old browsers, and colours or custom properties can come out subtly wrong.

Narrow `browserquery`

```
browserquery = "last 2 versions and not dead"
```

There is no way to disable the lightningcss pass. Note also that if lightningcss cannot parse Tailwind's output, cargo-leptos 0.3.6 falls back to emitting the unminified CSS with only a *trace*-level log — so a real problem is invisible unless you run with `-vv`. Older versions failed the build outright, which was arguably friendlier.

20.6 Watching and reloading

- **Editing a `.rs` file does re-run Tailwind.** cargo-leptos knows that CSS can come from source files when `tailwind-input-file` is set, so a Rust change triggers a style rebuild. Adding a utility class to a `view!` works in `watch` mode.
- **A style-only change reloads just the CSS**, over the live-reload websocket, without a full page refresh.
- **But `--hot-reload` cannot add new classes.** The view-patching hot reload updates markup in place without a rebuild, and a brand-new Tailwind class needs a rebuild to exist. Expect to restart occasionally.
- **Only the exact input-file path is watched**, not files it `@import`s. If you split your CSS across several files, list the others under `watch-additional-files`. There is also a known open bug where a change to the input file is detected but the new CSS is not applied until a manual restart.

20.7 Design tokens, once Tailwind is in

A brief note, because it is the thing that makes a Tailwind codebase pleasant or awful. In v4, customisation happens in CSS rather than JavaScript:

```
@import "tailwindcss";

@theme {
  --color-ink:   oklch(0.22 0.02 250);
  --color-paper: oklch(0.98 0.005 90);
  --color-accent: oklch(0.55 0.15 40);
  --font-display: "Libertinus Serif", Georgia, serif;
}
```

Listing 20.1. `style/tailwind.css` — tokens defined once, used everywhere.

Those become utilities automatically: `text-ink`, `bg-paper`, `border-accent`, `font-display`. Define the palette once, then never write a raw hex value in a `view!` again — that discipline is what keeps a hand-built site from drifting into fifteen shades of almost-grey.

Part VI

Quality and operations

Chapter 21

Testing this stack

21.1 Where the tests should go, and why not everywhere

There are currently no tests. The Playwright directory contains the starter template's example spec, and its dependencies are not even installed — `end2end/node_modules` does not exist.

A personal website does not need exhaustive testing, and pretending otherwise is how testing becomes a chore that gets abandoned. The useful question is narrower: *which failures would you not notice?* For this project there are exactly three, and they map onto three kinds of test.

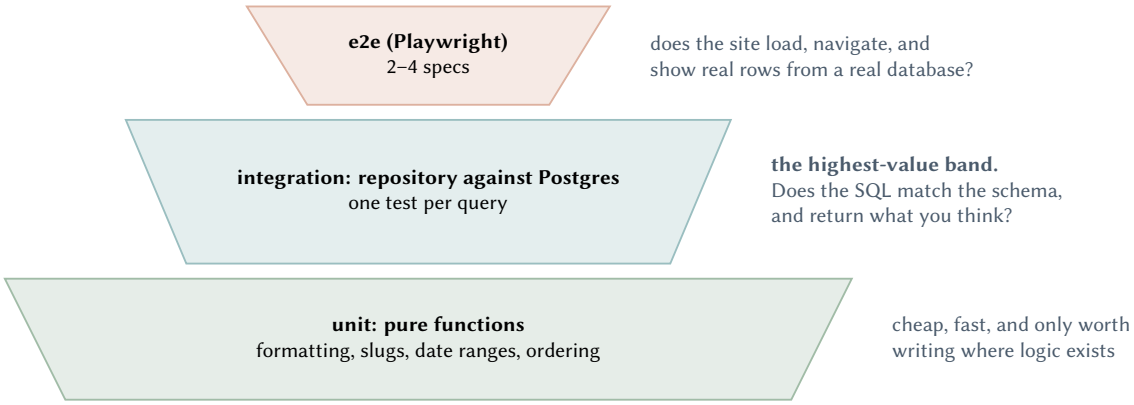


Figure 21.1. What to test, and roughly how much. The widths are deliberate — most of the value is in the middle band, not at the top or the bottom.

Why the integration band is widest

Diesel gives you compile-time-checked queries, so a query that compiles is *well-formed*. It is not necessarily *correct*: nothing at compile time tells you that `list_public()` actually filters on `public`, that ordering is reverse-chronological, or that a join returns one row per parent rather than the Cartesian product. Those are exactly the mistakes that reach a rendered page and look plausible.

Meanwhile, component tests in this stack are expensive relative to their yield — they need a WASM test runner or a headless browser — and there is very little logic in a component worth testing once data loading is factored out. Which is itself an argument for keeping logic out of components, as the Leptos book puts it: "the less of your logic is wrapped into your components themselves, the more idiomatic your code will feel and the easier it will be to test".

21.2 Testing a repository against a real database

Two viable mechanisms, and the choice depends on how much isolation you want to pay for.

21.2.1 Option A: transaction rollback, against one long-lived database

Diesel has this built in. `Connection::test_transaction` runs a closure inside a transaction that is never committed:

```
#[test]
fn list_publications_is_newest_first() {
    let mut conn = PgConnection::establish(&test_database_url()).unwrap();
    conn.test_transaction::<_, diesel::result::Error, _>(|conn| {
        insert_publication(conn, "older", 2019)?;
        insert_publication(conn, "newer", 2024)?;

        let rows = publication_items::table
            .order(publication_items::year.desc())
            .load::<PublicationItem>(conn)?;

        assert_eq!(rows[0].title, "newer");
        Ok(())
    });
    // the transaction is rolled back; the database is untouched
}
```

Listing 21.1. Fast, no containers, no cleanup.

`test_transaction` cannot wrap code that opens its own transaction

`begin_test_transaction` *panics* if a transaction is already open. So if a repository method wraps its work in `conn.transaction(|c| …)` — which it should, once you write multi-table inserts — you cannot test it this way. The way out is a design decision worth making early: **keep transaction control above the repository**. Repository methods take `&mut PgConnection` and do one thing; the caller decides what is atomic. That is better design independently of testing.

21.2.2 Option B: a container per test run

```
[dev-dependencies]
testcontainers-modules = { version = "0.15", features = ["postgres", "blocking"] }
```

```
use testcontainers_modules::{postgres, testcontainers::runners::SyncRunner};

let node = postgres::Postgres::default().start().unwrap();
let url = format!("postgres://postgres:postgres@{}:{}/postgres",
    node.get_host().unwrap(), node.get_host_port_ipv4(5432).unwrap());
// run migrations, then test
```

Slower to start, completely isolated, and — the real argument for it — it exercises your *migrations* as well as your queries. Depend only on `testcontainers-modules`; it re-exports `testcontainers` at a matching version.

Use both, for different jobs

`test_transaction` against your development database for the fast inner loop — it costs milliseconds and you will actually run it. Testcontainers in CI, where the migrations get applied from scratch and a broken `down.sql` is caught by `diesel migration redo -a`. That single CI step would have caught defects **D3** and **D4** the day they were written.

21.3 Testing server functions

A server function is an ordinary `async fn` on the server, so it is directly callable in a test — with one catch. If its body uses `leptos_actix::extract()`, it needs an `HttpRequest` in Leptos context, which a plain `#[test]` does not have.

Keep the server function thin so it barely needs testing

```rust
#[server]
pub async fn list_publications() -> Result<Vec<PublicationDto>, ServerFnError> {
    let pool: Data<DbPool> = leptos_actix::extract().await?;    // <- untestable part
    let repo = PublicationRepository::new(pool.get_ref().clone());
    Ok(repo.list().await?.into_iter().map(Into::into).collect())  // <- testable part
}
```

Everything worth testing is in `repo.list()` and the `From` impl, both of which are plain functions with no framework context. The server function is three lines of wiring. Test the three lines with Playwright, and the logic with ordinary Rust tests.

21.4 End to end

cargo-leptos already knows how to run Playwright — `end2end-cmd` and `end2end-dir` are set in `Cargo.toml`, so `cargo leptos end-to-end` builds the site, starts the server and runs the specs.

`npm ci`, not `npm install`

`end2end/package-lock.json` exists and `end2end/node_modules` does not, so the first step is an install — and because the lockfile is present and committed, the right command is `npm ci`, which installs exactly the locked versions. `npm install` would silently update the lockfile as a side effect of setting up the project. You will also need `npx playwright install` once, to fetch the browsers themselves.

e2e needs a database, and nothing currently gives it one

`cargo leptos end-to-end` builds and serves the real application, which means it needs a real Postgres with migrations applied and seed data present. Without that, every spec fails in a way that looks like a UI bug. Point `end2end-cmd` at a script that ensures the database first:

```
# end2end/run.sh
set -euo pipefail
docker compose up -d db
diesel migration run
psql "$DATABASE_URL" -f ../seeds/dev_seed.sql
npx playwright test
```

Two or three specs is enough, and they should assert on *content from the database*, not on static markup:

```
1. the home page renders the seeded name and every seeded contact link
2. /publications lists the seeded publications, newest first
3. /portfolio/<seeded-slug> renders; /portfolio/no-such-thing returns HTTP 404
```

Spec 3 is the valuable one, because it is the only test in the whole suite that would catch a regression in the status-code plumbing from Chapter 11 — and that is precisely the kind of thing

nobody notices by looking at the site.

Chapter 22

Running it: logging, health, shutdown

22.1 Replace the `println!` with `tracing`

There is currently exactly one diagnostic in the entire application:

```
println!("listening on http://{}", &addr);   // inside the app factory: once per worker
```

`tracing` plus `tracing-actix-web`

```
[dependencies]
tracing = { version = "0.1", optional = true }
tracing-subscriber = { version = "0.3", features = ["env-filter"], optional = true }
tracing-actix-web = { version = "0.7", optional = true }
# ... all three go in the ssr feature list
```

```
tracing_subscriber::fmt()
    .with_env_filter(tracing_subscriber::EnvFilter::from_default_env())
    .init();

App::new().wrap(tracing_actix_web::TracingLogger::default())
```

`RUST_LOG=info,egonik_site=debug` then controls verbosity without a rebuild, and every request gets a span with an id you can correlate errors against.

The one place logging genuinely matters here

`AppError::Internal` deliberately carries no detail (Chapter 11). That is the right call for the browser and it means the *only* record of what actually went wrong is the server-side log line. Without `tracing::error!` in the error-mapping layer, you have built a system that converts real database errors into the word "internal" and discards them. Log before you collapse.

22.2 A health endpoint

Three lines, and the first thing any deployment target will ask for:

```
#[actix_web::get("/healthz")]
async fn healthz(pool: web::Data<DbPool>) -> impl Responder {
    match web::block(move || pool.get()?.execute("SELECT 1")).await {
        Ok(Ok(_)) => HttpResponse::Ok().body("ok"),
        _         => HttpResponse::ServiceUnavailable().body("db unavailable"),
    }
}
```

Check the dependency, not just the process

A health check that returns 200 as long as the process is alive tells you nothing you did not already know from the process being alive. Touching the pool is what makes it useful — it distinguishes "the site is up" from "the site is up and can serve content". This is also the one Actix route in the whole application that is genuinely *not* better as a server function, because it must be callable by a monitor that knows nothing about Leptos.

22.3 Shutdown and worker tuning

```
HttpServer::new(app_factory)
    .workers(2)                          // 2 is plenty; the default is one per core
    .shutdown_timeout(10)                // finish in-flight requests on SIGTERM
    .bind(&addr)?
    .run()
    .await
```

`shutdown_timeout` is what makes a redeploy invisible to a visitor mid-request. On a personal site, `.workers(2)` is also a real saving: each worker gets its own blocking thread pool, and sixteen workers on a sixteen-core machine is a lot of idle infrastructure for a site serving a CV.

22.4 Run migrations at startup, or don't — but decide

`diesel_migrations` can embed the migration files into the binary and apply them on boot:

```
pub const MIGRATIONS: EmbeddedMigrations = embed_migrations!("migrations");
conn.run_pending_migrations(MIGRATIONS)?;
```

Embed them, for a single-instance personal site

The alternative — applying migrations as a separate deployment step — is correct for anything with multiple replicas, where two instances starting at once would race. With one instance, embedding removes an entire class of deployment mistake: the binary and the schema it expects can never disagree, because they ship together. Note that this also means the container needs no `diesel` CLI and no `migrations/` directory at runtime.

Chapter 23

Build, ship, deploy

23.1 What has to be in the image

A Leptos SSR application is two artefacts that must travel together: the server binary and the `site` directory holding the wasm bundle, the CSS and the assets. Ship one without the other and you get a server that returns HTML referencing a 404ing script.

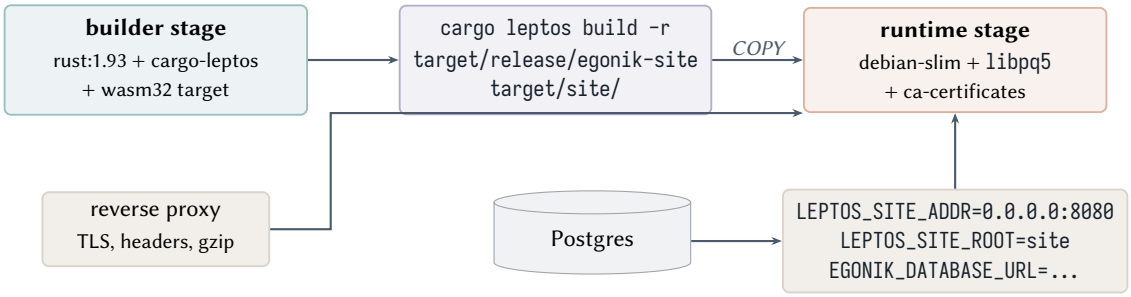


Figure 23.1. The deployment shape. Everything the container needs is produced by one `cargo leptos build --release`.

```
FROM rust:1.93-bookworm AS builder
RUN rustup target add wasm32-unknown-unknown \
 && cargo install cargo-leptos --locked
WORKDIR /app
COPY . .
RUN cargo leptos build --release -vv

FROM debian:bookworm-slim
RUN apt-get update \
 && apt-get install -y --no-install-recommends libpq5 ca-certificates \
 && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY --from=builder /app/target/release/egonik-site  /app/egonik-site
COPY --from=builder /app/target/site                 /app/site
ENV LEPTOS_SITE_ADDR="0.0.0.0:8080"
ENV LEPTOS_SITE_ROOT="site"
ENV LEPTOS_ENV="PROD"
ENV RUST_LOG="info"
EXPOSE 8080
CMD ["/app/egonik-site"]
```

Listing 23.1. A two-stage `Dockerfile`. `libpq5` in the runtime stage is the detail people miss — Diesel's `postgres` feature links native `libpq`.

> **`LEPTOS_ENV="PROD"` is not cosmetic**
>
> In `DEV`, Leptos injects the live-reload websocket client into every page. In `PROD` it does not. Forget the variable and your deployed site will try, forever, to open a websocket to `localhost:3001`.

Do not pass a path to `get_configuration`

`get_configuration(Some("./Cargo.toml"))` reads `Cargo.toml` at *runtime*. The image above ships neither `Cargo.toml` nor the source tree, so it would fail on boot. The `None` in the current `main.rs` is correct — keep it, and drive `LeptosOptions` through `LEPTOS_*` variables as the Dockerfile does.

23.2 The Compose file needs an app service

`docker-compose.yml` currently defines only Postgres, and it defines it with a database name that disagrees with `.env` (defect **D14**). Fixing both together makes a fresh clone work on the first try, which is the actual point of having a Compose file.

```
services:
  db:
    image: postgres:17
    restart: always
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: egonik_site          # <-- matches .env; no hyphen, so no quoting in psql
    ports: ["5439:5432"]
    volumes: ["postgres_data:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 10

  app:
    build: .
    depends_on:
      db: { condition: service_healthy }
    environment:
      EGONIK_DATABASE_URL: postgres://postgres:postgres@db:5432/egonik_site
      LEPTOS_SITE_ADDR: "0.0.0.0:8080"
      LEPTOS_SITE_ROOT: "site"
      LEPTOS_ENV: "PROD"
    ports: ["8080:8080"]

volumes:
  postgres_data:
```

Listing 23.2. The corrected `docker-compose.yml`.

Rename the database and drop the hyphen

`my-site` is a hyphenated identifier, which means every bare `psql` or DDL statement that names it needs double quotes — `ALTER DATABASE "my-site" …`. It works, and it is a permanent papercut. `egonik_site` costs nothing to switch to today, while the database holds no data.

23.3 Continuous integration

There is no `.github/` directory. The single highest-value workflow is short, and it is almost entirely made of things this document has already argued for.

```
name: ci
on: [push, pull_request]
jobs:
```

```yaml
check:
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:17
      env: { POSTGRES_PASSWORD: postgres }
      options: >-
        --health-cmd pg_isready --health-interval 5s --health-retries 10
      ports: ["5432:5432"]
  env:
    DATABASE_URL: postgres://postgres:postgres@localhost:5432/postgres
  steps:
    - uses: actions/checkout@v4
    - uses: dtolnay/rust-toolchain@stable
      with: { targets: wasm32-unknown-unknown, components: rustfmt, clippy }

    - run: cargo fmt --all --check
    - run: cargo clippy --no-default-features --features ssr -- -D warnings

    # 1. the server program
    - run: cargo check --no-default-features --features ssr
    # 2. the browser program  <-- the check `cargo leptos watch` can hide from you
    - run: cargo check --lib --no-default-features --features hydrate
                       --target wasm32-unknown-unknown

    # 3. migrations apply AND revert, every one of them
    - run: cargo install diesel_cli --no-default-features --features postgres --locked
    - run: diesel migration run
    - run: diesel migration redo -a          # would have caught defects D3 and D4

    # 4. schema.rs is not stale
    - run: diesel print-schema > /tmp/schema.rs && diff -u src/schema.rs /tmp/schema.rs

    - run: cargo test --no-default-features --features ssr
```

Listing 23.3. `.github/workflows/ci.yml` — the five checks that matter.

Why steps 2, 3 and 4 in particular

Step 2 exists because `cargo leptos watch` can leave the wasm target un-rebuilt, so "it works locally" is not evidence that both programs compile. Chapter 4 makes this a habit; CI makes it a guarantee.

Step 3 is the automated form of the discipline in Chapter 16. Every migration defect in this document is a `down.sql` defect, and every one of them is caught by `redo -a` in under a second.

Step 4 catches the specific mistake of committing a migration without regenerating `src/schema.rs`, which produces type errors describing a schema that exists nowhere.

Pin the toolchain

There is no `rust-toolchain.toml`. Add one, so CI and your machine cannot silently diverge:

```toml
[toolchain]
channel = "1.93.1"
targets = ["wasm32-unknown-unknown"]
components = ["rustfmt", "clippy"]
```

23.4 Commit `Cargo.lock`

`.gitignore` line 8 excludes `Cargo.lock`, and the file is consequently untracked. That rule is correct for a *library*, where consumers resolve their own dependency versions. This crate produces a binary, and for a binary the lockfile is what makes a build reproducible: without it, CI and your machine can resolve different versions of `leptos` and you get an error that exists on exactly one of them. Remove the line and commit the file. Defect **D13**.

23.5 And `.env` is not ignored

> **Verified on this machine**
>
> ```
> $ git check-ignore -v .env
>   (no output -- the file is NOT ignored)
> $ git status --short
>   ?? .env
> ```

`.env` contains live Postgres credentials, is untracked, and is *not* excluded by `.gitignore` — so a single `git add -A` commits it, and once it is in history it stays there. This is defect **D19**, and the fix is three lines:

```
.env
migrations/.diesel_lock
/site                    # the repo-dump artefact in the working tree
dumpSkip.txt
```

Listing 23.4. Additions to `.gitignore`.

Then add a committed `.env.example` with the keys and no values, so a fresh clone knows what to set. While you are in that file: the `.gitignore`, `README.md` and `LICENSE` are still the Leptos starter template's. The README documents how to use the template rather than how to run this site, which is a small thing that makes the repository read as unfinished.

Chapter 24

Pattern review

24.1 The short answer

> *The structure is right and the wiring is right. What is wrong is concentrated in one place — the `Repository` trait — and it has now produced hard evidence against itself. Everything else on the list below is a small, local fix.*

Since the first draft of this document you fixed both blockers it identified in the wiring: the pool is now built once above `HttpServer::new`, and the `ssr` gates have moved off the subject modules onto the files that actually need them. Both builds are green, and there is a working end-to-end path from Postgres to a button in the browser. That is the hard part of this stack and it is done.

24.2 What is working, and worth not second-guessing

Table 24.1. Patterns in the tree that are correct. Listed because knowing which decisions are settled is as useful as knowing which are not.

Pattern	Why it is right
Per-file `cfg` gates in `<subject>/mod.rs`	`dto` and `components` ungated, `model` and `repository` gated. Exactly the shape Ch. 8 argues for, and it is what makes a server function expressible.
Pool built once, `AppState` cloned per worker	Twelve pools became one. `app_state.clone()` is an `Arc` bump.
`AppState` holding repositories	Server functions ask for a capability rather than a database handle. §12.2.
Three structs per subject: `Item` / `Row` / `Dto`	`PublicationItem` is `Queryable`, `PublicationItemRow` is `Insertable` with no `id`, `PublicationItemDto` is the wire shape. This is *better* than the two-struct rule in Ch. 9 — a separate insert struct is how you stop passing a meaningless `id: 0` into a create.
`From<PublicationItemDto> for PublicationItemRow`	Conversion by trait, next to the model, in the direction the standard library expects.
`Action` + `.into_any()` in `ServerButton`	Correct 0.8 idioms: an `Action` for the imperative call, `pending()`/`value()` for state, `.into_any()` to reconcile three branches with different view types. Ch. 19.
`entrypoint/` as the process edge	The instinct that "the server lives here" is right. Ch. 13 fills it in.

24.3 The one structural problem: `Repository<T, U>`

Chapter 10 argued that this trait would not survive. It has now failed in the exact way predicted, and the evidence is in the tree rather than in an argument.

Verified: the trait is forcing three implementors to lie

Adding `create_article` to `Repository<T, U>` obliged every implementor to provide it. Three of the four cannot:

```
// src/portfolio/repository.rs
fn create_article(&mut self, article: PortfolioItemDto) -> anyhow::Result<()> { todo!() }

// src/job_experience/repository.rs
fn create_article(&mut self, article: JobExperienceItemDto) -> anyhow::Result<()> { todo!() }

// src/personal_information/repository.rs
fn create_article(&mut self, article: PersonalInformationDto) -> anyhow::Result<()> { todo!() }
```

And the compiler is already reporting it, three times over:

```
$ cargo check --no-default-features --features ssr
warning: unused variable: `article`
warning: unused variable: `article`
warning: unused variable: `article`
```

Two separate things went wrong, and they are worth naming separately because only one of them is about the trait.

The method name is subject-specific. "Article" is publication vocabulary. A portfolio item is not an article; neither is a job. The name could not generalise because the *concept* does not: what a shared trait can express is `create`, and even that is only meaningful when every implementor genuinely creates one thing from one DTO.

The trait has no shared behaviour left to describe. `get_all` and `get_by_id` happened to fit all four subjects when they were the only two methods. The third method fits one. The fourth — `list_public()` for portfolio, `newest_first()` for publications, `by_slug()` for both — will fit fewer. There is no interface here, only a coincidence that has now expired. Three `todo!()`s is what a coincidence looks like when you build on it.

Delete the trait; keep the structs

This is a smaller change than it sounds, because the bodies already exist and do not move.

```
impl PublicationsRepository {
    pub fn new(pool: DbPool) -> Self { Self { pool } }

    // &self, not &mut self -- see below
    pub async fn list_newest_first(&self) -> Result<Vec<PublicationItem>, RepoError> { /* ... */ }
    pub async fn by_id(&self, id: i32) -> Result<PublicationItem, RepoError> { /* ... */ }
    pub async fn upsert_many(&self, items: &[PublicationItemDto]) -> Result<usize, RepoError> { /* ... */ }
}
```

Listing 24.1. `src/publications/repository.rs` — inherent methods, no trait.

Delete `src/core/mod.rs`, delete the three `todo!()` stubs, and delete `use crate::core::Repository;` from the four repositories and from `bin/seed_publications.rs`. Nothing else changes: `AppState` still holds the same four types, and the call sites keep working because they were calling methods, not the trait.

Reintroduce a trait only when you have a *second implementation* of the same behaviour — an in-memory fake for tests is the usual first one. A trait with one implementor is a trait you are maintaining for free.

But isn't a repository trait "good practice"?

It is, in languages where the alternative is worse. The pattern comes from ecosystems where you need an interface to get dependency injection, mocking, or a runtime-swappable backend. Rust gives you the first two through ordinary generics and `cfg(test)`, and you do not need the third: there is one Postgres.

Rust's own guidance points the same way — generics exist to make a function usable by more callers, not to make a type substitutable in anticipation. And coherence means a generic `Repository<T>` cannot be specialised per entity anyway, so you write the same number of method bodies with worse type inference and worse error messages. The trait was costing you and paying nothing, which is why it grew a method that three implementors had to refuse.

24.4 Five local fixes, in the order they will bite

24.4.1 1. `&mut self`, and the clone it is forcing

The cost is now visible in your own code:

```
let state: Data<AppState> = extract().await?;
let mut repo = state.publications_repository.clone();   // <-- cloned only to get `mut`
let results = repo.get_all().unwrap();
```

Listing 24.2. `src/publications/server.rs`

That `.clone()` is not there for a reason of its own — it is there because `get_all` takes `&mut self` and `Data<AppState>` hands out shared references. The trait forces a clone at every call site to obtain a mutability nobody needs: `Pool::get` takes `&self`, and no method in any repository mutates anything. Change the receivers to `&self` and the line becomes:

```
let results = state.publications_repository.list_newest_first().await?;
```

Defect **D8**, and it goes away as part of deleting the trait.

24.4.2 2. `.unwrap()` in a server function

`repo.get_all().unwrap()` panics the Actix worker thread when the database is unreachable — which is a 500 with no body, no log line, and a poisoned worker, rather than an error the `ErrorBoundary` could render. In a server function, `?` is always available: `Result<T, ServerFnError>` has a blanket `From` for any `std::error::Error`. Use it.

24.4.3 3. Diesel is still being called on the async runtime

`server.rs` calls `repo.get_all()` directly inside an `async fn`. That is the blocking bug from Chapter 10, now live: the query holds an Actix worker thread for its whole duration. With twelve workers it is invisible; it is invisible right up until it is not. Wrap the query in `web::block` inside the repository, with `pool.get()` inside the closure.

24.4.4 4. The DTO has no serde derives — and this is the next wall

```
#[derive(Debug, Clone)]          // <-- no Serialize, no Deserialize
```

```
pub struct PublicationItemDto { /* ... */ }
```

Listing 24.3. src/publications/dto.rs

This compiles today only because `get_all_publications` returns `Vec<String>`. The moment it returns `Vec<PublicationItemDto>` — which is the next thing you will want, since `Vec<String>` throws away the year, journal and link the page needs — you will hit two errors in sequence:

1. `the trait bound `PublicationItemDto: serde::Serialize` is not satisfied`, four times per DTO, with notes mentioning `BrowserMockRes` and `JsonSerdeCodec`. Add `Serialize, Deserialize` to the derive.
2. `error[E0432]: unresolved import serde` in the *wasm* build — because `serde` is still `optional = true` with `dep:serde` inside the `ssr` feature. Ch. 6 has the two-line `Cargo.toml` fix.

Defect **D6**. Fixing it before you change the return type turns a confusing twenty-minute detour into a non-event.

24.4.5 5. `impl Into<...>` instead of `impl From<...>`

`bin/seed_publications.rs` opens with `#![allow(clippy::from_over_into)]`, which is the tell. Clippy is right: implement `From<Article> for PublicationItemDto` and you get `Into` generated for free, in both directions of inference, and the `allow` disappears. The reason the lint exists is that a hand-written `Into` does not give you the reciprocal `From`, so generic code bounded on `From` silently will not accept your type.

24.5 Two smaller things while you are in the files

- **`abs` is still `abs`.** It is now in three places — the column, the model, and the DTO — so the rename costs three edits today and more next week. Defect **D15**.
- **`components.rs` has a redundant import.** `use leptos::prelude::*;` followed by `use leptos::{component, view, IntoView};` — the prelude already provides all three. Harmless, and worth deleting so the next file you write copies the right pattern.

24.6 The revised next step

Chapter 26's M1 was "get one slice from Postgres to the screen". You have done that in its rough form, so the remaining work in M1 is to make the same path carry real data:

> **What to do next, concretely**
>
> 1. Make `serde` unconditional in `Cargo.toml`, and add `Serialize, Deserialize` to all four DTOs. **D6**
> 2. Delete `src/core/mod.rs` and the trait; convert the four repositories to inherent methods taking `&self`; delete the three `todo!()` stubs. **D8**
> 3. Wrap the queries in `web::block`, with `pool.get()` inside the closure.
> 4. Change `get_all_publications` to return `Result<Vec<PublicationItemDto>, ServerFnError>`, with `?` instead of `.unwrap()`.
> 5. Replace `ServerButton` with a `Resource` + `Transition` so the list is server-rendered rather than fetched on click. Keep the button somewhere until then — it is a genuinely useful hydration test (§7).
> 6. Add `external_id` + `UNIQUE`, and make the seeder an upsert, before you run it again. §14.4
> 7. Extract `openalex.rs` and `service.rs`; reduce the binary to composition. Ch. 14

Steps 1–4 are perhaps an hour and remove four of the five defects in this chapter. Step 6 is the one with a deadline attached: it needs to happen before the next `cargo run --bin seed_publications`, or you will be deduplicating rows by hand.

Part VII

The plan

Chapter 25

The defect register

25.1 How to read it

Two tables. The first lists **defects** — things that are wrong in code or schema that already exists. The second lists **gaps** — things that are absent. The distinction matters for sequencing: a defect gets more expensive the longer it stays, because code accumulates around it; a gap costs nothing until you need it.

Severities mean something specific here:

BLOCKER
: Either it breaks a command you will run this week, or it will corrupt data, or it prevents the next piece of work from being written at all.

HIGH
: It will cost meaningfully more to fix later than now, usually because the fix touches a foreign key, a wire format, or something with content already in it.

MEDIUM
: Real, worth fixing, no deadline.

LOW
: Housekeeping. Fix it while you are in the file.

Everything in both tables was confirmed by reading the code, compiling it, or running it against the live database.

25.2 Defects

Table 25.1. Defects: things that are wrong today.

#	Sev.	What is wrong	Fix
D1	BLOCKER	All three foreign-key columns are declared `SERIAL`, so each has its own sequence and a `DEFAULT nextval(...)`. An insert that omits the FK silently attaches the child to an arbitrary parent, then later fails intermittently. Verified against the live database.	Declare them `INTEGER NOT NULL REFERENCES` …. Drop the three stray sequences. §15.4
D38	BLOCKER	`get_connection_pool()` is called *inside* the `HttpServer::new` factory, which runs once per worker. Verified: 12 workers on this machine × r2d2's default `max_size` of 10 = **120 connections** against a Postgres whose `max_connections` is 100. Will not fail in development.	Build the pool and `AppState` above `HttpServer::new`; clone the state into each worker. §12.2

Table 25.1 continued

#	Sev.	What is wrong	Fix
D3	BLOCKER	Two `down.sql` files drop the parent table before the child, so the foreign key blocks it. `diesel migration revert` fails and the whole chain becomes unrevertible. Verified.	Reverse the order: children first. §16
D4	BLOCKER	`create_publications_item/down.sql` drops `publication_itens` — a typo. Verified: `ERROR: table "publication_itens" does not exist`.	`DROP TABLE IF EXISTS publication_items;`
D5	BLOCKER	`lib.rs` gates all four subject modules on `ssr`, so no `#[server]` function or component inside them can exist in the browser build. This now affects real code: both `publications/components.rs` and the new `publications/server.rs` are invisible to the wasm build.	Move the gate down to `model.rs` and `repository.rs`. Ch. 8
D6	BLOCKER	`serde` is `optional = true` and enabled only by `ssr`, while every model derives `Serialize`. Verified: the browser build fails with `error[E0432]: unresolved import serde`.	Make `serde` unconditional. Ch. 6
D14	HIGH	`.env` points at database `my-site`; `docker-compose.yml` creates `mydatabase`. A fresh clone connects to an empty database.	Rename both to `egonik_site` (no hyphen). §23
D7	HIGH	`JobExperienceItem`, `JobInstitution`, `PersonalInformation` and `ContactInformation` have all-private fields; `PortfolioItem` and `PublicationItem` do not. Nothing outside the module can read them.	Add `pub` to every field, consistently. §9
D8	HIGH	`Repository<T>` takes `&mut self` although `Pool::get` takes `&self`, and its methods are synchronous.	`&self`, plus `web::block` at the call site. Ch. 10
D16	HIGH	`accomplishments` and `responsabilities` are `VARCHAR(255)` single columns holding what are logically lists.	`TEXT`, or a `job_highlights` child table with ordering.
D23	HIGH	No foreign key declares `ON DELETE`, so the default `NO ACTION` applies and deleting any parent row errors.	`ON DELETE CASCADE` for owned children, `RESTRICT` for institutions.
D27	HIGH	`contact_informations` is one-to-one in intent and one-to-many in the schema; nothing prevents four contact rows per person.	`UNIQUE (person_id)`, or replace with a `contact_links` table. §15
D28	HIGH	`job_institutions` points the wrong way: an institution belongs to one experience, so an employer must be duplicated per role.	Promote `institutions` to its own table; `job_experiences` holds the FK.

Table 25.1 continued

#	Sev.	What is wrong	Fix
D24	MEDIUM	No index on any FK column; Postgres indexes only the referenced side. Honest framing: at this data volume Postgres will sequentially scan regardless and should — the argument is schema hygiene plus the row-locking cost of `ON DELETE CASCADE`, not page latency.	One `CREATE INDEX` per FK column. Free, so do it.
D19	MEDIUM	`.env` is untracked but *not* in `.gitignore` — verified with `git check-ignore`. One `git add -A` commits it permanently. The exposure today is small (the credential is `postgres:postgres` on a container bound to `localhost`); the habit is what matters, and there is no `.env.example` for a fresh clone.	Add `.env` to `.gitignore`; commit a `.env.example`.
D2	MEDIUM	`src/entrypoint/` held a `routes/` subtree the compiler never saw, because `mod.rs` was empty. Now restructured into `http.rs` and `application_state.rs` — both declared, both still empty.	Fill both, or remove them. §12.2
D9	MEDIUM	`dotenvy` and `anyhow` are unconditional dependencies and are compiled into the wasm bundle. Verified with `cargo tree`. `dotenvy` reads `.env` files — in a browser.	Mark both `optional` and move them into the `ssr` feature.
D11	MEDIUM	The portfolio, job-experience and personal-information repositories all report `"Can't get publication items from db"`. Copy-paste.	Typed errors per subject; the mistake then cannot be expressed. Ch. 11
D12	MEDIUM	`get_connection_pool` calls `dotenv()`, reads `env::var` and `.expect()`s, all inside a constructor.	Load config once in `main`; pass values in; return `Result`. §12
D13	MEDIUM	`Cargo.lock` is gitignored, though this crate produces a binary. Builds are not reproducible.	Remove the `.gitignore` line; commit the lockfile.
D15	MEDIUM	`responsabilities` is misspelled; `abs` is opaque and shadows the SQL `ABS()` function name. Both are about to become permanent API surface.	`responsibilities`, `abstract_text`. Free today.
D25	MEDIUM	No `CHECK` constraints. A job can end before it began; a publication year can be 20260.	`CHECK (date_to IS NULL OR date_to ≥ date_from)` and a year range.
D26	MEDIUM	No table has `created_at` / `updated_at`, and the `diesel_manage_updated_at()` helper that the initial migration installed is unused.	Add both columns now; they cannot be backfilled later.
D29	MEDIUM	`tags` stores the tag text per item, so duplicates and near-duplicates are unavoidable and unlimited.	`tags` + `portfolio_item_tags` join table with a composite PK.
D33	MEDIUM	The new `Publications` component is `fn`, not `pub fn`, so no route can reach it.	Add `pub`.

Table 25.1 continued

#	Sev.	What is wrong	Fix
D10	LOW	`chrono` is compiled into the wasm bundle with default features, including the parts that want a system clock and timezone database.	Keep it unconditional (dates do cross the wire) but add `default-features = false, features = ["serde"]`.
D17	LOW	`println!("listening on ...")` is inside the `HttpServer` factory, so it prints once per worker thread.	Move it above `HttpServer::new`, and use `tracing::info!`.
D18	LOW	`Files::new("/assets", &site_root)` maps `/assets` onto the entire site root, not the assets directory. Inherited from the template.	`Files::new("/assets", format!("{site_root}/assets"))`.
D20	LOW	The `csr` *binary* target does not compile: `leptos::mount_to_body` moved in 0.8. Verified — `error[E0425]`. The lib target compiles.	Delete the `csr` feature and the two dead `main` functions.
D30	LOW	`README.md`, `LICENSE` and `.gitignore` are still the `leptos-rs/start-actix` template's; `Cargo.toml` has no description, authors, or repository.	Rewrite the README as how to run *this* site.
D32	LOW	Every migration uses `CREATE TABLE IF NOT EXISTS`, which turns a schema conflict into silent divergence instead of an error.	Drop `IF NOT EXISTS` from `CREATE`; keep `IF EXISTS` on `DROP`.
D21	LOW	`migrations/.diesel_lock`, the `site` dump artefact and `dumpSkip.txt` are untracked and unignored.	Add all three to `.gitignore`.
D22	LOW	Migration directory names carry a `-0000` suffix that Diesel does not generate, and mix singular and plural.	Let `diesel migration generate` name them.
D31	LOW	Repositories import Diesel's internal method traits (`query_dsl::methods::{FilterDsl, SelectDsl}`) rather than `diesel::prelude::*`.	`use diesel::prelude::*;`
D34	LOW	`http = { version = "1.3.1", optional = true }` is enabled by no feature and used by nothing. Verified: the only `http::` path in `src/` is `actix_web::http::`. It compiles to nothing and implies a dependency you have not got.	Delete the line.
D35	LOW	`serde_json` is enabled by `ssr` and used nowhere in `src/`. Verified by `grep`.	Delete it until something needs it.
D36	LOW	`wasm-bindgen` and `console_error_panic_hook` are unconditional, so they are compiled into the *native server* build as well — the mirror image of D9.	Move both into the `hydrate` feature.

Table 25.1 continued

#	Sev.	What is wrong	Fix
D37	LOW	`app.rs` declares both a `WildcardSegment` route *and* a `Routes fallback`. The wildcard matches first, so the fallback string `"Not found."` is unreachable dead code that looks like the 404 handler.	Delete the fallback, or point it at `NotFound`.

25.3 Gaps

Table 25.2. Gaps: things that do not exist yet. The order is roughly the order to build them in.

#	What is missing	First concrete step
G1	**Mostly done.** `AppState` now holds the four repositories and is registered with `app_data`. What remains is moving its construction out of the worker factory (D38) and returning `Result` instead of panicking.	§12.2
G2	**Started.** `publications/server.rs` has a `#[server] async fn get_all_publications() -> Result<(), ServerFnError>`. To become useful it needs: a `Vec<PublicationDto>` return type, `pub`, an ungated module (D5), and a body that extracts `AppState` and calls the repository through `web::block`.	Ch. 10, §12
G3	No page renders data. `app.rs` is the starter template's click counter; one real route.	`src/ui/` with a `Shell` and one page per route. Ch. 18
G4	All seven tables are empty, and there is no seed mechanism. An empty list and a broken query look identical.	A seed migration with your real content. Ch. 17
G14	No `slug` column anywhere, so detail URLs would have to be `/portfolio/7`.	Add `slug TEXT NOT NULL UNIQUE` in the same migration as D1.
G6	No application error type and no `ErrorBoundary` anywhere, so a failed query renders as nothing. (The 404 path *does* work — verified. What is missing is the 500 case.)	`src/error.rs` with `AppError: FromServerFnError`. Ch. 11
G7	No `Resource`, `Suspense` or `Transition`, and no decision about SSR mode per route.	The read pattern in Ch. 19; pick `SsrMode` per route.
G8	Tailwind is not set up; `style/main.scss` is three lines of Sass.	Four steps, no Node required. Ch. 20
G5	No write operations anywhere: no `Insertable`, no `New*` struct, no `insert_into`. No authoring path, and no decision about whether there should be one.	Decide between SQL migrations, Markdown files, or an authenticated admin. §17
G9	No logging, no health endpoint, no graceful shutdown. One `println!`.	`tracing` + `tracing-actix-web`, `/healthz`, `shutdown_timeout`. Ch. 22

Table 25.2 continued

#	What is missing	First concrete step
G10	No tests at all. The Playwright spec is the untouched starter example and its dependencies are not installed.	One repository integration test; three Playwright specs. Ch. 21
G11	No CI, no `rust-toolchain.toml`, no fmt/-clippy gate.	The five-check workflow in §23.
G12	No `Dockerfile`, no app service in Compose, no deployment story.	Two-stage build; remember `libpq5`. Ch. 23
G13	No SEO metadata beyond the template `<Title text="Welcome to Leptos"/>`: no description, no Open Graph, no `robots.txt`, no sitemap.	Per-page `<Title>` and `<Meta>`; `robots.txt` in `assets/`.

One defect that is not in the table

`src/app.rs` still sets `<Title text="Welcome to Leptos"/>`. It is trivial and it is also the single most visible unfinished thing about the site — it is what appears in a browser tab and in search results. Worth thirty seconds before anything else in this document.

Chapter 26

The roadmap

26.1 The sequencing argument

The temptation is to start with the visible part, because the visible part is what feels like progress. Resist it in one specific way: **repair the foundation first**, because four of the five blockers are things that make the next piece of work either impossible or wrong.

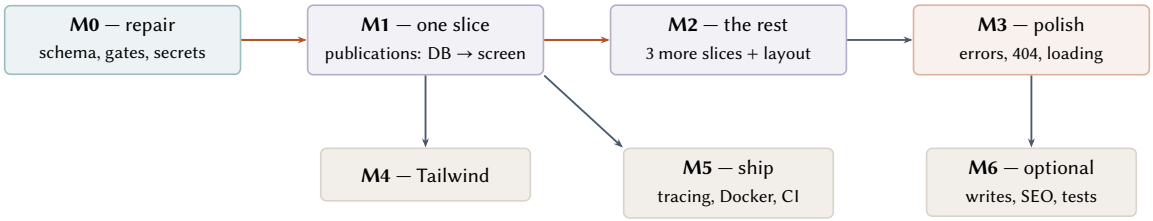


Figure 26.1. Milestone dependencies. M0 unblocks everything; M1 is the one that proves the system works end to end. M4 through M6 are genuinely independent of each other.

26.2 M0 — Repair the foundation

Everything here is a defect, not a feature, and all of it is cheap *today* because every table is empty and there is one commit.

M0, in order

1. **Secrets and hygiene first** (five minutes, and irreversible if you skip it): `.env`, `migrations/.diesel_lock`, `site` and `dumpSkip.txt` into `.gitignore`; remove `Cargo.lock` from it and commit the lockfile. D19 D21 D13
2. **Rewrite the four migrations to reach the schema in Figure 15.2** — one coherent target, not twelve negotiations. That single change covers D1 D3 D4 D15 D16 D23 D24 D25 D26 D27 D28 D29 D32 and G14.
3. **Rename the database** to `egonik_site` in both `.env` and `docker-compose.yml`, then `diesel database reset` and `diesel migration redo -a`. D14
4. **Regenerate** `src/schema.rs` and commit it with the migrations.
5. **Fix the dependency table** (Table 6.1): `serde` unconditional; `dotenvy` and `anyhow` `ssr`-only; `chrono` with `default-features = false`; `wasm-bindgen` and `console_error_panic_hook` into `hydrate`; delete `http` and `serde_json`; delete the `csr` feature and the two dead `main` functions. D6 D9 D10 D20 D34 D35 D36
6. **Move the module gates down** from `lib.rs` into each `<subject>/mod.rs`. D5
7. **Resolve `src/entrypoint/`**: put `AppState` in `application_state.rs` (§12.2) and the Actix wiring in `http.rs`, or remove both. Do not leave them empty and declared. D2
8. **Delete the dead `Routes` fallback** in `app.rs`. D37
9. **Make model fields `pub`**, and `Publications` `pub`. D7 D33

10. **Change `Repository` to `&self`** — or delete the trait now, since Ch. 10 argues it should not survive. D8
11. **Fix the title.** `<Title text="Eduardo Gonik"/>`.

Done when: `diesel migration redo -a` succeeds; both `cargo check` invocations from Ch. 4 pass; `git status` shows no `.env`.

26.3 M1 — One slice, all the way through

Take one *subject — publications, because it is the only table with no children — and carry it from Postgres to the rendered page. Do not start the other three until this one works.*

Why this matters

Everything you are about to learn the hard way lives in this one slice: the gate, the DTO, the `From` impl, the server function, the pool extraction, `web::block`, the resource, the suspense boundary. Discovering all of that once, on the simplest table, is much cheaper than discovering it four times in parallel — and once it works, the other three slices are transcription rather than design.

M1, in order

1. Seed real content: three publications, via a migration. G4
2. Move the pool and `AppState` construction *above* `HttpServer::new`. D38 G1
3. `publications/dto.rs`: `PublicationDto`, ungated.
4. `publications/model.rs`: drop the serde derives; add `From<PublicationItem>`.
5. `publications/repository.rs`: `list()` taking `&self`, wrapped in `web::block`, ordered newest-first.
6. `publications/api.rs`: `#[server] list_publications()`. G2
7. `publications/components.rs`: grow the placeholder into the `Resource`/`Transition` pattern. G7
8. Route it at `/publications`. G3

Done when: `curl localhost:3000/publications` returns HTML *containing the seeded titles* — not an empty shell that fills in later. That is the test that proves server rendering, not just hydration, is working.

26.4 M2 — The remaining slices, and the shell

Now repetition, plus the one genuinely new problem: the other three subjects all have child tables.

- `src/ui/layout.rs`: `Shell` with header, nav, `<Outlet/>`, footer. Convert `app.rs` to the nested-route form. Ch. 18
- `portfolio` with its tags, `job_experience` with its institution, `personal_information` with its contact links — each needs an *aggregate* DTO (`PortfolioItemDto` containing `Vec<TagDto>`), assembled in the repository with Diesel's `belonging_to` plus `grouped_by`, not with a query per row.
- `/portfolio/:slug` detail page, which is where the 404 path gets exercised for real.

Done when: every route in Table 18.1 renders seeded content, and `/portfolio/nonexistent` returns HTTP 404 — checked with `curl -i`, because a browser will show you the page either way.

26.5 M3 — Errors, states, and edges

- `src/error.rs` with `AppError` implementing `FromServerFnError`; migrate the server functions off bare `ServerFnError`. G6
- `ErrorBoundary` on every page; one shared skeleton component with fixed heights so loading does not shift the layout.
- A real 404 page instead of the string `"Not found."`.
- Decide `SsrMode` per route. For a content site where crawlers matter, `SsrMode::Async` on the content routes ships complete HTML rather than a streamed shell.

26.6 M4 — Tailwind and the actual design

Chapter 20 is the mechanism; this milestone is the part that is not engineering. Do it *after* M2, deliberately: styling a page whose content is real is a different and much better experience than styling a page full of placeholder text, because the line lengths, the heading lengths and the awkward cases are all in front of you.

26.7 M5 — Make it shippable

`tracing` and `tracing-actix-web`; `/healthz`; `shutdown_timeout`; embedded migrations; the two-stage `Dockerfile`; the app service in Compose; the five-check CI workflow; `rust-toolchain.toml`. G9 G11 G12

Done when: `docker compose up` on a clean machine serves the site with content.

26.8 M6 — Whatever is actually next

Independent, and to be picked up when each one starts to matter rather than on a schedule: the authoring decision (G5), SEO and social previews (G13), the test suite (G10), a tags page, a blog, an RSS feed. This is the milestone that never finishes, which is correct for a personal site.

Chapter 27

Conventions and runbooks

27.1 Naming

Small decisions, made once, that stop the codebase from reading like four different projects.

Table 27.1. Naming conventions to adopt.

Thing	Convention	Example
table	plural, `snake_case`	`portfolio_items`
FK column	singular parent + `_id`	`portfolio_item_id`
join table	both tables, alphabetical	`portfolio_item_tags`
index	`<table>_<cols>_idx`	`tags_portfolio_item_idx`
Diesel model	singular, matches the row	`PortfolioItem`
DTO	model name + `Dto`	`PortfolioItemDto`
server fn, read	`list_*` / `get_*`	`list_portfolio`, `get_portfolio_item`
server fn, write	verb first	`create_portfolio_item`
component	`PascalCase`, and `pub`	`PortfolioCard`
migration	`create_*` / `add_*` / `fix_*`	`add_slug_to_portfolio_items`

Two specific points, because the current code does both ways:

- **No abbreviations in schema columns.** `abs` saved five characters and costs every future reader a lookup.
- **Repository names should share one shape.** Today there are three: `PublicationsRepository`, `PortfolioItemsRepository`, `JobExperienceItemRepository`. Pick `<Subject>Repository` and rename the other two.

27.2 The language decision, which is urgent

This one deserves its own section because it is about to become expensive, and because the codebase has already started answering it two ways at once.

Every identifier, comment and column in the repository is in English. The one component written so far renders Spanish:

```
view! { <p> "Aca van las publicaciones" </p> }
```

Those are two *different* decisions wearing the same clothes, and they should be separated explicitly:

The code's language.
English, and it already is — with two slips (`responsabilities`, `publication_itens`) that are the visible cost of drifting. Every dependency, every error message and every piece of documentation you will read is in English; matching it is the path of least friction.

The site's language.
Entirely your call, and it has no bearing on the first decision. A Spanish site with an English codebase is completely normal.

Decide monolingual or bilingual before writing the DTOs

This is the part that is urgent. If the site will ever be bilingual, it changes the schema and the router, not just the strings:

- the schema needs per-locale text — either a `locale` column plus a composite key on every content table, or a `<table>_translations` child table;
- the DTOs need to carry the resolved locale, or the server function needs a locale argument;
- the router needs a locale segment (`/es/portfolio`) or the decision to negotiate from the `Accept-Language` header;
- every URL slug needs a per-locale variant if you want `/es/portafolio`.

Retrofitting that after four subjects exist means touching every table, every DTO and every route. Deciding it now costs one sentence. **The recommendation is monolingual** — pick the language your readers use and write the site in it — because a bilingual personal site doubles the authoring work forever, and the authoring work is the thing that actually determines whether a personal site stays current.

27.3 Runbook: adding a new content type

The whole flow, so the next subject takes an hour rather than an afternoon of rediscovery.

Adding a content type, nine steps

1. `diesel migration generate create_talks`
2. Write `up.sql`: `id` as `GENERATED ALWAYS AS IDENTITY`, `slug TEXT NOT NULL UNIQUE`, the content columns, `position`, `created_at`/`updated_at`, every FK as `INTEGER NOT NULL REFERENCES … ON DELETE …`, an index per FK, the `CHECK` constraints.
3. Write `down.sql`: children first, `DROP TABLE IF EXISTS`.
4. `diesel migration run && diesel migration redo` — *never skip the redo.*
5. `diesel print-schema > src/schema.rs`
6. `src/talks/`: `mod.rs` with the gates, `dto.rs` (ungated), `model.rs` (`ssr`, with the `From` impl), `repository.rs` (`ssr`, `&self`, `web::block`), `api.rs` (ungated `#[server]`), `components.rs` (ungated, `pub`).
7. `pub mod talks;` in `lib.rs` — *ungated.*
8. Add the route in `app.rs`; add a nav link in `ui/layout.rs`.
9. Seed a row, then run both `cargo check` invocations and `curl` the route.

27.4 Checklist before every commit

```
cargo fmt --all
cargo clippy --no-default-features --features ssr -- -D warnings
cargo check --no-default-features --features ssr
cargo check --lib --no-default-features --features hydrate --target wasm32-unknown-unknown
```

```
# if migrations changed:
diesel migration redo -a && diesel print-schema | diff -u src/schema.rs -
git status            # is .env still untracked AND ignored?
```

27.5 Five rules that summarise the whole document

If everything else here is forgotten, these are the ones that pay for themselves.

1. **Two programs, one source tree.** Every type is in the server, the browser, or both, and you say which. Check both builds before every commit.
2. **Two structs per content type.** The row struct is Diesel's and stays on the server. The DTO is the wire's and goes everywhere. `From` converts.
3. **Never block the async runtime.** Every Diesel call goes through `web::block`, with `pool.get()` inside the closure.
4. **A migration is a reversible pair.** `diesel migration redo` after every one.
5. **A server function is a public endpoint.** Filter in the query, not in the component; authorise in the body; leak nothing in the error.

Part VIII

Reference

Chapter A

Command reference

A.1 Daily loop

```
docker compose up -d db                 # the database must be running first
cargo leptos watch                      # builds both programs, serves on :3000, reloads
cargo leptos watch -vv                  # ... and shows the sass/tailwind command lines
```

A.2 The two-build check — run both, always

```
cargo check --no-default-features --features ssr
cargo check --lib --no-default-features --features hydrate --target wasm32-unknown-unknown

# NOT this: it succeeds while checking neither program (see Chapter 7)
# cargo check
```

A.3 Migrations

```
diesel migration generate create_talks   # writes an up.sql / down.sql pair
diesel migration run                     # apply pending
diesel migration revert                  # roll back the most recent
diesel migration redo                    # revert + re-apply: THE step that proves down.sql works
diesel migration redo -a                 # ... for every migration
diesel migration list                    # what is applied
diesel database reset                    # drop, create, run all -- destroys data
diesel print-schema > src/schema.rs      # regenerate; commit with the migration
diesel print-schema | diff -u src/schema.rs -    # is schema.rs stale?
```

A.4 Database

```
psql "$DATABASE_URL"                     # a shell
psql "$DATABASE_URL" -c '\dt'            # list tables
psql "$DATABASE_URL" -c '\d+ tags'       # one table: types, defaults, indexes, constraints
psql "$DATABASE_URL" -c '\ds'            # list sequences -- how defect D1 was found
psql "$DATABASE_URL" -f seeds/dev_seed.sql
```

A.5 Release and run

```
cargo leptos build --release

# the binary needs its environment -- see Chapter 7
LEPTOS_OUTPUT_NAME=egonik-site \
LEPTOS_SITE_ROOT=target/site \
LEPTOS_ENV=PROD \
./target/release/egonik-site
```

A.6 Diagnostics

```
# what is actually in the browser bundle?
cargo tree --target wasm32-unknown-unknown --no-default-features --features hydrate --depth 1

# why is this crate in the wasm build?
cargo tree --target wasm32-unknown-unknown --no-default-features --features hydrate -i dotenvy

# did hydration happen?
curl -s -o /dev/null -w '%{http_code}\n' localhost:3000/pkg/egonik-site.js   # must be 200

# is the page really server-rendered, or an empty shell?
curl -s localhost:3000/publications | grep -c '<li'

# does a missing page really 404?
curl -s -o /dev/null -w '%{http_code}\n' localhost:3000/nonexistent

# is a file in the build at all?  (put a syntax error in it; silence means dead)
cargo check --no-default-features --features ssr
```

A.7 This document

```
cd docs && tectonic -X compile main.tex        # produces main.pdf
```

Chapter B

Glossary

SSR — server-side rendering
Running your components on the server to produce HTML. In this project it is a Cargo feature, `ssr`, not a runtime mode.

Hydration
The browser taking over server-rendered HTML: attaching event listeners and reconnecting reactivity without re-creating the DOM. Enabled by the `hydrate` feature. If it fails, the page looks correct and does nothing — Chapter 7.

CSR — client-side rendering
Shipping an empty shell and building the page entirely in the browser. The `csr` feature. Not used here, and its binary target does not currently compile.

Server function
An `async fn` marked `#[server]`. The macro compiles the body into the server only, and replaces it on the client with an HTTP call. It is a **public endpoint**, not private RPC.

DTO — data transfer object
A plain serde-serialisable struct that crosses the network. Distinct from the Diesel row struct by design — Chapter 9.

Resource
Leptos's async-data primitive for *reads*. Runs during SSR and serialises its result into the HTML so hydration does not refetch. Contrast `LocalResource`, which does not run during SSR at all.

Action
Leptos's async primitive for *writes*. Runs when dispatched, never during server rendering.

Suspense / Transition
Boundaries that render a fallback while nested resources load. `Suspense` shows the fallback on every reload; `Transition` only on the first.

Hydration mismatch
The server's markup and the client's first render disagreeing, usually from something non-deterministic in a `view!`. Produces inexplicable behaviour rather than an error.

r2d2 pool
A connection pool. `Arc`-backed, so `Clone` is cheap and `&self` suffices; never wrap it in another `Arc`.

Blocking the runtime
Calling synchronous code (all of Diesel) inside an async task, holding a worker thread that cannot then serve anyone else. Fixed with `web::block` — Chapter 10.

Composition root
The one place that knows how everything is wired: `main.rs`. Config is loaded there, the pool is built there, nothing below it reads the environment.

Migration
A reversible *pair* of SQL scripts. If `down.sql` does not work, it is not a migration — Chapter 16.

Chapter C

Sources

Claims in this document come from one of three places, and it is worth being able to tell which.

Observation.
Anything in a *Verified* box was run on this machine against this repository, this database, and the installed toolchain (rustc 1.93.1, cargo-leptos 0.3.6, PostgreSQL 17 on port 5439). Quoted output is real, occasionally trimmed for width.

Reading the source.
Behaviour of cargo-leptos 0.3.6 (the Tailwind pipeline, the hardcoded `v4.2.1`, the CSS concatenation order, the "Done" substring check) and of `leptos_actix` 0.8.7 (that `leptos_routes` registers server-function paths itself, that `extract` is `use_context::<Request>()` plus `T::extract`) was taken from the published crate sources rather than from prose documentation.

Documentation.
The rest: the Leptos book (`book.leptos.dev`), `docs.rs` for `leptos` 0.8.20, `leptos_actix` 0.8.7, `server_fn` 0.8.13, `diesel` 2.3, `actix-web` 4, `r2d2`, `tokio`; the Actix databases guide; the Tailwind v4 source-detection documentation; the Rust API Guidelines; and the official `leptos-rs/start-actix` and `start-axum-workspace` templates.

Version numbers current as of **4 August 2026**. The three that will matter when they change: `leptos` 0.8.20 (0.9 is in beta and will move `Resource` and error APIs), `cargo-leptos` 0.3.6 (its pinned Tailwind version is a build-behaviour change), and `diesel-async` 0.9.2 (still 0.x; revisit only if connection acquisition ever becomes a measured bottleneck).

> *Where this document and the code disagree, the code is right and this document is out of date. It was written against commit `befcba5` plus the uncommitted work in progress, and the working tree moved three times during the writing — which is a good sign about the project and a caution about the document.*